

**FACULTY  
OF MATHEMATICS  
AND PHYSICS**  
Charles University

**MASTER THESIS**

Bc. Martin Čorovčák

**Universal Object Mapping Assistant:  
LLM-Assisted Migration of Application  
Persistence Layers**

Department of Software Engineering

Supervisor of the master thesis: Ing. Pavel Koupil, Ph.D.

Study programme: Software and Data Engineering

Study branch: Big Data Processing

Prague 2026

I declare that I carried out this master thesis on my own, and only with the cited sources, literature and other professional sources. I understand that my work relates to the rights and obligations under the Act No. 121/2000 Sb., the Copyright Act, as amended, in particular the fact that the Charles University has the right to conclude a license agreement on the use of this work as a school work pursuant to Section 60 subsection 1 of the Copyright Act.

In ..... date ..... ..

Author's signature

I would like to express my sincere gratitude to my supervisor, Ing. Pavel Koupil, Ph.D., for his invaluable guidance, support, and encouragement throughout the development of this thesis. I would also thank the members of the department for their consultations and insights. I would like to extend my gratitude to the e-INFRA CZ and the department for their LLM services – even when I struggled sometimes, without them, this work would never be possible to attain. I am also grateful to my family and friends for their support and understanding.

Title: Universal Object Mapping Assistant: LLM-Assisted Migration of Application Persistence Layers

Author: Bc. Martin Čorovčák

Department: Department of Software Engineering

Supervisor: Ing. Pavel Koupil, Ph.D., Department of Software Engineering

Abstract: Every data-driven application rests on a persistence layer, and the polyglot-persistence era increasingly forces teams to migrate that layer across data-model paradigms—from relational to document or graph—an error-prone rewrite in which a query that still compiles can silently return the wrong data. This thesis asks whether that migration can be automated with correctness evidence grounded in *execution* rather than the surface plausibility of generated code, and whether a verification-first workflow driven by an *open, mid-size language model (LLM)* can rival the frontier-scale coding agents of the largest AI companies. We present the *Universal Object Mapping (UOM) Assistant*: a semi-deterministic workflow and a tool that translates .NET persistence layers (Entity Framework Core, NHibernate, Dapper) into Java Spring Data document and graph targets, admitting a translation only after it compiles against the real toolchain, executes on a live database, and passes a deterministic execution-equivalence check over result counts and boundary samples. Across 900 query translations over six framework pairs, 80.7% were strictly execution-equivalent to their source (96.7% document, 64.7% graph) and 85% of runs finalized without human intervention. A small head-to-head study under an identical checker suggests the open model can match the most popular state-of-the-art agent harness (Claude Code) and outperform the rest at a fraction of the cost—indicating that, where a genuine execution oracle exists, an aligned domain workflow can substitute for raw model scale.

Keywords: Large Language Model LLM Code Translation Persistence Layer Migration Object Mapping Framework ORM ODM OGM

# Contents

|                                                          |           |
|----------------------------------------------------------|-----------|
| <b>Introduction</b>                                      | <b>4</b>  |
| <b>1 Related Work</b>                                    | <b>7</b>  |
| <b>2 Problem Definition</b>                              | <b>9</b>  |
| 2.1 Goals . . . . .                                      | 9         |
| 2.2 Non-Goals . . . . .                                  | 11        |
| 2.3 Scope: Frameworks, Databases, and Versions . . . . . | 12        |
| 2.4 Running Example . . . . .                            | 13        |
| 2.4.1 Input: the source artifacts . . . . .              | 13        |
| 2.4.2 Output: the target artifacts . . . . .             | 14        |
| 2.4.3 Why even this is non-trivial . . . . .             | 17        |
| 2.5 Requirements . . . . .                               | 17        |
| <b>3 The Cost of Abstraction: ORM Overhead</b>           | <b>20</b> |
| 3.1 Why Overhead Matters for Migration . . . . .         | 20        |
| 3.2 Experimental Setup . . . . .                         | 21        |
| 3.2.1 Dataset and Queries . . . . .                      | 21        |
| 3.2.2 Isolating the ORM Overhead . . . . .               | 22        |
| 3.2.3 Measurement and Environment . . . . .              | 23        |
| 3.3 Results . . . . .                                    | 24        |
| 3.4 Implications for the Migration Assistant . . . . .   | 27        |
| <b>4 The Translation Workflow</b>                        | <b>28</b> |
| 4.1 High-Level View . . . . .                            | 28        |
| 4.1.1 A Formal Model of the Workflow . . . . .           | 31        |
| 4.1.2 The Model Portfolio . . . . .                      | 32        |
| 4.2 A Trace of a Running Example . . . . .               | 34        |
| 4.3 The Stages in Detail . . . . .                       | 37        |
| 4.3.1 Input Extraction . . . . .                         | 37        |
| 4.3.2 Schema Inspection . . . . .                        | 37        |

|          |                                                                       |           |
|----------|-----------------------------------------------------------------------|-----------|
| 4.3.3    | Translation Generation . . . . .                                      | 38        |
| 4.3.4    | Compilation and Execution . . . . .                                   | 39        |
| 4.3.5    | Equivalence Checking . . . . .                                        | 40        |
| 4.3.6    | Evaluation: LLM-as-a-Judge . . . . .                                  | 40        |
| 4.3.7    | Human Intervention . . . . .                                          | 41        |
| 4.3.8    | Finalization . . . . .                                                | 41        |
| <b>5</b> | <b>Design Decisions</b>                                               | <b>42</b> |
| 5.1      | A Free ReAct Agent versus an Orchestrated Graph . . . . .             | 42        |
| 5.2      | Typed Shared State and Stateless Nodes . . . . .                      | 43        |
| 5.3      | Keeping the Reasoning Context Clean . . . . .                         | 44        |
| 5.4      | Prompts as Stage Interfaces . . . . .                                 | 45        |
| 5.5      | Structured Output Shaped to the Request . . . . .                     | 45        |
| 5.6      | The Verification Layers . . . . .                                     | 46        |
| 5.6.1    | Compilation and Execution in Sandboxes . . . . .                      | 47        |
| 5.7      | Semantic Equivalence . . . . .                                        | 48        |
| 5.7.1    | The execution and serialization contracts . . . . .                   | 48        |
| 5.7.2    | Background: structural diffing and the deep distance metric . . . . . | 49        |
| 5.7.3    | Tolerant comparison and the swapped-sorting check . . . . .           | 51        |
| 5.7.4    | A worked example . . . . .                                            | 52        |
| 5.8      | Bounded Loops, Retries, and Fallbacks . . . . .                       | 54        |
| <b>6</b> | <b>Input and User Interaction</b>                                     | <b>56</b> |
| 6.1      | The Workspace . . . . .                                               | 56        |
| 6.2      | Anatomy of a Request . . . . .                                        | 59        |
| 6.3      | Ready-Made Pairs and Custom Configuration . . . . .                   | 60        |
| 6.4      | Following a Run . . . . .                                             | 60        |
| 6.5      | The Human-in-the-Loop Hand-off . . . . .                              | 62        |
| 6.6      | IDE Integration . . . . .                                             | 62        |
| 6.7      | MCP and ACP Protocol Interfaces . . . . .                             | 63        |
| <b>7</b> | <b>Architecture and Implementation</b>                                | <b>64</b> |
| 7.1      | System Context . . . . .                                              | 64        |
| 7.2      | Container View . . . . .                                              | 65        |
| 7.3      | The Orchestrator . . . . .                                            | 67        |
| 7.4      | The Frontend . . . . .                                                | 69        |
| 7.5      | Observability . . . . .                                               | 70        |
| 7.6      | Deployment . . . . .                                                  | 70        |
| 7.7      | Extensibility: Adding a Framework Is Mechanical . . . . .             | 71        |

|          |                                                                    |            |
|----------|--------------------------------------------------------------------|------------|
| <b>8</b> | <b>Experimental Evaluation</b>                                     | <b>73</b>  |
| 8.1      | Experimental Design . . . . .                                      | 73         |
| 8.2      | Source Inputs and Target Stores . . . . .                          | 74         |
| 8.3      | Metrics . . . . .                                                  | 76         |
| 8.4      | Baseline: the Initial Workflow Design . . . . .                    | 81         |
| 8.5      | The Redesigned Generation Stage . . . . .                          | 81         |
| 8.6      | Headline Results . . . . .                                         | 83         |
| 8.7      | Resource Profile . . . . .                                         | 87         |
| 8.8      | Do the Judges Track the Ground Truth? . . . . .                    | 88         |
| 8.9      | Head-to-Head: the Workflow versus General-Purpose Agents . . . . . | 88         |
| <b>9</b> | <b>Discussion</b>                                                  | <b>93</b>  |
| 9.1      | Issues Encountered and Lessons Learned . . . . .                   | 93         |
| 9.2      | Limitations . . . . .                                              | 94         |
| 9.3      | Threats to Validity . . . . .                                      | 95         |
| 9.4      | Future Work . . . . .                                              | 97         |
|          | <b>Conclusion</b>                                                  | <b>99</b>  |
|          | <b>Declaration of AI Usage</b>                                     | <b>101</b> |
|          | <b>Bibliography</b>                                                | <b>102</b> |
|          | <b>Acronyms</b>                                                    | <b>113</b> |
| <b>A</b> | <b>Electronic Attachments</b>                                      | <b>115</b> |
| A.1      | Complete Execution Traces . . . . .                                | 115        |
| A.2      | Experimental Deliverables Bundle . . . . .                         | 116        |
| <b>B</b> | <b>Supplementary Material from the Implementation</b>              | <b>117</b> |
| B.1      | Structured-Output and Tool Schemas . . . . .                       | 117        |
| B.2      | Per-Node System Prompts . . . . .                                  | 121        |
| B.3      | Contextual Validation Harness Snippets . . . . .                   | 126        |
| B.4      | A Complete Equivalence-Check Example . . . . .                     | 128        |
| B.5      | A Complete Translation Input . . . . .                             | 131        |
| B.6      | Sandbox Configuration Files . . . . .                              | 136        |

# Introduction

Every data-driven application rests on a *persistence layer*: the boundary where in-memory objects meet a database. Object-mapping frameworks make that boundary ergonomic. An object-relational, object-document, or object-graph mapper lets a developer manipulate persistent state as ordinary objects while the framework compiles those manipulations into queries against the store [1]. For a long time the assumption behind the boundary was that one database – almost always relational – would serve the whole application.

That assumption no longer holds. In the era of *polyglot persistence*, teams routinely move a persistence layer from one data-model paradigm to another – relational to document, or to graph – to fit an evolving access pattern or a different stack [2, 3]. To make it harder, a foreign-keyed table has no single representation in a document store or a graph [4], so the translator must decide how to represent every association, rewrite every query into a foreign paradigm, and preserve the logical behavior of code spread across dozens of entities and hundreds of queries. Done by hand it is slow, costly, and dangerous: a query that still compiles can silently return the wrong data.

The obvious move, in a year when large language models (LLMs) write fluent code on demand, is to hand the whole job to a state-of-the-art coding agent [5, 6, 7]. These systems produce idiomatic output at remarkable speed. But here *plausible is not the same as correct*. A translated query that reads well, type-checks, and runs without error can still return a subtly different result set, and nothing on the surface of the code reveals it. General-purpose agents also improvise a random course of action on every run, so their output is neither reproducible nor auditable [8] – exactly the properties an engineer migrating production data needs most.

This thesis grounds correctness in *execution*. Rather than hand the task to a single autonomous agent, we constrain a language model inside a semi-deterministic *workflow* that fixes the order of generation, compilation, execution, comparison, and judgment in the structure of the system itself, and grants the model autonomy only *inside* individual stages. Every candidate translation is compiled against the real target toolchain, executed against a live database holding logically equivalent data, and checked for equivalence against the source’s own results before a human sees it. The decisive evidence is produced by deterministic tools – real compilers, real databases, a structural



equivalence check – and a language model is trusted only where its judgment cannot silently corrupt a result. The equivalence check itself is deliberately economical – it compares result-set counts and boundary samples, trading exhaustiveness for validation fast enough to run inside every repair loop – so what it establishes is strong execution-grounded *evidence* of correctness rather than a formal guarantee; strengthening this witness is discussed as future work.

Three questions organize the work.

- Q1. Can an application’s persistence layer be migrated *across fundamentally different data-model paradigms* – relational to document and relational to graph – automatically, with correctness evidence that rests on *execution* rather than on the surface plausibility of the generated code?
- Q2. Is a language model more *reliable* – more often correct, more robust to its own failures, and needing less human intervention – inside a semi-deterministic, verification-first workflow than when the whole task is delegated to an autonomous coding agent?
- Q3. Must one reach for the largest *frontier* model, or can a purpose-built workflow driven by an *open, mid-size* model match the flagship agentic products of the largest AI companies – and at what cost?

To answer them, the thesis designs, builds, and evaluates the *Universal Object Mapping (UOM) Assistant*. It translates the persistence-layer artifacts – entity classes, mapping configuration, and queries – of three .NET source frameworks (Entity Framework Core with [LINQ](#), NHibernate with [HQL](#), and Dapper over raw [SQL](#)) into two non-relational Java Spring Data targets spanning the document and graph paradigms. The workflow is given a precise formal treatment as a state machine, each stage carries a measured *autonomy level*, and the order of verification cannot be skipped or rearranged by any model. The assistant is exposed both as an interactive web application and through programmatic [MCP](#) and [ACP](#) interfaces. It is architected so that adding a framework is a mechanical extension rather than a redesign, and it comes with a characterization of the framework-attributable overhead of the .NET sources that a performance-aware target selector would need.

The rest of the thesis is structured the following way. Chapter [1](#) situates the work against object-mapping and multi-model data management, language-model code generation, agentic workflows, execution-grounded self-repair, and code-evaluation methodology. Chapter [2](#) states the goals, non-goals, scope, and testable requirements, and introduces the running example. Chapter [3](#) characterizes the framework-attributable overhead of the .NET [ORM](#) sources. Chapter [4](#) formalizes the translation workflow stage by stage, and chapter [5](#) explains the design decisions behind it – above all the

choice of a semi-deterministic graph over an autonomous agent, and the execution-grounded verification at its center. Chapter 6 describes the interactive, observable UI of the assistant, and chapter 7 the architecture and its extensibility. Chapter 8 reports the experimental evaluation against the requirements, and chapter 9 discusses the threats to validity, the lessons learned, and the future work.

# Chapter 1

## Related Work

Cross-paradigm data mapping was formalized by the category-theoretic *MM-cat* line, which represents relational, document, and graph schemas in one model and transforms between them [9], later extended to schema evolution [3] and other tools.

The closest predecessor is the *ORMorpher* framework for translating and optimizing *ORM* code [10], built on a rule-based engine that adapts queries across .NET frameworks [11] and an integer-linear-programming advisor that selects a target framework under performance constraints [12]. It shares this thesis’s goal but relies on hand-authored rules, so correctness rests on rule coverage rather than observed behavior.

Brown et al. [13] showed large models to be effective few-shot learners, a capability used for programming by Codex and its *pass@k* metric [14]. TransCoder demonstrated unsupervised program translation across C++, Java, and Python [15]. Spracklen et al. [16] quantify that models routinely emit calls to non-existent *APIs*, which is why this thesis pins framework versions and compiles every candidate. ExeCoder grounds translation in whether the output runs [17]; this thesis extends that from “does it run” to “does it return the same data” across a paradigm boundary. Earlier natural-language-to-*SQL* work such as ATHENA++ [18] targets query authoring rather than migration between executable dialects.

For multi-step tasks the literature offers chain-of-thought prompting [19], its search-based generalization Tree of Thoughts [20], and the ReAct paradigm interleaving reasoning with tool use [21]. Anthropic [22] distinguish a *workflow* that follows predefined code paths from an *agent* that directs its own control flow, and StateFlow casts *LLM* task-solving as a state machine, reporting better success and cost than free-form agency [23]; the assistant follows that design on the LangGraph runtime [24]. Constraining autonomy is further motivated by the “lost in the middle” effect [25], which argues for clean per-stage context and on-demand knowledge injection, in the spirit of retrieval-augmented generation [26] and context engineering [27].

Evidence on self-correction is mixed. Self-debugging helps when feedback is informative [28], yet Olausson et al. [29] find self-repair yields only modest gains that

depend on feedback quality. Lin et al. [30] analyze the cost-effectiveness of executing candidate code during LLM-based repair and find that its benefit hinges on the executed checks being aligned with the final acceptance oracle and on using them sparingly – a finding this thesis’s redesign applies directly. Where behavior cannot be checked mechanically, the assistant defers to an LLM judge, whose biases Zheng et al. [31] characterize and which we treat as unreliable and score against ground truth.

On measuring generated code, CodeBLEU combines n-gram, syntax-tree, and data-flow overlap with a reference [32], but rewards resemblance over behavior, so we use it only as a stability proxy. Behavioral evaluation follows the *pass@k*-against-an-oracle tradition [14], which we extend from query unit-test pass/fail to cross-data-store result equivalence over a structural diff [33]. Benchmark contamination motivated *LiveBench* [34], a continuously refreshed LLM leaderboard, which we use to select the model best suited for each stage, while mutation analysis [35] could measure the verification layers’ discriminative power and is flagged as future work.

Each tradition leaves a gap: multi-model work formalizes cross-paradigm mapping but generates no runnable code; the ORMorpher lineage generates it by rules without executing it; code-translation models produce runnable output but rarely cross a data-model boundary; and the evaluation literature warns that surface metrics and contaminated benchmarks cannot replace an execution oracle. No prior system, to our knowledge, combines cross-paradigm persistence-layer translation, driven by a language model and verified by execution equivalence against a live database, orchestrated as a semi-deterministic, observable workflow. Designing, implementing, and testing that combination is the contribution of this thesis.

# Chapter 2

## Problem Definition

Modern applications reach persistent data through an *object-mapping framework* that maps in-memory objects onto an underlying store and exposes a query interface over it. Depending on the data model, such a framework is called an [Object-Relational Mapping \(ORM\)](#) for relational databases [1], an [Object-Document Mapping \(ODM\)](#) for document stores, or an [Object-Graph Mapping \(OGM\)](#) for graph databases [4]. The framework, together with its entity definitions, mapping configuration, queries, and surrounding persistence code, constitutes the application’s *persistence layer*. Migrating that layer – to fit an evolving access pattern, consolidate on a different framework, or target another language ecosystem – is not a mechanical rename: it requires transforming entities, mapping configurations, queries, and framework-specific code while preserving externally observable behavior, and because source and target frameworks differ in data models, mapping strategies, query interfaces, and supported features, these transformations are intricate and error-prone – the cross-paradigm form of the classical object-relational impedance mismatch [36].

This chapter states what the [Large Language Model \(LLM\)](#)-based assistant is meant to achieve: section 2.1 fixes the goals, section 2.2 delimits what is out of scope, section 2.3 pins down the concrete frameworks and databases, section 2.4 introduces the running example used throughout the thesis, and section 2.5 distills the goals into testable requirements that the evaluation (chapter 8) revisits.

### 2.1 Goals

The overarching goal is an assistant that, given the database-layer artifacts of an application written against a *source* framework, produces the equivalent artifacts for a chosen *target* framework with *execution-grounded evidence of correctness* rather than mere syntactic plausibility. Concretely, the thesis pursues the following goals.

**G1 – Translation across paradigms.** Translate the persistence-layer artifacts of a

source [ORM](#) into the equivalent artifacts of a target [ODM](#) or [OGM](#), crossing the relational/document and relational/graph boundaries. The unit of translation is twofold: *schema* (entity classes and their mapping configuration) and *queries* (the framework-specific query methods), independently or together.

- G2 – Correctness by Execution.** A translation is surfaced to the user only after it has been compiled with the real target toolchain ([Command Line Interface \(CLI\)](#) tools and language-specific libraries), executed against a live database, and checked for logical equivalence<sup>1</sup> with the source’s results (section 5.6) [17]. The check compares result-set counts and boundary samples – an economical witness chosen so that validation stays fast enough to run inside every repair loop; it yields strong evidence of equivalence rather than proof, a boundary stated precisely in section 9.2.
- G3 – Generality.** Cover a space wide enough to show that the approach is not tied to a single pair of frameworks: three structurally different source query dialects ([LINQ](#), raw [SQL](#), and [HQL](#)) and two distinct non-relational paradigms (document and graph). Adding a framework should be a mechanical extension of the engine, not a redesign of the approach (section 7.7).
- G4 – Usability and Observability.** Deliver the assistant as an interactive [User Interface \(UI\)](#), in which a user submits source artifacts, watches the translation proceed step by step, inspects every intermediate result, and intervenes when necessary. Every run is fully traced and reproducible.
- G5 – Experimental Evaluation.** Demonstrate that the assistant meets the above goals through a systematic evaluation (chapter 8) that measures correctness, robustness, and the amount of human intervention required through quantitative and statistical metrics.
- G6 – Empirical Grounding of the Source Frameworks.** Quantify the raw, time and memory overhead of the .NET source frameworks ([EF Core](#), [NHibernate](#), [Dapper](#)) with database execution removed, establishing a cost baseline for future performance-aware target selection (chapter 3). This is a characterization, not an optimization (cf. non-goal N2).

The theoretical background and experimental validation are the subject of this thesis; the implementation was first delivered as a research project by the author

---

<sup>1</sup>Logically equivalent queries return the same result sets – the same primitive values with comparable data types across the underlying stores – though possibly in different formats (e.g. C# vs. Java entity instances).

within the *Adaptive Data Management (ADaM) research group*, building on the rule-based *ORMorpher* line of work [10, 11]. Chapter 1 positions the thesis against that background.

## 2.2 Non-Goals

Equally important is what the assistant is *not* expected to do; stating non-goals keeps the contribution well-defined.

**N1 – Not a data-migration (ETL) tool.** The assistant translates *code* (schema and queries), not the data itself. Populating logically equivalent target databases from the relational source is a separate, one-time, semi-automatic step performed with established *Extract, Transform, Load (ETL)* tooling (MongoDB Relational Migrator [37] and the Neo4j ETL Tool [38]); it is a precondition for validation, not part of the translation pipeline (section 7.1).

**N2 – Not a performance optimizer.** The assistant aims at *correct*, idiomatic translations, not at selecting the fastest target framework or tuning the generated queries. Performance-aware framework *selection* is the province of the complementary *ILP*-based advisor in the *ORMorpher* line [12] and is left to future work (section 9.4).

**N3 – Not a general-purpose code translator.** The scope is the persistence layer. Business logic, presentation code, and application wiring outside the data-access layer are not translated.

**N4 – Not a general-purpose assistant.** The assistant is a semi-deterministic graph, not an open prompt. Unlike general-purpose coding assistants (Claude Code<sup>2</sup>, GitHub Copilot<sup>3</sup>) or chatbots (ChatGPT<sup>4</sup>, Gemini<sup>5</sup>), the initial prompt must follow a specific format, and the user can intervene only at the end of a constant number of self-corrected attempts.

**N5 – Bounded automation, not full autonomy.** When automated repair fails within a bounded number of attempts, the system deliberately pauses and hands control to a human rather than retrying indefinitely or guessing. A fully autonomous, open-ended agent is not a goal of this thesis (the rationale is given in section 5.1).

---

<sup>2</sup><https://code.claude.com/docs/en/overview>

<sup>3</sup><https://docs.github.com/en/copilot>

<sup>4</sup><https://chatgpt.com/>

<sup>5</sup><https://gemini.google.com/>

**N6 – A fixed, curated set of frameworks and versions.** The assistant targets specific, pinned framework versions (section 2.3) rather than arbitrary or legacy ones. This is what makes grounded, compile-against-the-real-API generation possible.

## 2.3 Scope: Frameworks, Databases, and Versions

The assistant translates from three .NET relational source frameworks into two Java Spring Data non-relational targets (part of the Spring Boot<sup>6</sup> ecosystem). The source side covers the three dominant .NET access styles [11]: a full ORM with LINQ<sup>7</sup> (Entity Framework Core), a mature ORM with its own query language (NHibernate, LINQ<sup>8</sup> or HQL<sup>9</sup>), and a thin micro-ORM over raw SQL [39] (Dapper). The target side covers the two principal non-relational paradigms: document (Spring Data MongoDB) and graph (Spring Data Neo4j). Table 2.1 summarizes the frameworks, data models, query interfaces, and pinned versions.

| Role   | Framework             | Paradigm               | Query interface                                        | Version |
|--------|-----------------------|------------------------|--------------------------------------------------------|---------|
| Source | Entity Framework Core | ORM (relational)       | LINQ                                                   | 10      |
| Source | NHibernate            | ORM (relational)       | LINQ / HQL                                             | 5.5     |
| Source | Dapper                | micro-ORM (relational) | raw SQL                                                | 2.1     |
| Target | Spring Data MongoDB   | ODM (document)         | Query/Criteria <sup>a</sup> , Aggregation <sup>b</sup> | 5.0     |
| Target | Spring Data Neo4j     | OGM (graph)            | Cypher-DSL <sup>c</sup>                                | 8.1     |

**Table 2.1** Supported frameworks, data models, query interfaces, and pinned versions. The source frameworks run on .NET 10; the target frameworks on Java 25 / Spring Boot 4.0.

<sup>a</sup><https://docs.spring.io/spring-data/mongodb/reference/5.0/mongodb/template-query-operations.html>

<sup>b</sup><https://docs.spring.io/spring-data/mongodb/reference/5.0/mongodb/aggregation-framework.html>

<sup>c</sup><https://neo4j.github.io/cypher-dsl/2025.3.0/>

The relational source is a Microsoft SQL Server 2022 instance pre-loaded with the *WideWorldImporters* sample database [40]; the document and graph targets are MongoDB 8.2.6<sup>10</sup> and Neo4j 2026.02.2<sup>11</sup>, populated with logically equivalent data through

<sup>6</sup><https://spring.io/projects/spring-boot>

<sup>7</sup><https://learn.microsoft.com/en-us/dotnet/csharp/linq/>

<sup>8</sup><https://nhibernate.info/doc/nhibernate-reference/querylinq.html>

<sup>9</sup><https://nhibernate.info/doc/nhibernate-reference/queryhql.html>

<sup>10</sup><https://www.mongodb.com/docs/v8.2/>

<sup>11</sup><https://neo4j.com/docs/>



the one-time [ETL](#) step (N1), which uses heuristics based on primary/foreign-key relationships, attribute counts, and more (documented by the respective tools). Fixing the versions is a deliberate design choice: because the exact target [API](#) is known, the assistant can generate code against the real dependency manifest (a pinned `.csproj` or `pom.xml`) instead of relying on the language model’s recollection of an [API](#) surface that changes between releases (section [5.4](#)).

## 2.4 Running Example

To make the problem concrete – and to provide a thread that recurs through the workflow (chapter [4](#)) and design (chapter [5](#)) chapters – we introduce a single small example: one entity and one query, migrated from Entity Framework Core to Spring Data MongoDB. Even at this minimal size the migration already requires genuine modelling decisions.

### 2.4.1 Input: the source artifacts

Listing [2.1](#) shows the source *schema*: a Customer entity in Entity Framework Core, mapped to a relational table and holding a one-to-many association to its orders by foreign key. Listing [2.2](#) shows the source *query*, expressed in [LINQ](#): all customers that have at least one order placed after a cut-off date.

```
1 [Table("Customers", Schema = "Sales")]
2 public class Customer
3 {
4     [Key]
5     public int CustomerID { get; set; }
6     public required string CustomerName { get; set; }
7     public ICollection<Order> Orders { get; set; }
8 }
```

**Listing 2.1** Source schema – an Entity Framework Core entity ([LINQ](#) source dialect).

```
1 // Customers with at least one order placed after a cut-off date.
2 db.Customers
3 .Where(c => c.Orders.Any(o => o.Date > cutoff));
```

**Listing 2.2** Source query – a [LINQ](#) expression over the entity of listing [2.1](#).

## 2.4.2 Output: the target artifacts

Crucially, there is no single “correct” target: each framework offers more than one way to model the relationship and express the query, each with different annotations, a different query idiom, and different performance and memory characteristics,<sup>12</sup> while all must return the same customers. We give three valid translations: two in Spring Data MongoDB (orders *referenced* in a separate collection, or *embedded* in the customer document) and one in Spring Data Neo4j (orders reached over a graph *relationship*). Choosing among them is exactly the modelling decision the assistant – or its user – must make.

**Option A: MongoDB with referenced documents.** Listing 2.3 keeps orders in their own collection and links them with the `@DocumentReference` annotation;<sup>13</sup> each column, including the source primary key, maps to a `@Field`, and the unique generated key `id` is mapped separately. Because the orders live in a separate collection, the `LINQ` . Any over a navigation property becomes a MongoDB *aggregation pipeline* (listing 2.4) that `$lookup`-joins<sup>14</sup> the orders collection and then matches on the date.

```
1 @Document("customers")
2 class Customer {
3     @Id String id;
4     @Field("customerId") Integer customerId;
5     @Field("customerName") String customerName;
6     @DocumentReference List<Order> orders;    // joined from a
7     ↪ separate collection
8 }
```

**Listing 2.3** Target schema, option A – Spring Data MongoDB with the orders held as a `@DocumentReference` (inter-document, normalized).

<sup>12</sup>The embed-versus-reference trade-off and its read/write and storage implications are discussed in MongoDB’s data-modeling guidance: <https://www.mongodb.com/docs/manual/data-modeling/>.

<sup>13</sup>Spring Data MongoDB document references: <https://docs.spring.io/spring-data/mongodb/reference/mongodb/mapping/document-references.html>.

<sup>14</sup>MongoDB `$lookup` aggregation stage: <https://www.mongodb.com/docs/manual/reference/operator/aggregation/lookup/>.

```

1 // Referenced variant: $lookup-join the orders collection, then
  ↳ match on date.
2 var agg = Aggregation.newAggregation(
3     Aggregation.lookup("orders", "customerId", "customerId",
4         ↳ "orders"),
5     Aggregation.match(Criteria.where("orders.date").gt(cutoff))
6 );
7 return mongoTemplate
8     .aggregate(agg, "customers", Customer.class)
  .getMappedResults();

```

**Listing 2.4** Target query, option A – the LINQ expression of listing 2.2 as a MongoDB aggregation pipeline over referenced documents.

**Option B: MongoDB with embedded documents.** Alternatively, the orders can be *embedded* as a nested array inside each customer document (listing 2.5). The field is a plain `List<Order>` with no `@DocumentReference`, so no second collection and no join exist. The query (listing 2.6) is correspondingly simpler: a single find with an `$elemMatch` criterion<sup>15</sup> that asks whether the embedded orders array contains an element past the cut-off – the direct counterpart of the LINQ `.Any`.

```

1 @Document("customers")
2 class Customer {
3     @Id String id;
4     @Field("customerId") Integer customerId;
5     @Field("customerName") String customerName;
6     List<Order> orders; // embedded sub-documents
7     ↳ (intra-document, denormalized)
8 }

```

**Listing 2.5** Target schema, option B – Spring Data MongoDB with the orders *embedded* as a nested array (intra-document).

<sup>15</sup>MongoDB `$elemMatch` query operator: <https://www.mongodb.com/docs/manual/reference/operator/query/elemMatch/>; exposed in Spring Data as `Criteria.elemMatch`.

```

1 // Embedded variant: no $lookup -- match the nested orders array
  ↳ directly.
2 Query query = Query.query(
3     Criteria.where("orders").elemMatch(Criteria.where("date").gt(
4         ↳ cutoff)));
return mongoTemplate.find(query, Customer.class);

```

**Listing 2.6** Target query, option B – the same LINQ expression as a single find with an \$elemMatch over the embedded array.

**Option C: Neo4j as a graph.** In Spring Data Neo4j the entities become *nodes* and the association becomes a typed *relationship*. Listing 2.7 maps Customer with @Node and connects it to its orders with @Relationship(type = "PLACED").<sup>16</sup> The query (listing 2.8) is a graph *traversal*: a Cypher-DSL<sup>17</sup> statement matches the (:Customer)-[:PLACED]->(:Order) path, filters on the order date, and returns the distinct customers.

```

1 @Node("Customer")
2 class Customer {
3     @Id @GeneratedValue String id;
4     @Property("customerId") Integer customerId;
5     @Property("customerName") String customerName;
6     @Relationship(type = "PLACED", direction =
7         ↳ Relationship.Direction.OUTGOING)
8     List<Order> orders;
}

```

**Listing 2.7** Target schema, option C – Spring Data Neo4j with the orders reached over a PLACED relationship.

<sup>16</sup>Spring Data Neo4j relationship mapping: <https://docs.spring.io/spring-data/neo4j/reference/object-mapping/metadata-based-mapping.html>.

<sup>17</sup>The Neo4j Cypher-DSL builds Cypher statements type-safely: <https://neo4j.github.io/cypher-dsl/>.

```

1 // Graph variant: traverse the PLACED relationship and filter on
  ↳ the order date.
2 var c = Cypher.node("Customer").named("c");
3 var o = Cypher.node("Order").named("o");
4 Statement statement = Cypher.match(c.relationshipTo(o, "PLACED"))
5     .where(o.property("date").gt(Cypher.parameter("cutoff")))
6     .returningDistinct(c)
7     .build();
8 return neo4jTemplate.findAll(statement, Customer.class);

```

**Listing 2.8** Target query, option C – the same LINQ expression as a Cypher-DSL relationship traversal in Spring Data Neo4j.

### 2.4.3 Why even this is non-trivial

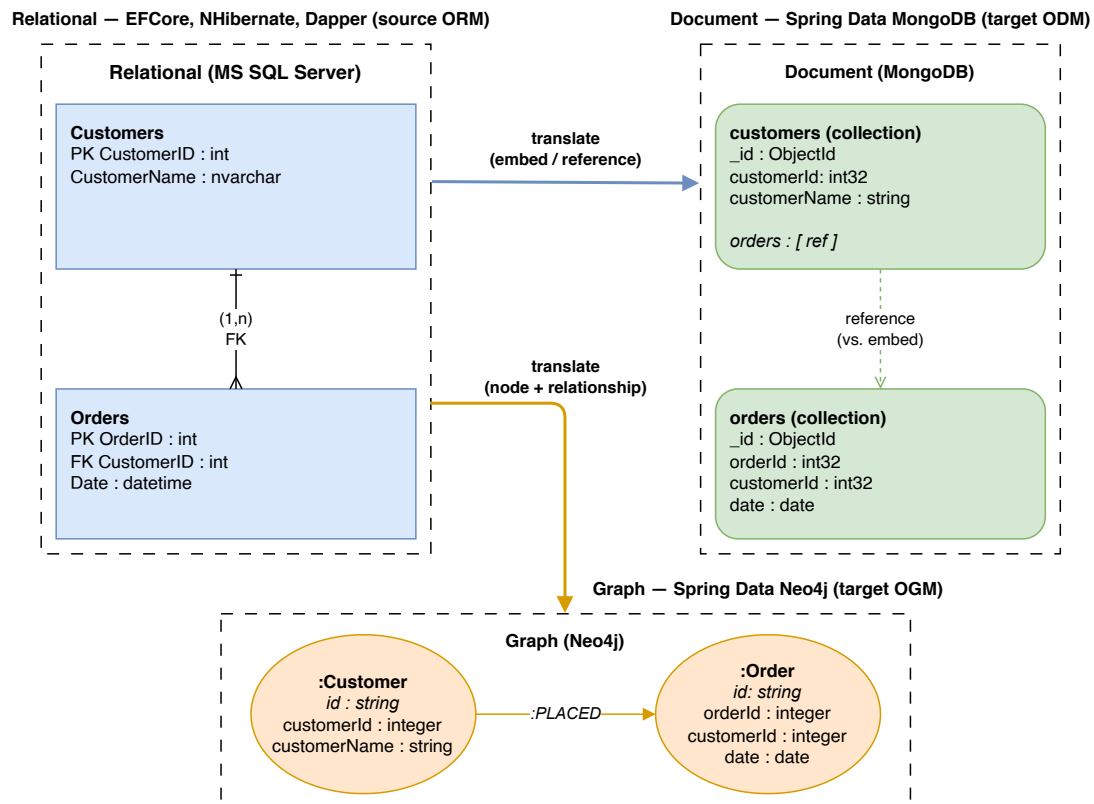
Figure 2.1 makes the data-model mismatch concrete: the same Customers/Orders foreign-key association has no one-to-one image in either target paradigm – a foreign-keyed table is neither a document nor a graph, so the translator must *decide* how to represent the association. Figure 2.2 shows the same pattern using the ORM/ODM/OGM framework abstraction. This example exhibits, in miniature, every difficulty the assistant must handle.

Types must be remapped; target may have a technical identifier, but the source primary key must be preserved as a comparable business key. The query must be rewritten into a different idiom – the same relational navigation becomes a document aggregation, an embedded-array `$elemMatch`, or a graph traversal depending on the chosen model – while returning logically the same result set, so code that merely compiles is not enough. A realistic application multiplies this by dozens of entities and hundreds of queries, each carrying a small decision that can silently break correctness. This is precisely the slow, manual, risky work the assistant is meant to automate *and* verify.

## 2.5 Requirements

The goals translate into four properties a translation must satisfy – the criteria against which the approach is evaluated in chapter 8.

**R1 – Transformation correctness.** The translated schema and query must be syntactically valid and must compile against the pinned target toolchain. Non-compiling output is rejected before it can reach the user.

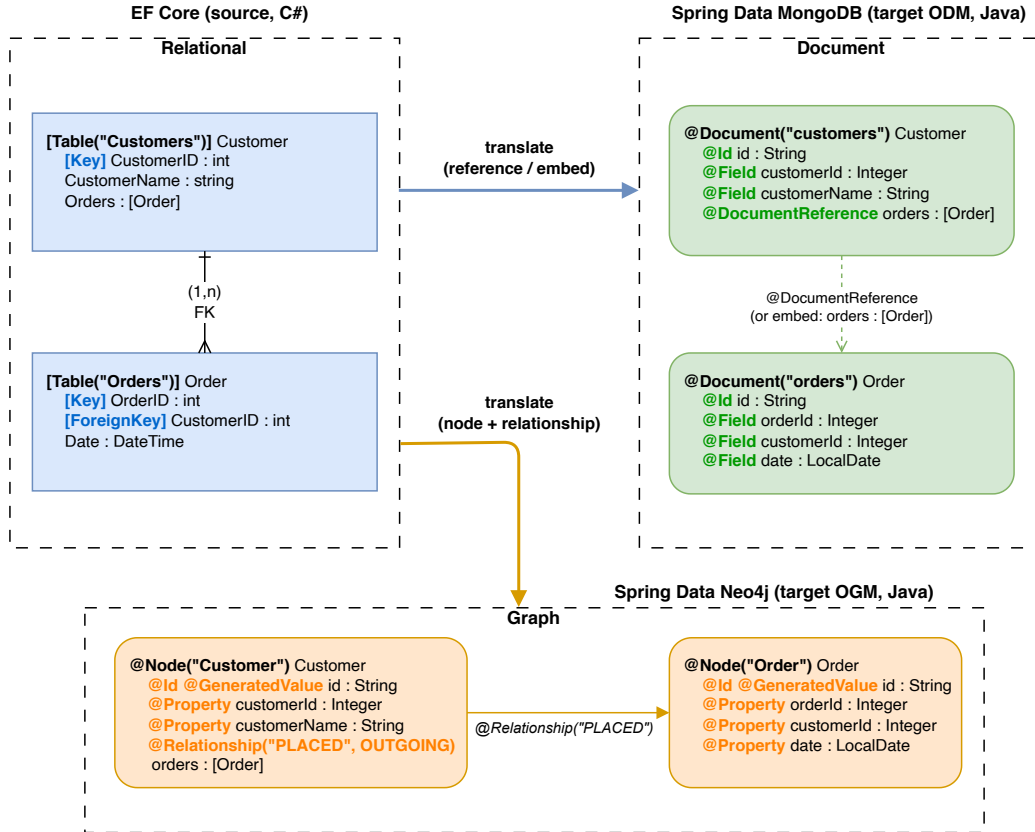


**Figure 2.1** The same relational association translated into the two target models. The document model must choose between embedding and referencing the orders; the graph model turns the entities into nodes joined by a typed relationship.

**R2 – Semantic preservation.** When executed against a live target database holding logically equivalent data, the translated query must return logically the same data as the source query, up to differences that are provably harmless (e.g. field ordering, floating-point precision, or a deterministic reversal of sort order). Equivalence is established by structural comparison of the execution results, not by inspection of the code (section 5.6).

**R3 – Robustness.** The system must recover from the typical failure modes of language-model code generation – hallucinated APIs, type mismatches, malformed structured output – through bounded, automated self-repair, and must degrade gracefully (a controlled hand-off to a human) when repair does not converge.

**R4 – Minimal manual intervention.** A correct translation should require as little human effort as possible. The amount of required intervention – measured as the fraction of runs that reach the human-handoff state – is a key metric in the



**Figure 2.2** The same relational association as figure 2.1 translated into the two target models, but using the ORM/ODM/OGM framework abstraction.

evaluation.

Requirements **R1** and **R2** are objective, machine-checkable correctness conditions; **R3** and **R4** concern the assistant's behavior when the model's first attempt is wrong. The workflow (chapter 4), its design decisions (chapter 5), and the validation machinery (section 5.6) together make them achievable.

## Chapter 3

# The Cost of Abstraction: ORM Overhead

The assistant treats the source frameworks (section 2.3) as interchangeable origins of a translation, but they are not interchangeable on the *performance* side: the abstraction that lets an [ORM](#) map rows to objects imposes a per-query cost in time and memory that differs by more than an order of magnitude across Entity Framework Core, NHibernate, and Dapper.

This chapter is a case study that measures that cost directly, to establish a premise the rest of the thesis relies on: [ORM](#), [ODM](#), and [OGM](#) frameworks carry genuinely different per-query overheads, so migrating between them is a workload-driven decision. From an application’s point of view, the assistant therefore supports migration between a mapping framework and a database *together*, not a database engine alone. Where Abrahám’s ORMorpher thesis [11] benchmarked seven .NET frameworks executing complete queries against a live database, this thesis isolates the runtime that belongs to the framework itself, with the database removed from the measurement – the former answers *how fast is the whole stack*, the latter *how much does the mapping layer add*. Section 3.1 explains why the second question is the relevant one for a migration assistant, section 3.2 describes the experiment, and sections 3.3 and 3.4 present and interpret the results.

### 3.1 Why Overhead Matters for Migration

The assistant is deliberately *not* a performance optimizer (non-goal N2, section 2.2); its job is correct, idiomatic translation. Performance nonetheless frames the work in three ways that justify a dedicated measurement.

First, it establishes a *baseline*. The supported source frameworks span the abstraction spectrum: Dapper is a thin micro-[ORM](#) that materialises rows with almost no



machinery,<sup>1</sup> NHibernate is a full **ORM** whose `ISession` maintains a first-level cache of materialised objects,<sup>2</sup> and **EF Core** compiles each **LINQ** expression tree into a provider query before executing it.<sup>3</sup> Quantifying each one’s cost at rest tells a developer what they pay for the abstraction they chose, and what they stand to gain or lose by migrating.

Second, it scopes future work. The complementary **ORMorpher** advisor performs performance-aware framework *selection* via **Integer Linear Programming (ILP)** [12], and section 9.4 discusses extending the assistant to suggest a target on cost grounds. Any such recommendation needs a per-query cost model, and the overhead measured here is exactly its framework-attributable term – the one a cost-based optimizer can influence, since database execution time is fixed by the query and data.

Third, it motivates the breadth of the source side: where the same logical query can be written as hand-written **SQL** or routed through a heavyweight **ORM**, knowing the overhead of each option makes a migration an informed choice rather than a lateral move. Taken together, a developer who only reasons about which database to move to, and not which mapping layer might sit in front of it, is missing a term that this chapter shows can dominate the query’s cost.

## 3.2 Experimental Setup

The experiment has three ingredients, described in turn: the dataset and query suite, reused unmodified from Abrahám’s benchmark so the two studies stay comparable (section 3.2.1); the interception technique that removes the database from the timing loop (section 3.2.2); and the measurement harness and hardware (section 3.2.3).

### 3.2.1 Dataset and Queries

To keep the two experiments comparable, this study reuses Abrahám’s dataset and query suite unmodified [11]: Microsoft’s *Wide World Importers* sample database [40], roughly thirty tables across the Application, Purchasing, Sales, and Warehouse domains on **SQL Server**. Its schema and the exact **SQL** for every query are documented in Abrahám’s Sections 1.4.1–1.4.2.

The suite contains 23 read-only (**SELECT**) queries in eight categories, each exercising a distinct mapping concern (table 3.1), from the trivial (a single row mapped to an entity) to the structurally demanding (multi-table relationships, set operations) – framework overhead being a function of how much work the mapping layer must do, not a single number.

---

<sup>1</sup><https://github.com/DapperLib/Dapper>

<sup>2</sup><https://nhibernate.info/doc/nhibernate-reference/architecture.html>

<sup>3</sup><https://learn.microsoft.com/ef/core/querying/how-query-works>

| Group | Queries | Mapping concern exercised                                                                  |
|-------|---------|--------------------------------------------------------------------------------------------|
| A     | A1–A4   | Entity projection: row-to-entity, column subset, split projection, stored-procedure result |
| B     | B1–B6   | Selection: indexed/non-indexed filters, range, IN-list, text search, paging                |
| C     | C1–C3   | Aggregation: count, max, sum over an expression                                            |
| D     | D1–D3   | Relationships: one-to-many, many-to-many, optional one-to-many                             |
| E     | E1–E2   | Result modification: sorting with limit, distinct projection                               |
| F     | F1–F2   | JSON: object and array queries                                                             |
| G     | G1–G2   | Set operations: union, intersection                                                        |
| H     | H1      | Metadata lookup: column data type                                                          |

**Table 3.1** The eight query categories of the benchmark suite, reused from Abrahám [11]. Identifiers (A1–H1) match across both experiments. Only read queries are tested.

### 3.2.2 Isolating the ORM Overhead

The methodological core is removing the database from the timing loop. Against a live server, wall-clock time is dominated by query planning, disk and network I/O, and result transfer, and the framework’s own work – building the command, opening a reader, materialising objects – is a small fraction the database masks; Abrahám’s full-database benchmark thus finds cheap queries nearly indistinguishable across frameworks [11]. To expose the mapping overhead, this thesis intercepts each framework at the lowest practical level.

For every query, a recording pass first runs it against the real database through a logging interceptor and captures three artefacts: the exact SQL command text the framework emits, the query plan’s *estimated* row count, and the output schema (column names, ordinals, and data types). During benchmarking the database call is then *mocked*: the command is short-circuited and an in-memory `DbDataReader` is constructed from the recorded schema and row count, so the framework receives a reader and materialises results exactly as in production, without the database round-trip. The `DbDataReader` behaves as a pre-filled in-memory table whose columns hold a constant default value of the [Common Language Runtime \(CLR\)](#) type mapped from the underlying SQL data type<sup>4</sup>, with row count matching the plan’s estimate.

The row count is taken from the plan’s *estimate* (`StatementEstRows`) rather than the query’s *actual* count, because obtaining the actual count means executing the query to completion, which may be potentially long-running – given that some queries (e.g. A4, section 3.3) return tens of thousands of rows. The estimate is a byproduct of

<sup>4</sup><https://learn.microsoft.com/en-us/dotnet/framework/data/adonet/sql/linq/sql-clr-type-mapping>

plan compilation, computed by SQL Server’s cost-based optimizer from column and index statistics [41]. Such estimates track the actual count closely when statistics are current [42], which is the case here since the dataset [40] is static and each query is compiled at least once before the estimate is read (also the experimental suite runs each query at least once in pilot phase, before measurements, see section 3.2.3); they diverge mainly under complex, correlated multi-table joins [43], similarly to the *D*-category queries here – so a mis-estimated count cannot be ruled out as a contributor to the anomalies discussed in section 3.3. This was not independently verified; doing so is left as future work (section 9.4).

The interception is as close to the metal as possible, capturing the maximum framework-attributable work – method dispatch, expression-tree compilation, identity-map bookkeeping, allocation – and nothing of the database [44]. Only read queries are tested, so change tracking and caching are disabled following the same read-only configuration as Abrahám’s benchmark (Section 1.4.3 [11]): EF Core’s `DbContext` uses `QueryTrackingBehavior.NoTracking`, and NHibernate is benchmarked through a plain `ISession` with no second-level cache configured. What that leaves active for NHibernate, and what cannot be turned off while using `ISession`, is the *first-level* cache – the identity map a session maintains for the lifetime of its persistence context.<sup>5</sup> Consequently, section 3.3 attributes part of NHibernate’s heavier memory profile directly to it.

This mocking strategy carries its own threat to validity, which is stated here rather than deferred, because it concerns the measurement itself. Because every column of the synthetic `DbDataReader` holds a *constant default value* of its mapped CLR type, the benchmark may *understate* the true mapping cost: a stream of identical values can lower the real price of materialization – string interning and small-object reuse, cache-friendly access patterns, and branch prediction all favor uniform data – and it hides the cost of data-dependent work that a real result set would incur, such as null handling, variable-length string allocation, culture-sensitive conversions, and JSON parsing. The measured overhead is therefore best read as a *lower bound* on the framework-attributable cost, and the relative ordering across frameworks (the finding the chapter relies on) is more robust than any single absolute figure.

What remains in the measurement is precisely the price of the abstraction. The mocking machinery is shared across the three frameworks and lives in the project’s repository in `benchmarks/Common/Mock` module [45].

### 3.2.3 Measurement and Environment

Measurements were collected with `BenchmarkDotNet` [46], which handles warm-up, multi-invocation iteration, outlier removal, and the separation of one-time JIT costs from

---

<sup>5</sup><https://nhibernate.info/doc/nhibernate-reference/architecture.html>

steady state,<sup>6</sup> reporting each method’s mean, standard error, and standard deviation with managed-heap allocations and per-generation garbage-collection counts. Allocated memory matters as much as time: on a managed runtime, allocation pressure is a large part of an **ORM**’s true cost and drives garbage collection, which triggers once heap allocations exceed a dynamically tuned threshold.<sup>7</sup>

All results were produced on the machine in table 3.2, which differs from Abrahám’s (AMD Ryzen 5 5600H); absolute times across the two experiments are therefore not directly comparable, but relative behaviour and memory within each are.

| Component                 | Specification                                                           |
|---------------------------|-------------------------------------------------------------------------|
| OS                        | Windows 11 (10.0.26100, 24H2)                                           |
| CPU                       | 12th Gen Intel Core i7-1260P, 2.10 GHz (12 physical / 16 logical cores) |
| RAM                       | 48 GB                                                                   |
| Runtime                   | .NET 8.0.24, X64 RyuJIT (JIT)                                           |
| Harness                   | BenchmarkDotNet v0.15.8                                                 |
| Database (recording pass) | Microsoft SQL Server (Wide World Importers [40])                        |

**Table 3.2** Testing environment for the overhead experiment.

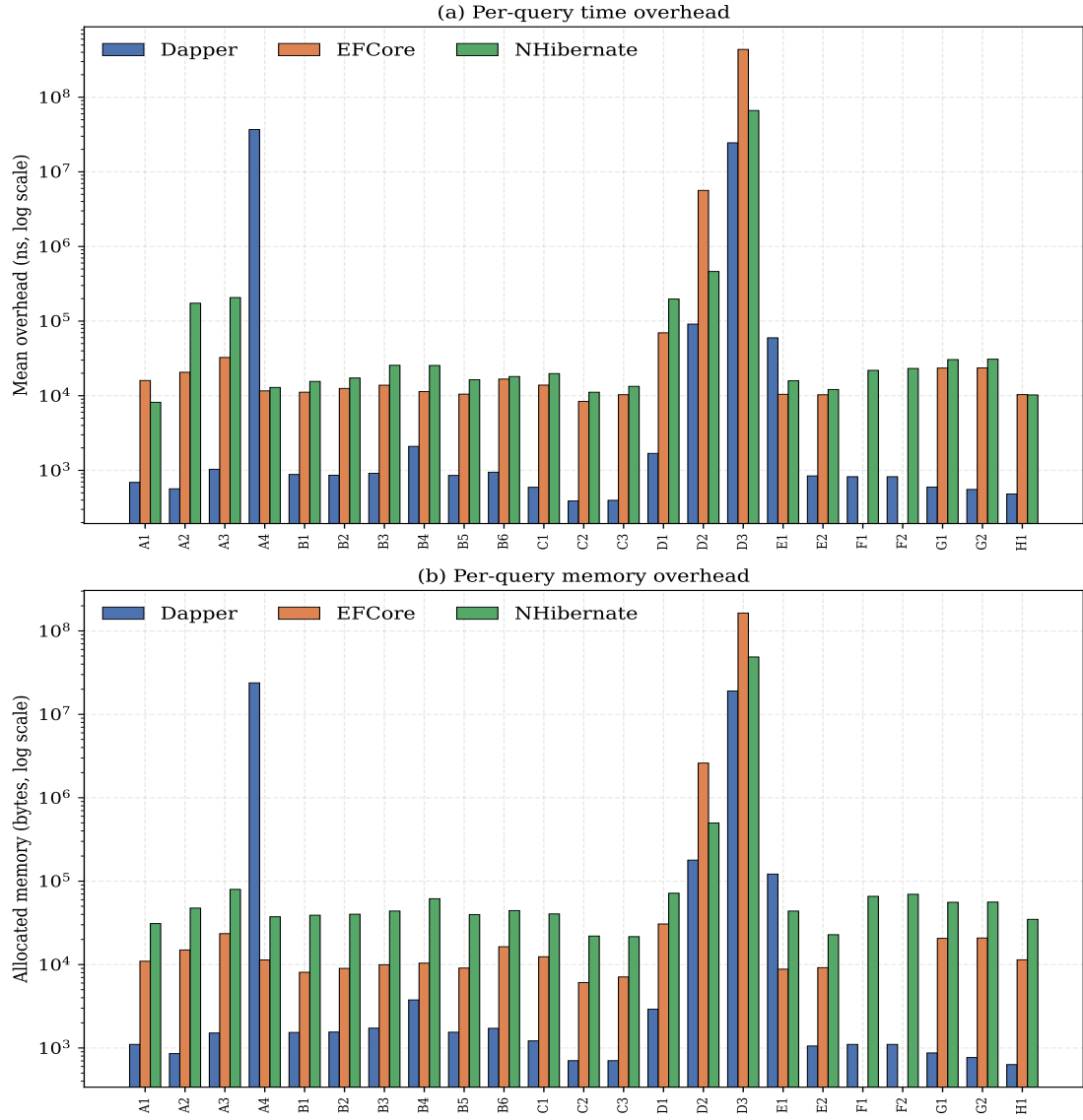
### 3.3 Results

Figure 3.1 reports the mean time overhead and allocated memory for all 23 queries on a logarithmic scale. The dominant pattern is unambiguous and consistent: across the suite, Dapper’s overhead is roughly an order of magnitude below that of the two full **ORM**s. For the entity-projection query A1, Dapper materialises a row in 690 ns while **EF** Core takes 16.0  $\mu$ s and NHibernate 8.2  $\mu$ s — a 23 $\times$  and 12 $\times$  gap respectively. Taken over the whole suite, the median per-query time overhead of **EF** Core is about 21 $\times$  Dapper’s and that of NHibernate about 20 $\times$ . The memory picture is the same shape: A1 allocates 1.1 KB under Dapper, 11 KB under **EF** Core, and 31 KB under NHibernate. NHibernate is consistently the heaviest allocator, consistent with its session-based architecture, in which each `ISession` maintains a first-level cache (a persistence context) of every object it materialises,<sup>8</sup> even where its time overhead is competitive with **EF** Core’s.

<sup>6</sup><https://benchmarkdotnet.org/articles/guides/how-it-works.html>

<sup>7</sup><https://learn.microsoft.com/dotnet/standard/garbage-collection/fundamentals>

<sup>8</sup><https://nhibernate.info/doc/nhibernate-reference/architecture.html>



**Figure 3.1** Per-query (a) time overhead and (b) memory overhead with the database mocked, on a logarithmic scale. Dapper is roughly an order of magnitude cheaper than the two full **ORMs** across the suite. Note the query-shape-dependent anomalies (A4, D2, D3), discussed in the text.

The order-of-magnitude rule has revealing exceptions that only the mocked measurement makes visible, and they all stem from query shape rather than framework identity. The stored-procedure query A4 inverts the ranking: Dapper buffers the full 66,741-row result eagerly — by default its Query method reads the entire reader into

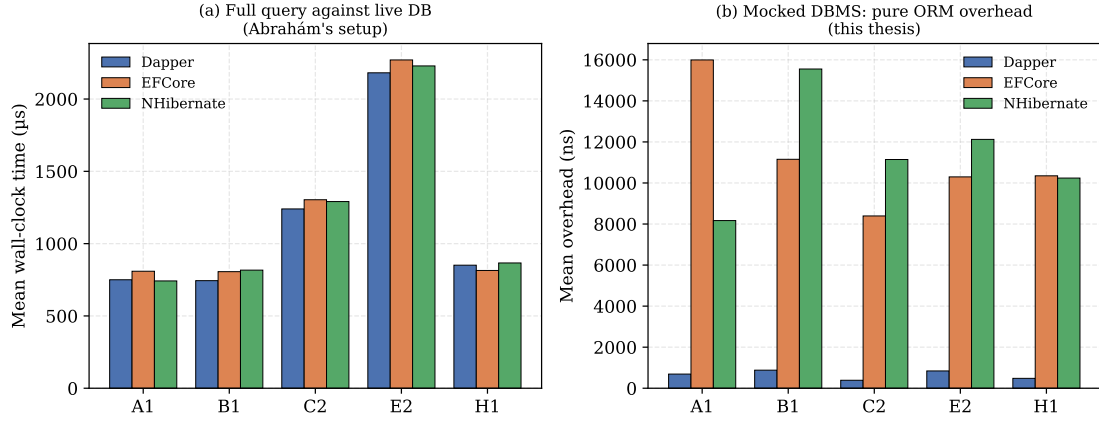
an in-memory list<sup>9</sup> — spending 36.9 ms and allocating 24 MB, while the full **ORMs** stay near 12  $\mu$ s. The optional-relationship query D3 produces the largest cost of the entire study — **EF Core** takes 437 ms and allocates 164 MB. The cause is *data duplication*: eagerly loading an entire table together with a one-to-many navigation in a single query makes **EF Core** emit a **SQL JOIN** that repeats every joined-row column across all of its related rows, so the framework transfers and materialises a heavily inflated result set — the effect that the framework’s split-query feature exists to avoid.<sup>10</sup> The many-to-many query D2 shows a milder version of the same effect (5.6 ms, 2.6 MB for **EF Core**). For F1–F2, **Dapper** and **NHibernate** handle these at typical cost, though only over mocked constant strings rather than actual **JSON** data; **EF Core** was dropped from the **JSON** queries, though a simple constant value (e.g. "{}") could represent a **JSON**, it was not pursued: it would mostly measure the cost of parsing a specific **JSON** payload rather than the framework-attributable mapping overhead we try to isolate, and is left as future work, together with quantifying that cost as a function of payload size (section 9.4). These cases underline that framework overhead is not a scalar property of a framework but a function of the framework *and* the query.

The contrast with full-database execution is the central result. Figure 3.2 places, side by side, the same five inexpensive queries as measured by Abrahám against a live database (panel a) and by the mocked benchmark here (panel b). Against the database the three frameworks are nearly identical — A1 takes 750, 809, and 742  $\mu$ s for **Dapper**, **EF Core**, and **NHibernate**, a spread under 10% — because the fixed cost of executing the query dwarfs the mapping cost. Subtract the database, and the same three frameworks diverge by more than 20 $\times$ .

---

<sup>9</sup><https://www.learndapper.com/misc/buffered-unbuffered>

<sup>10</sup><https://learn.microsoft.com/ef/core/querying/single-split-queries>



**Figure 3.2** The same five inexpensive queries measured (a) end-to-end against a live database (Abrahám’s setup [11], in  $\mu\text{s}$ ) and (b) with the database mocked to isolate ORM overhead (this thesis, in ns). End-to-end the frameworks are within  $\sim 10\%$ ; the database execution masks a  $>20\times$  difference in pure mapping overhead.

### 3.4 Implications for the Migration Assistant

Three conclusions carry into the rest of the thesis, read as empirical motivation for the assistant’s design. What this case study establishes is the underlying premise, not the specific numbers: mapping frameworks impose per-query costs that differ by more than an order of magnitude for identical query semantics, so the framework-attributable term the assistant carries across a migration is not negligible. Whether that term grows or shrinks once the target is an ODM/OGM is not measured here and is left to future work (section 9.4). Within the frameworks actually measured, this performance significance is exactly why the assistant preserves query semantics rather than rewriting them and leaves cost-driven choices to an explicit, opt-in advisory step (section 9.4). Overhead is query-shaped, not framework-shaped: the A4, D2, and D3 anomalies show that any future cost model must be per-query, matching the granularity at which the assistant already operates on queries. And the methodology is reusable: the recording-and-mocking harness is framework-agnostic, so as the assistant grows to treat raw SQL as a first-class source and ODM/OGM mappings as targets, the same technique can quantify the overhead those new paradigms introduce.

These benchmarks cover only the three .NET source frameworks; the Spring Data MongoDB and Neo4j targets were not benchmarked, as the JVM tooling and the very different cost profiles of document and graph stores fall outside this comparison, and extending the study to the target side is a natural continuation (section 9.4). Even confined to the relational sources, the experiment establishes the quantitative significance against which the assistant’s translations are made.

# Chapter 4

## The Translation Workflow

The assistant does not translate in a single step, nor does it delegate the whole task to one autonomous language-model agent which can decide freely what to do next. Instead, the translation is carried out by a semi-deterministic *workflow*: a fixed arrangement of specialized stages, each responsible for one well-defined sub-problem, connected by transitions (edges) whose direction is determined by the outcome of the previous stage/stages. This chapter describes that workflow at the level of *what each stage contributes* to a correct translation and *why* it is shaped the way it is.

The high-level overview of the workflow is presented in section 4.1. A concrete example of a real workflow trace is visually demonstrated in section 4.2. Each pipeline stage is described in detail in section 4.3.

The design decisions behind this shape – the justification for preferring an orchestrated workflow over a free agent, the typed state, the context management, the prompting strategy, the verification layers and the semantic-equivalence machinery the workflow enforces, and the bounded self-repair loop – are the subject of chapter 5. The concrete artifacts behind the discussion – the data schemas, the system prompts, the harness templates, and a full execution trace – are reproduced in chapter B.

### 4.1 High-Level View

The workflow is realized as a semi-deterministic *state machine*: a directed graph of nodes over a shared, typed state object, executed by the LangGraph runtime<sup>1</sup> [24]. A node reads the current state, performs its task – possibly invoking a language model or an external tool – and returns an update that is merged back into the state; an edge then decides which node runs next. The design follows the broader move in language-model systems away from a single monolithic prompt and towards *compound* systems [47] in which reasoning and acting are interleaved across several controlled steps, of which the

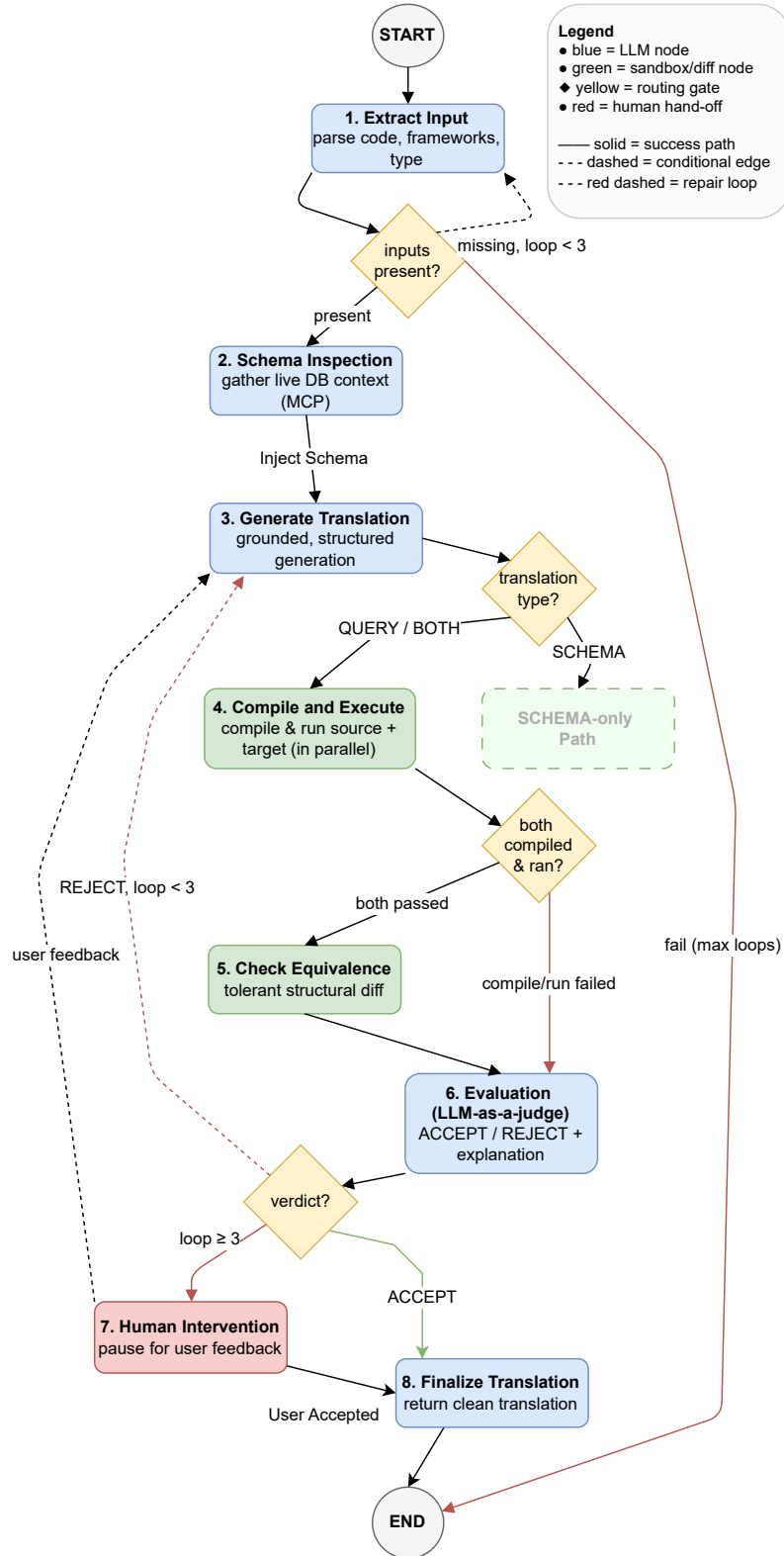
---

<sup>1</sup>LangGraph, a library for building stateful, multi-actor applications over language models [24].



*ReAct* paradigm [21] is the primary example. The assistant adopts that interleaving but constrains it: rather than letting one agent improvise the whole control flow, the order in which generation, compilation, execution, comparison, and judgment occur is fixed in advance, so that every translation is produced and checked in the same reproducible, explainable and observable way (section 5.1).

Figure 4.1 shows the compiled graph. Execution begins at a single entry transition and terminates in one of a small number of well-defined end states: a finalized translation, a graceful abort when the request cannot be understood, or a human (user) intervention when automated repair does not converge. The path taken between entry and exit is shaped by two factors: the *scope* of the request (whether the user asked to translate a schema, a query, or both) and the *outcome* of each verification stage along the way.



**Figure 4.1** State machine diagram representing the schema and query translation workflow.

It is useful to distinguish three kinds of nodes, because the distinction reflects an important division of labor between the language model and deterministic code/algorithms (section 5.2). *Generative* (blue *AI* rectangles in the diagrams) nodes consult a language model to produce or judge content. *Deterministic* (light blue *tool calls* or purple *human* rectangles) nodes contain no model call at all: they either prepare the exact inputs for a subsequent tool or execute that tool and record its result. This separation is what allows the verification steps to run in a fixed, reproducible order rather than depending on a model remembering to invoke them. Table 4.1 lists the nodes of the graph and the default model behind each generative one.

| Stage                   | Kind          | $\alpha$ | Default model   |
|-------------------------|---------------|----------|-----------------|
| Input extraction        | generative    | 1        | qwen3.5-122b    |
| Schema inspection       | generative    | 3        | deepseek-v4-pro |
| Translation generation  | generative    | 3        | kimi-k2.7-code  |
| Validation preparation  | deterministic | 0        | —               |
| Schema/query validation | deterministic | 0        | —               |
| Equivalence preparation | deterministic | 0        | —               |
| Equivalence check       | deterministic | 0        | —               |
| Evaluation (judge)      | generative    | 1        | deepseek-v4-pro |
| Human intervention      | interrupt     | —        | —               |
| Finalization            | generative    | 2        | kimi-k2.7-code  |

**Table 4.1** The stages of the translation workflow, classified by kind, with the autonomy level  $\alpha$  (definition 2) and the default model behind each generative stage. Model names are logical aliases resolved at run time against the platform described in table 4.2; deterministic stages invoke no model.

#### 4.1.1 A Formal Model of the Workflow

We can also formalize the workflow. We model the assistant as a *typed state-transition system*, in the spirit of finite state-machine formulations of language-model task solving [23], in which the graph fixes the possible transitions while a model may decide only what happens *within* a node.

**Definition 1** (Translation workflow). A translation workflow is a tuple

$$W = (\Sigma, C, I, O, N, \delta, s_0, F),$$

where  $\Sigma$  is the state space — the set of values of a typed record whose fields carry the extracted inputs, the schema summary, the generated harnesses, the validation results, the equivalence diffs, and the loop counters (overview of the concrete fields can be found in table 5.1);  $C \subseteq \Sigma$  is the context, the fields fixed at entry (framework pair, scope, source

code, run configuration) that no node may overwrite;  $I$  and  $O$  are the input and output interfaces, narrow projections of  $\Sigma$  that a caller must supply and receives back (section 5.2);  $N = \{n_1, \dots, n_k\}$  is the set of nodes, each a partial function  $n_i : \Sigma \rightarrow \Sigma$  that reads the state and returns a merged update;  $\delta : \Sigma \rightarrow N \cup F$  is the routing function, a guard evaluated on the current state that selects the next node (or an end state) after a node completes;  $s_0 \in \Sigma$  is the initial state; and  $F = \{FINALIZED, ABORTED, HANDOFF\}$  is the set of terminal states. A run is the sequence  $s_0, s_1, \dots$  with  $s_{t+1} = n(s_t)$  for  $n = \delta(s_t)$ , terminating when  $\delta(s_t) \in F$ .

Each node is further characterized by a signature  $n = (\text{pre}_n, \beta_n, \text{post}_n, \alpha_n)$ , where the *precondition*  $\text{pre}_n : \Sigma \rightarrow \{\top, \perp\}$  is the guard that must hold for  $n$  to run,  $\beta_n$  is the body (a deterministic computation, a constrained model call, or a bounded tool-using agent), the *postcondition*  $\text{post}_n : \Sigma \rightarrow \{\top, \perp\}$  is the check applied to  $n$ 's output before the run may proceed, and  $\alpha_n$  is the *autonomy level* defined below. A node whose body is a model call is *structured* when its codomain is a typed schema  $T \subset \Sigma$  validated mechanically, and *unstructured* when it is free text admitted only if it satisfies  $\text{post}_n$ .

**Definition 2** (Autonomy level). *The autonomy level  $\alpha_n$  of a node grades how much control over its own execution the node cedes to a language model:*

$$\alpha_n = \begin{cases} 0 & \text{deterministic: no model call (code or tool only);} \\ 1 & \text{single model call, structured output against a typed schema;} \\ 2 & \text{single model call, unstructured output under a post-constraint;} \\ 3 & \text{tool-using ReAct agent, restricted toolset, bounded steps;} \\ 4 & \text{tool-using ReAct agent, unrestricted toolset.} \end{cases}$$

The autonomy level of each stage is recorded in table 4.1. The production workflow admits no node at  $\alpha = 4$ : the abandoned single-agent prototype (section 5.1) was in effect a single  $\alpha = 4$  node, and the redesign redistributes that autonomy into individual stages capped at  $\alpha = 3$ . The *determinism ratio*  $\rho = |\{n \in N : \alpha_n = 0\}|/|N|$  then makes the workflow determinism quantifiable (e.g. four of the ten stages are fully deterministic ( $\rho = 0.4$ )).

### 4.1.2 The Model Portfolio

A single language model is not used throughout. Each stage poses a different problem – structured output extraction, exploratory tool use, code generation, or judge verdict – and these differ in the capabilities they reward, so the workflow assigns to each stage a model suited to it, drawing on a small portfolio of open-weight models served by the *e-INFRA CZ AI-as-a-Service* platform [48] through an *OpenAI-compatible API*<sup>2</sup>. Table 4.2 summarizes the portfolio. The selection follows two principles seen in the

<sup>2</sup>OpenAI API Specification: <https://developers.openai.com/api/reference/overview>

| Stage (role)             | Model                | Params       | Quant.     | Context | T / top-p |
|--------------------------|----------------------|--------------|------------|---------|-----------|
| Generation, finalization | Kimi-K2.7-Code [51]  | —            | int4       | ~262 k  | default   |
| Input extraction         | Qwen3.5-122B [52]    | 122 B / 10 B | fp8        | ~262 k  | 0.7 / 0.8 |
| Schema inspection        | DeepSeek-V4-Pro [53] | —            | fp4        | ~1.05 M | default   |
| Evaluation (judge)       | DeepSeek-V4-Pro [53] | —            | fp4        | ~1.05 M | default   |
| Generation fallback      | Qwen3.5 [54]         | 397 B / 17 B | fp8 / int4 | ~262 k  | 0.6 / —   |
| Extraction fallback      | gpt-oss-120b [55]    | 120 B        | mxfp4      | ~131 k  | default   |
| Local (Ollama) fallbacks | Qwen3 (27–30 B)      | 27–30 B      | —          | —       | —         |

**Table 4.2** The open-weight models behind the generative stages, served by e-INFRA [48]. Parameter counts are given as *total/active* for *Mixture-of-Experts (MoE)* models where the published name discloses them; a dash denotes a figure not encoded in the released name. Quantization and context window are as configured on the serving platform. *T/top-p* are the request sampling parameters as configured on the platform and left **unchanged by us**; *default* marks a stage where the platform sets neither and the provider default applies. Two additional platform-fixed parameters not shown per row: Qwen3.5-122B pins top-*k* = 20 and presence-penalty = 1.5, the 397 B fallback pins presence-penalty = 1; per-request output is capped at 32 k tokens (Qwen, gpt-oss) or 48–64 k (Kimi, DeepSeek). The two local fallback models are served separately through Ollama by XRG at KSI MFF UK [50].

literature. First, strong *reasoning-capable* models are used where the task is open-ended and benefits from intermediate *Chain-of-Thought (CoT)* reasoning [19] – exploratory schema inspection and code generation – whereas a fast instruction-following non-reasoning model suffices for the near-mechanical extraction step. Second, and more importantly, the model that *judges* a translation is deliberately drawn from a different family than the one that *produces* it: evaluating one’s own output is a known weak point of language models [49], and the quality of the feedback that drives repair – rather than the act of repair itself – has been identified as an important constraint on iterative self-correction [29]. Using an independent judge mitigates the self-assessment bias documented in the *LLM-as-a-judge* literature [31]. Lastly, the *LLM* leaderboard rankings by benchmark results provided by *LiveBench*<sup>3</sup> [34] are used to select the best-performing models for the above categories available on the *e-INFRA CZ* platform at the time of writing.

When a stage does not specify a model, the platform default applies. We do not tune sampling: temperature, top-*p*, and the other decoding parameters are left at the values the serving platform configures for each model (table 4.2). Every generative stage is, moreover, backed by an ordered *fallback chain* of alternative models, so that a transient outage or a persistent failure at one model escalates to the next rather than aborting the run; the principle behind these chains, and the bounded retry policy that accompanies them, is discussed in section 5.8.

<sup>3</sup><https://livebench.ai/>

## 4.2 A Trace of a Running Example

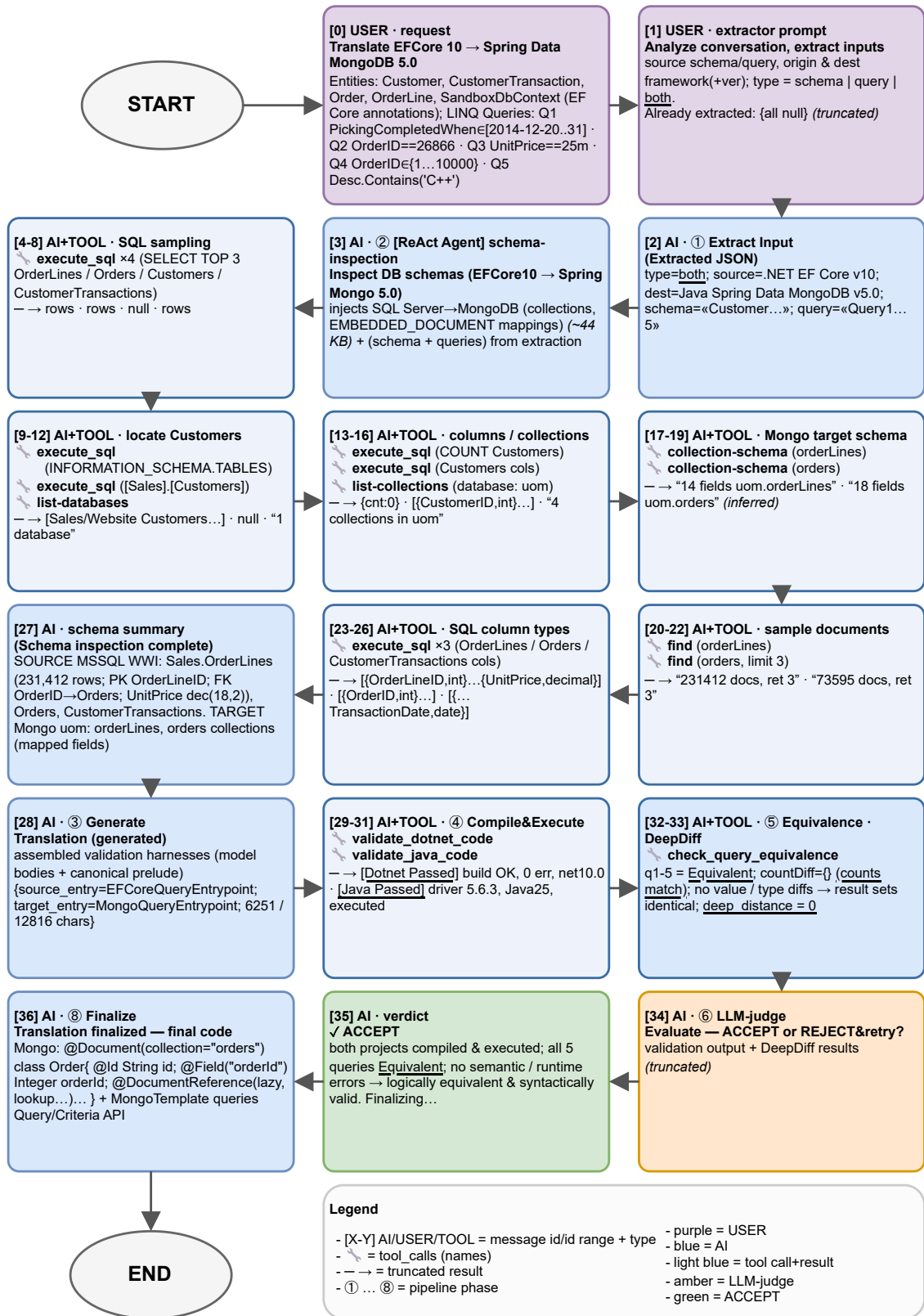
To make the abstract graph concrete, consider the running example introduced in section 2.4: an Entity Framework Core (EFCore) entity together with a [LINQ](#) query, to be translated into Spring Data MongoDB. The request therefore has scope *BOTH*. Table 4.3 provides a summary of the stages a concrete successful run visits, the routing decision taken after each, and the effect on the shared state. A complete, tool-level trace of the same run – every model call and tool result, as captured by the tracing backend<sup>4</sup> – is reproduced in figure 4.2.

| #  | Stage                   | Routing decision and effect                                       |
|----|-------------------------|-------------------------------------------------------------------|
| 1  | (entry)                 | Required inputs not yet present → extract them.                   |
| 2  | Input extraction        | Frameworks, code, and scope identified → inputs complete.         |
| 3  | Schema inspection       | Source and target schemas summarized into the migration context.  |
| 4  | Translation generation  | Source- and target-side validation harnesses authored and saved.  |
| 5  | Validation preparation  | Compilation/execution of both sides requested.                    |
| 6  | Query validation        | Both sides compiled and ran → proceed to comparison.              |
| 7  | Equivalence preparation | Comparison of the two result sets requested.                      |
| 8  | Equivalence check       | Result sets compared; differences (if any) recorded.              |
| 9  | Evaluation (judge)      | Evidence reviewed → ACCEPT.                                       |
| 10 | Finalization            | Clean target code projected from the validated harness; run ends. |

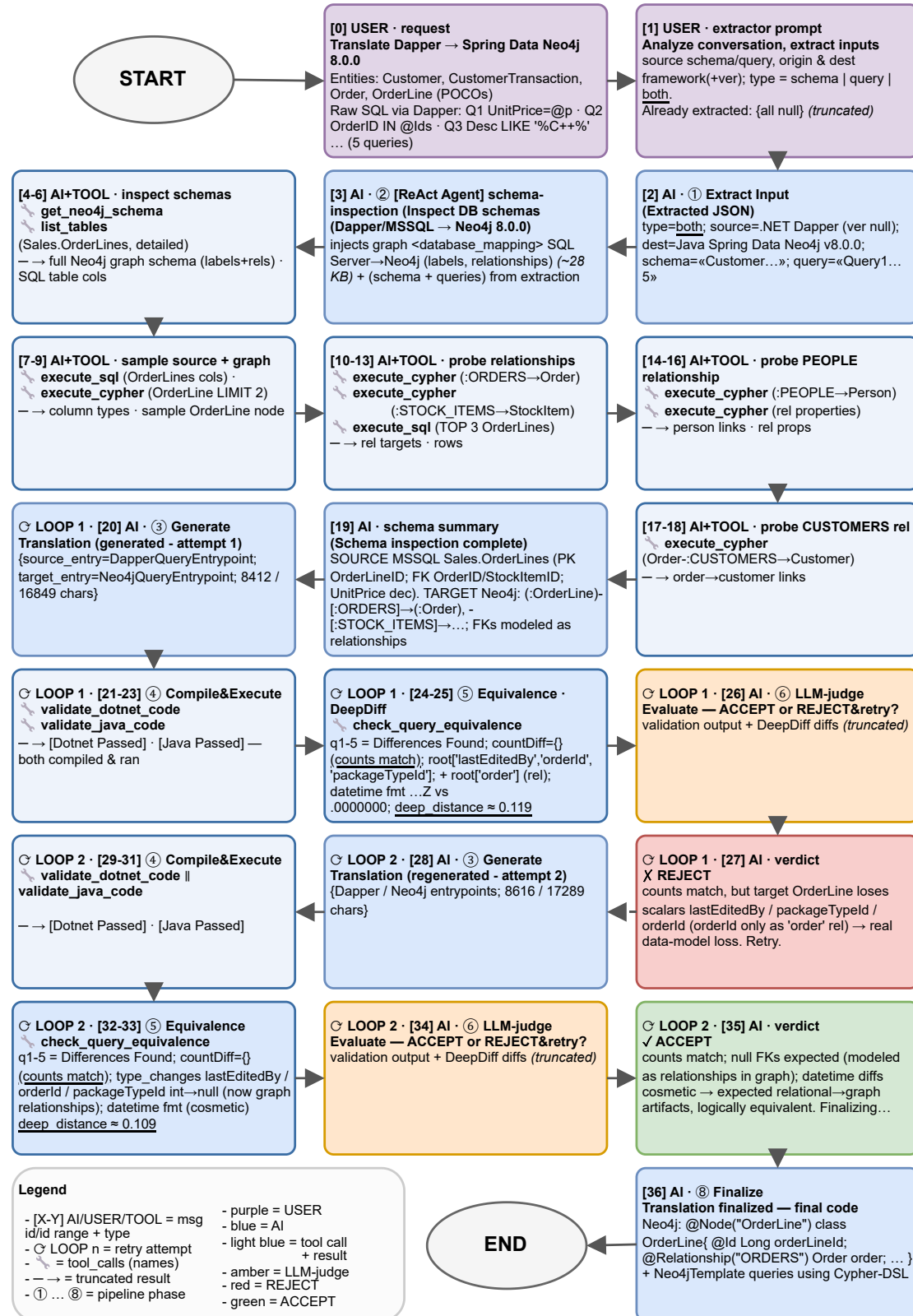
**Table 4.3** The stages visited by a successful end-to-end run of the running example (scope *BOTH*), the routing decision taken after each stage, and the resulting effect on the shared state. The same sequence is shown structurally in figure 4.1.

A run that does not succeed on the first attempt diverges from this ideal path at a verification stage: a compilation failure, a semantic mismatch, or a rejection by the judge sends control back to translation generation with corrective feedback, and the cycle repeats under a fixed loop count before the run either succeeds or is handed to a human. A concrete example of this case is shown in figure 4.3.

<sup>4</sup>Traces are captured with LangSmith and via Pydantic Logfire; see <https://docs.smith.langchain.com/> or <https://pydantic.dev/docs/logfire/get-started>.



**Figure 4.2** A complete trace of a successful end-to-end run of EFCore to Spring Data MongoDB successfully translated at first attempt. The trace shows the feedback loop in action: the first attempt succeeds, so no repair is needed.



**Figure 4.3** A complete trace of a successful end-to-end run of Dapper to Neo4j successfully translated at second attempt, with an equivalence failure on the first. The trace shows the feedback loop in action: the first attempt fails in equivalence checking, the judge rejects it, and the second attempt succeeds.



## 4.3 The Stages in Detail

Each subsection below describes one stage in terms of its purpose, what it consumes and produces, the role of the language model where one is involved, and the principle that motivates its design. The full data schemas and prompts are deferred to sections [B.1](#) and [B.2](#).

### 4.3.1 Input Extraction

The workflow must begin from a precise, typed description of the task, but the user expresses that task in free-form conversation<sup>5</sup>. The first stage bridges this gap: it reads the conversation and recovers the structured parameters the rest of the pipeline depends on – the source and target frameworks (and, if mentioned, their versions), the scope of the request (schema, query, or both), and the raw source code to be translated. This is mainly a classification-and-extraction problem rather than a reasoning one, which is why it is entrusted to a fast instruction-following model rather than a reasoning model. The model returns its answer as a typed object rather than as markdown/free-text output, so that the result can be validated mechanically against an expected shape before the workflow proceeds; the schema is reproduced in section [B.1](#). The code is captured line by line so that the original indentation – significant in the source languages and easy for a model to corrupt – is preserved exactly. And a dedicated error field lets the model report, as data, that the request cannot be fulfilled (for instance, that no translatable code was supplied), allowing the workflow to terminate cleanly with an explanation rather than proceeding on malformed input.

Extraction is robust by construction. If the model returns nothing usable, or an object still missing the minimum required fields, the stage is retried; after a fixed number (i.e. three) of unsuccessful attempts the run ends gracefully rather than looping indefinitely. This bounded-retry discipline, shared by every loop in the graph, is the mechanism by which the system honors the robustness requirement [2.5](#).

### 4.3.2 Schema Inspection

Before any code is generated, the assistant examines the *actual* databases involved in the migration. An [LLM](#) asked to translate a persistence layer in the abstract will easily invent wrong collection names, field types, or relationship cardinalities. Grounding generation in the live database schema is therefore the established protection against such hallucination [[56](#), [26](#)], and is the first of the verification layers set out in section [5.6](#). This stage connects to the live source and target databases, inspects the schema or the entities relevant to the code at hand using specialized tools, and condenses them into a

---

<sup>5</sup>Also a structured input is possible, which skips the extraction stage, that is mainly used by [MCP](#) tool input if used with other Copilots. See chapter [6](#).

compact textual summary that is later supplied to the translation stage as part of its context (section 5.3).

The database connections are not bespoke integrations but are exposed through the [Model Context Protocol \(MCP\)](#)<sup>6</sup> [57], the same open standard the project uses for all external tool access. Because inspecting a large schema is open-ended and may require many successive queries, the stage is entrusted to a reasoning-capable model and is given a mechanism to keep only the most recent observations in its working context, so that inspecting a large database does not overflow the context window (also discussed in section 5.3). Should the [MCP](#) tools fail to inspect databases, the stage degrades gracefully: it records that inspection was skipped and the workflow continues without a schema summary rather than failing – in that case, the subsequent translation stage will try to guess the database’s mapping from input schema and run queries against live databases (if they are accessible) – so if the guesses are incorrect and retry budget is exhausted, the translation may eventually converge to human intervention.

### 4.3.3 Translation Generation

This is the stage where the translation is actually produced, and for that reason, it uses deliberate step-by-step generation process, which is the longest, most expensive and important stage in the pipeline. For those reasons, it runs the strongest reasoning model in the portfolio. It consumes the framework names, the scope, the source code, and the schema summary, and it is supplied with a system prompt assembled specifically for the framework pair at hand – carrying the translation contract, the framework-specific rules, the exact pinned dependency versions, and worked structural examples. The construction of that prompt is the subject of section 5.4.

A design decision made here propagates through the rest of the workflow: this stage does *not* emit the polished, user-facing translation. Instead it authors the material needed to *verify* a translation – runnable harnesses for the source and target sides – and defers production of the clean answer until after that material has passed every check (section 4.3.8). The published result is thus never an unverified claim by the model but always a projection of code that has demonstrably compiled, executed, and preserved the query’s meaning. This reflects the principle, established in execution-based evaluation of generated code, that correctness should be decided by running the code rather than by inspecting it [14, 17].

The stage operates as a tool-calling agent in the ReAct style [21]: it may consult documentation and the web to get an unfamiliar [API](#) right before committing – [Retrieval-Augmented Generation \(RAG\)](#) [26] in its tool-driven form, where the model retrieves

---

<sup>6</sup>The Model Context Protocol, an open standard for connecting language-model applications to external tools and data sources [57].

grounding passages on demand instead of drawing on a fixed corpus<sup>7</sup> – and it concludes by recording its result through a series of dedicated *save tools*. Notably, it is *not* given the database-inspection tools (the schema was already summarized upstream) nor the validators (verification is performed by the dedicated downstream stages); restricting the tool surface keeps the agent focused on generation and prevents it from exhausting its maximum step count in redundant exploration, a known problem of ReAct agents being exposed to many redundant tools [58]. Only the genuinely variable part of the code is authored by the model; the unchanging scaffolding on which the validators depend is supplied deterministically, a separation explained in section 5.3 and reproduced in section B.3. The generation schema and the finishing action are documented in section B.1.

#### 4.3.4 Compilation and Execution

Producing syntactically plausible code is not the same as producing code that builds and runs, and the gap between the two is exactly where LLM code generation fails – through hallucinated APIs, missing dependencies, or type errors [16]. The workflow therefore compiles and executes both the source and the target translations against real CLI tools and databases. This is carried out by two deterministic stages: a preparation stage assembles the precise validation request, and an execution stage runs it. Splitting the work this way – rather than trusting the generative model to remember to compile its own output – is what makes the order and the content of verification reproducible.

For a schema-only request, only the translated entities are compiled, and each schema entity is queried (using `.findOne()/ .fetch(...limit(1))` method to obtain the first record in a result-set for mapping validation<sup>8</sup>. For a request involving queries, both sides are compiled, each entity schema is queried once as in schema-only case, and query harnesses are executed and – because the two sides are independent – they are processed in parallel. Each side is built and run inside an isolated, disposable sandbox provisioned for the purpose<sup>9</sup> [59]; the execution writes its query results out in a normalized form that the next stage can compare. The full mechanics of the sandboxes – how projects are assembled, how files are transported, and how build output is captured – are described in section 5.6.1. What matters at the level of the workflow is the contract: each side reports whether it compiled and ran, and a failure here does not abort the run but is fed back to the generation stage for correction.

---

<sup>7</sup>Documentation search is provided through the Spring and Microsoft documentation services exposed over MCP; open-web search uses the Tavily API, <https://www.tavily.com/>.

<sup>8</sup>Path not shown in figure 4.1; but is structurally very similar to it

<sup>9</sup>Sandboxes are provisioned and executed through a self-deployed Daytona instance [59].

### 4.3.5 Equivalence Checking

Compilation establishes that a translation is well-formed; it says nothing about whether the translated query *returns the same data* as the original. Establishing that is the role of the equivalence check, reached only once both sides have executed successfully. It compares the result sets the two sides produced – their row counts and representative sample records – and records any differences for the downstream judge and for any subsequent repair attempt. The comparison is deliberately tolerant of differences that are provably immaterial to meaning, such as field ordering, harmless numeric-precision variation, or a reversed sort order, in keeping with the semantic preservation requirement 2.5. Comparing *observed execution results* rather than the code itself is what makes the check a genuine test of semantic equivalence rather than of surface similarity, echoing the executability-oriented approach to code translation [17]. The comparison is computed with the DeepDiff library<sup>10</sup> [33]; because its algorithm, its tolerance rules, and the serialisation contract that makes the two result sets comparable are central to the correctness argument of this thesis, they are treated in full in section 5.7 rather than here.

### 4.3.6 Evaluation: LLM-as-a-Judge

The equivalence check produces evidence – compiler outcomes and a structural diff of the results – but evidence still has to be interpreted: a recorded difference may be a harmless artifact or a genuine semantic error. The workflow resolves this with an independent language model acting as a judge, a now-standard technique for automated quality assessment whose strengths and biases are characterized by Zheng et al. [31]. The judge reads the accumulated evidence and returns a binary verdict – accept or reject – together with a written justification. The verdict decides whether the run proceeds to finalization or returns for another attempt.

Two design choices follow directly from the literature. The judge is required to supply a justification, never a bare verdict, both for observability and because the written reasoning is the repair signal – reasoning stated before the verdict, in the CoT tradition [19] – and the usefulness of that signal, not the repair mechanism, is what controls whether iterative correction succeeds [29]. And the judge runs a different model from the generator, so that the system does not grade its own work; the judge’s role is to interpret deterministic evidence, not to reframe correctness, which keeps its authority bounded and is the basis of the trust argument in section 5.6.

---

<sup>10</sup>DeepDiff, a structural difference engine for Python objects [33].

### 4.3.7 Human Intervention

Automated self-repair is powerful but not unconditionally reliable; the empirical finding that it is “*not a silver bullet*” [29] is taken seriously in the design. When repeated automated attempts fail to converge, or when the generator cannot produce well-formed output at all, the workflow does not loop indefinitely or emit a hallucinated result. It pauses and hands control to a human reviewer, showing the current state – the validated artifacts produced so far, the judge’s explanation, the recorded differences, and the entire chat conversation with intermediate results – for inspection. The reviewer either accepts the current translation, sending the run to finalization, or rejects it with feedback, which re-enters the repair cycle exactly as an automated rejection would. This controlled hand-off is the graceful-degradation half of requirement 2.5, and the fraction of runs that reach it is the metric of requirement 2.5 that the evaluation reports (chapter 8).

### 4.3.8 Finalization

The last stage runs only after a translation has been accepted – by the judge, by a clean compilation in the schema-only case, or by a human reviewer. Its task is not to translate but to *extract*: it takes the already-validated harness, which is the authoritative source of truth, and projects from it the clean, production-ready target code the user will see, stripping away the scaffolding that existed only for verification while preserving every annotation and predicate that the checks confirmed. Because its input is fixed once validation has passed, finalization is a constrained projection rather than a fresh generation, which guarantees that the published answer is always a faithful rendering of verified code.

Separating finalization from generation serves a second purpose specific to the evaluation. Because the published code is the artifact compared across experimental runs by a structural code-similarity metric (chapter 8), it must be produced under fixed conditions: this stage therefore pins one specific model with no fallback and temperature set to zero.

# Chapter 5

## Design Decisions

The shape of the workflow presented in chapter 4 is the outcome of a sequence of design decisions, several of which reversed an earlier, more autonomous prototype of the assistant. This chapter tries to present and explain those decisions. It follows the pattern, now standard in the explored literature on LLM systems, between a *workflow* – a system in which models and tools are orchestrated along predefined code paths – and an *agent* – a system in which the model dynamically directs its own process and tool calls [22]. Section 5.1 justifies choosing the former over the latter; section 5.2 describes the typed state that carries the workflow; section 5.3 explains how each model call’s context is kept clean and how the migration-specific knowledge is routed; section 5.4 defines the prompting strategy and its design choices; and section 5.5 records the redesign of the structured-output interface. The second half of the chapter turns to correctness: section 5.6 explains the four verification layers we use, section 5.7 the query semantic-equivalence at its center – the requirements 2.5 and 2.5 are satisfied there – and section 5.8 the bounded self-repair loop that turns verification failures into corrections.

### 5.1 A Free ReAct Agent versus an Orchestrated Graph

The first prototype of the assistant was a single ReAct agent [21]: one reasoning model equipped with every tool the task could need – database inspection, documentation search, both compilers, and the query equivalence checker – and instructed to translate, validate, and finish with one structured answer. This design is attractive on paper because the model decides for itself when to inspect, generate, and verify. In practice, it showed exactly the failures that the explored literature attributes to premature agentic autonomy [22]. The agent would skip or reorder verification steps, since nothing but the prompt compelled it to compile its own output; it would burn its maximum step count on redundant database exploration and reach the recursion limit before saving

a result; the verbose compiler and diff output it pulled into its own context degraded subsequent generation; and, because every run improvised a different tool sequence, no two runs were comparable, which made systematic evaluation impossible.

The production design switches the responsibility. The *order* of generation, compilation, execution, comparison, and judgment is fixed by the graph’s edges – the routing function  $\delta$  of definition 1 – and cannot be skipped or rearranged by any model. The model’s autonomy is spread into individual, self-contained stages, where it is genuinely useful – deciding which documentation to consult before generating, or which entities to inspect in a large schema. Each such inner agent, or *subagent*<sup>1</sup>, runs with a deliberately restricted set of tools (section 4.3.3) and a bounded step count. In the vocabulary of definition 2: the abandoned prototype was a single node at autonomy level  $\alpha = 4$ , whereas the production workflow caps every node at  $\alpha \leq 3$  and keeps four of the ten stages fully deterministic ( $\rho = 0.4$ ). The result is a *semi-deterministic* system: creative where creativity is required (temperature unchanged), deterministic everywhere correctness is compulsory. This is what makes every run observable, explainable, reproducible in structure, and directly comparable in the experimental evaluation (chapter 8).

## 5.2 Typed Shared State and Stateless Nodes

The stages communicate exclusively through a single shared *state object*: a typed record whose fields carry the extracted inputs, the schema summary, the generated harnesses, the validation results, the equivalence diffs, and the loop counters. A node is a pure function of this state – it reads what it needs, performs its task, and returns a partial update that the runtime merges back. The LangGraph graph is implemented using *Google’s Pregel* ecosystem [60]: a robust system for large-scale graph processing, specifically tailored for low-level LLM orchestration. Because the complete state is externalized, the runtime can checkpoint it at every transition, which is what allows a run to pause indefinitely at the human-intervention stage and resume later without loss (section 4.3.7). Because nodes are stateless, each can be independently retried on transient failure, cached when its inputs have not changed, and observed in isolation by the tracing backend (*LangSmith* or *LogFire*). The state purposefully distinguishes a narrow *Input* ( $I$  in definition 1) interface (what a user must supply) and a narrow *Output* ( $O$ ) interface (what a user receives back) from the wider internal *State* ( $\Sigma$ ), so the graph presents a stable API surface to the surrounding application (chapter 7). The context  $C$  of definition 1 is the entry-fixed subset of  $I$  – the framework pair, the scope, and the source code – which no node overwrites. Table 5.1 groups the most important fields by interface; the full dataclass listing is reproduced in listing B.4.

---

<sup>1</sup>In LangGraph terms also referred to as *subgraph*



| Interface                                | Key fields                                                                                                                                                                                                   | Role                                                                                                                                                                    |
|------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| $I$<br>(InputState)                      | <code>messages</code><br><code>source_{schema,query}_code</code><br><code>translation_type</code><br><code>{source,destination}_target</code>                                                                | user-facing conversation channel<br>raw source code to translate<br>scope: SCHEMA   QUERY   BOTH<br>framework pair (+ versions)                                         |
| $O$<br>(OutputState)                     | <code>translated_{schema,query}_code</code><br><code>*_validation_{schema,harness}_code</code><br><code>explanation_message</code>                                                                           | clean, user-facing target code<br>the four runnable validation harnesses<br>final judge / reviewer justification                                                        |
| $\Sigma \setminus (I \cup O)$<br>(State) | <code>schema_context</code><br><code>*_query_validation_results</code><br><code>query_equivalence_deep_diffs</code><br><code>translation_messages</code><br><code>{extraction,translation}_loop_count</code> | schema summary, held out of messages<br>executed source/target result sets<br>recorded equivalence differences<br>isolated repair-loop channel<br>bounded-loop counters |

**Table 5.1** The most important fields of the shared state, grouped by the interface of definition 1 they belong to.  $\Sigma$  (the State record) extends both  $I$  and  $O$ ; the internal fields listed last are private to the graph. Message channels carry conversational history; `add_messages` reducers merge updates by identifier.

### 5.3 Keeping the Reasoning Context Clean

LLMs degrade measurably as their input context grows and as relevant information is buried among irrelevant text [25]. That is why managing what enters a model’s context is an engineering concern of the same rank as prompt wording [27]. Four mechanisms allow this to be done systematically.

First, huge intermediate artifacts live in dedicated state fields, not in the conversational history: the schema summary, the harness code, and the equivalence diffs are stored as data and injected into a prompt only where and when they are needed. Second, the verification loop runs on a message channel separate from the user-facing (UI) conversation, so that iterative trial-and-error logs never pollute the primary channel. Third, the interior agents (*subagents*) apply context editing: the schema-inspection agent, which may issue dozens of database queries, retains only its most recent tool results



once a token threshold is exceeded, and the generation agent likewise *edits* its context – clearing stale tool results – rather than summarizing its history, since summarization was observed to discard the task statement itself under length pressure (section 9.1). Fourth, a failed attempt is never fed verbatim into the next one, but instead, the *repair feedback* is distilled deterministically from: the judge’s explanation, the last few tool outputs, and the diffs. Additionally, we neutralize high-attention verdict tokens such as `REJECTED` in the repair feedback, because in observed experiments some models copied such tokens (i.e. *ACCEPT/REJECT*) verbatim into a code field.

## 5.4 Prompts as Stage Interfaces

A direct consequence of the orchestration design is that there is no single “assistant prompt.” Each generative stage carries its own system prompt, addressed to a specific “persona” with a single responsibility – an information extractor, a database schema inspector, a translation architect, an evaluator, and a finalizer that is explicitly told its input is already validated and its job is extraction, not re-translation. Each prompt is short, relative to its instruction set, so the rules that matter for a stage are not diluted by rules that do not [25]. Furthermore, each can be paired with the model best suited to its task (section 4.1.2), and each can be revised and audited in isolation. The complete prompts are reproduced verbatim in section B.2 and are not re-explained here.

Notably, the generation prompt is the only one assembled *dynamically*, because it must carry the ground truth for the specific framework pair at hand (section 5.6). It is built at run time from four blocks: (i) the *translation contract* – numbered rules covering framework semantic differences (e.g. `MongoTemplate` with the `Query/Criteria API`, type-safe Cypher-DSL rather than string concatenation), input/output rules (structured output design and positive/negative rules), and the division of labor with the later validators; (ii) the verbatim *pinned dependency manifests* of both sides, so the model generates against the exact `API` the sandbox will build with; (iii) a *target-language mapping reference* of worked structural examples [13]; and (iv) the *validation-body structure reference* – the region of the validation harness below the seam marker whose structure the model’s output must imitate. The related human message is smaller – the direction and scope, the schema summary, and the input code – with the distilled feedback note of section 5.3 appended on repair attempts.

## 5.5 Structured Output Shaped to the Request

The original design requested *provider-native structured output*: the model was given a Pydantic-derived<sup>2</sup> `JSON` schema for the complete translation – all harness fields at

---

<sup>2</sup><https://pydantic.dev/docs/validation/latest/get-started/>

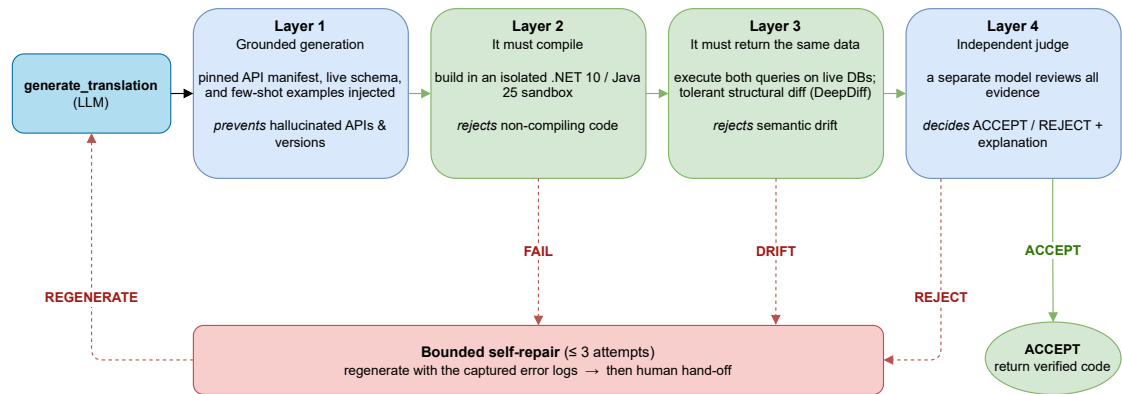
once + all finalized production-ready translations – and the provider was expected to enforce the strict schema per the OpenAI specification. The schema itself was *shaped to the request*: fields irrelevant to the requested scope (query fields for a schema-only translation, and vice versa) were pruned from the schema at run time, so the model could neither waste tokens on them nor hallucinate content for them, and cross-field validators checked, for instance, that the declared entry-point class actually appears in the submitted code. Failures of this validation were retried *in place* with the validator’s error messages fed back.

This design worked until the provider platform’s model generation changed underneath it: the newer agentic-tuned models stopped honoring large single-shot **JSON** schemas, truncating or collapsing code-sized fields (more in section 9.1). We had to redesign this interface from scratch. The generation stage requests *no* structured output, but instead, the agent is given a single state-writing tool, `save_translation`, that saves the source and target harness bodies. Saving through a tool call proved more robust than schema-constrained decoding for huge code-sized payloads. Moreover, if the agent finishes without a complete save, the node forces one final model call in which the save tool is the only permitted action, and only if that too fails does the run surface a structured-output error and divert to the human hand-off. The finalization stage went one step further: it transports code as fenced blocks in markdown, the one format the provider never truncated, and parses them deterministically. The schemas and tool definitions are reproduced in section B.1.

## 5.6 The Verification Layers

Every translation passes a stack of verification layers in which the decisive evidence is produced by *deterministic tools* – compilers, real database engines, and a structural difference algorithm (*DeepDiff*). The layers form a funnel (figure 5.1): each admits only translations that survived the previous one, each targets a distinct class of errors, and the only non-deterministic layer runs last, when everything that *can* be decided deterministically already has been. A failure at any layer does not abort the run, but rather, it is converted into structured feedback for the repair loop of section 5.8.

Each deterministic layer is, in the terms of definition 1, a node *postcondition*  $\text{post}_n$ : a check on the stage’s output that the routing function  $\delta$  reads to decide what happens next. Every distinct failure maps to a fixed, machine-decided action – another repair attempt, an outright acceptance, or a hand-off to a human once the repair budget is spent (terminal state  $\text{HANDOFF} \in F$ ). Table 5.2 enumerates that mapping; only one row (a residual value difference) ever reaches the fallible judge.



**Figure 5.1** The four verification layers as a funnel. Each layer admits only translations that survived the previous one; layers 2 and 3 are fully deterministic, and the language model appears only at the boundaries – as the generator upstream and as the interpreting judge downstream.

| Failure / outcome                                | Deterministic signal (layer)              | Route                |
|--------------------------------------------------|-------------------------------------------|----------------------|
| Uncompilable code (hallucinated API, type error) | compiler diagnostics (L2)                 | RETRY                |
| Execution error (crash, no results file)         | captured build/run log (L2)               | RETRY                |
| Result-count mismatch                            | DeepDiff countDiff (L3)                   | RETRY                |
| Row-order-only difference                        | swapped-sort re-check clears it (L3)      | ACCEPT               |
| Exact match (empty diff)                         | zero deep distance (L3)                   | ACCEPT               |
| Field/value difference in samples                | DeepDiff values_changed (L3)              | judge → ACCEPT/RETRY |
| Judge verdict on residual diff                   | ACCEPT/REJECT + reasoning (L4)            | ACCEPT/RETRY         |
| Repair budget exhausted                          | loop counter $\geq$ cap ( $\delta$ guard) | HANDOFF              |

**Table 5.2** How each verification outcome is routed. The *signal* is produced by a deterministic tool (compiler, sandbox, or DeepDiff); the *route* is fixed by  $\delta$ , not chosen by a model. RETRY feeds the signal back as repair feedback (section 5.8); HANDOFF is taken only when the repair budget is exhausted.

### 5.6.1 Compilation and Execution in Sandboxes

Every validation executes inside an isolated, disposable sandbox provisioned through the *Daytona* framework [59] – a container-in-container (*Docker-in-Docker*) environment in which only the database endpoints are reachable. Two sandbox images exist, one per

target language (a *.NET 10 SDK* image with *.NET CLF*<sup>3</sup> and a *Java 25* image with *Maven CLF*<sup>4</sup>). These are built once into named *snapshots* from which every validation starts a container, and the lifecycle logic transparently repairs stuck containers (section 5.8).

A single validation creates a fresh project directory inside the sandbox – named by thread and timestamp – containing the pinned build manifest, the assembled harness, and a generated `run.sh` that exports the database connection strings and invokes the real toolchain (`dotnet build/run`, `Maven compile-and-execute`). All files are transported base64-encoded. The harness writes its query results as a `JSON` file, which the orchestrator downloads, parses, and stores in the graph state for Layer 3; the tool’s textual verdict – a `[Validation Passed]` or `[Validation Failed]` marker with the build log – enters the isolated message channel, where it gates progression or becomes repair feedback. Executing the code, rather than inspecting it, is the standard of evidence established by execution-based benchmarks [14, 17]. The layer is *symmetric*: both the source and target harnesses are compiled and executed the same way.

## 5.7 Semantic Equivalence

Compilation proves well-formedness, but it says nothing about logical/semantic meaning. The requirement 2.5 asks whether the translated query returns *the same data* as the original, a question that is undecidable in general from the code alone, and across data models as different as `SQL` tables, document collections, and property graphs. The assistant therefore adopts the executability-oriented standard of the code-translation literature [17, 15]: run both programs on logically equivalent inputs and compare what they output, and therefore equivalence is established over *observed execution results*.

### 5.7.1 The execution and serialization contracts

Each query harness reduces a query’s result set to a compact, comparable digest with three components: the total row/document/record *count*, the *first sample* record under a deterministic ordering by a unique key, and the *last sample* record under the same ordering. Alongside the digest, each harness records the query’s native form – the generated `SQL` text, the MongoDB filter or aggregation pipeline, the rendered Cypher statement – for the judge and the human reviewer. The count catches cardinality errors, wrong joins, and mistranslated filters and the first/last samples catch field-level errors – dropped columns, wrong types, shifted values. The digest is deliberately first/last sample-based rather than full – the implications are examined in section 9.3.

---

<sup>3</sup>`mcr.microsoft.com/dotnet/sdk:10.0` (<https://hub.docker.com/r/microsoft/dotnet-sdk>)

<sup>4</sup>`bellsoft/liberica-openjdk-debian:25-cds` (<https://hub.docker.com/r/bellsoft/liberica-openjdk-debian>)

Comparing a C# object with a Java object is only meaningful if both are first projected into a common representation. The digests are therefore serialized to [JSON](#) by a *pair of mirrored serializers* – `System.Text.Json` on the .NET side and *Jackson 3* on the Java side<sup>5</sup>:

- *Property naming* is normalized to *camelCase* with a shared special-casing rule for trailing acronyms (`CustomerID` and `customerId` both serialize as `customerId`).
- *Property order* is alphabetized on both sides, so field order should not differ (but it may, which is not a problem).
- *Timestamps* are rendered in a single strict form – ISO-8601, converted to [UTC](#), exactly millisecond precision – which bridges the .NET `DateTime`, Java `Date`, and `ZonedDateTime`.
- *Decimal and floating-point values* are emitted with a fixed three-decimal scale (half-up rounding on the Java side), bridging [SQL](#) decimal, C# decimal, and Java `BigDecimal`.
- *Enumerations* are serialized as their *camelCased* names, and *reference cycles* between navigation properties are suppressed.

The fixed tolerances are a deliberate simplification, and they cut both ways: the three-decimal scale can be inappropriately *lenient* for monetary values (a genuine sub-cent discrepancy is rounded away) and inappropriately *strict* for accumulated floating-point aggregates (a harmless last-place difference is flagged), and the millisecond timestamp normalization can likewise mask a genuine sub-tolerance drift. The tolerances are constant across all runs and systems so that no arm is advantaged, but they remain a construct-validity threat that section 9.3 revisits.

### 5.7.2 Background: structural diffing and the deep distance metric

The comparison engine is *DeepDiff* [33]<sup>6</sup>, a structural difference algorithm for arbitrarily nested object trees. Given two objects, it produces a *diff report*: a set of typed change records – `values_changed`, `type_changes`, `dictionary_item_added/removed`, `iterable_item_added/removed`, `repetition_change` – each addressed by the path of the affected element (e.g. `root['orders'][0]['unitPrice']`). An empty report means the objects are equal. For the reader's reference, algorithm 1 sketches the algorithm in the order-insensitive mode the assistant uses: mappings are compared key by

---

<sup>5</sup><https://learn.microsoft.com/en-us/dotnet/api/system.text.json> and <https://github.com/FasterXML/jackson>.

<sup>6</sup><https://zepworks.com/deepdiff/current/>

key, scalars up to a configured number of significant digits, and unordered collections are matched in two phases – first exactly, by hashing every item with a structural hash (*DeepHash*<sup>7</sup>), then approximately, by pairing the remaining items so that the closest items are compared with each other rather than positionally.

---

**Algorithm 1** Order-insensitive structural diff (DeepDiff, simplified)

---

**Require:** objects  $t_1, t_2$ ; significant digits  $s$

**Ensure:** diff report  $\Delta$  (set of typed, path-addressed change records)

```

1: function DIFF( $t_1, t_2, path$ )
2:   if type( $t_1$ )  $\neq$  type( $t_2$ ) then
3:     report type_change at  $path$ ; return
4:   end if
5:   if  $t_1, t_2$  are scalars then
6:     if  $t_1 \neq t_2$  up to  $s$  significant digits then
7:       report values_changed at  $path$ 
8:     end if
9:   else if  $t_1, t_2$  are mappings then
10:    report keys in  $t_2 \setminus t_1$  as added, in  $t_1 \setminus t_2$  as removed
11:    for common key  $k$  do
12:      DIFF( $t_1[k], t_2[k], path.k$ )
13:    end for
14:   else ..... unordered iterables
15:      $H_i \leftarrow$  multiset of DeepHash( $x$ ) for  $x \in t_i, i \in \{1, 2\}$ 
16:     cancel items with equal hashes; report count mismatches as repetition_
change
17:     pair remaining items of  $t_1 \times t_2$  by minimal pairwise deep distance
18:     for each pair  $(x, y)$  do
19:       DIFF( $x, y, path$ )
20:     end for
21:     report unpaired items as iterable_item_added/removed
22:   end if
23: end function

```

---

Alongside the report, DeepDiff can compute a scalar summary of *how different* the two objects are – the *deep distance*.

**Definition 3** (Deep distance). Let  $\Delta(t_1, t_2)$  be the diff report of two objects and let  $\|t\|$  denote the number of atomic elements (scalars, keys, and container slots) that make up an

---

<sup>7</sup><https://zepworks.com/deepdiff/current/deephash.html>

object  $t$ . The deep distance of  $t_1$  and  $t_2$  is

$$\text{dist}(t_1, t_2) = \frac{|\text{ops}(\Delta(t_1, t_2))|}{\|t_1\| + \|t_2\|},$$

where  $|\text{ops}(\Delta)|$  counts the edit operations in the report (for pairs of numeric leaves, their normalized numeric distance is used instead of a unit count). The measure is a structural analogue of the Levenshtein edit distance, normalized so that  $\text{dist} \in [0, 1]$ , with  $\text{dist}(t_1, t_2) = 0$  if and only if the report is empty.<sup>8</sup>

### 5.7.3 Tolerant comparison and the swapped-sorting check

The two JSON digests are compared per query using DeepDiff. The counts are compared exactly – there is no tolerable reason for two equivalent queries to disagree on cardinality. Next, the first/last samples are compared: container ordering is ignored (`ignore_order`, with repeated items still reported via `report_repetition`), numeric values are compared to three significant digits (`significant_digits=3`) – matching the serializers’ fixed scale – and the two pairing heuristics’ cut-offs (`cutoff_distance_for_pairs` and `cutoff_intersection_for_pairs`) are raised to 1, so that no candidate pair of deviating items is ever excluded from comparison by a heuristic shortcut. The comparison computes the *deep distance* of definition 3, and a distance of exactly zero is accepted as equality. Algorithm 2 summarizes the procedure; there,  $\text{dist}(\Delta)$  denotes the deep distance recorded in the diff report  $\Delta$ .

---

**Algorithm 2** Sorting-tolerant equivalence check for one query

---

**Require:** source digest  $S = (c_S, \text{first}_S, \text{last}_S)$ , target digest  $T = (c_T, \text{first}_T, \text{last}_T)$

**Ensure:** EQUIVALENT, or a structural diff report

```

1:  $\Delta_c \leftarrow \text{DeepDiff}(c_S, c_T)$  ..... counts: exact comparison
2:  $\Delta_f \leftarrow \text{DeepDiff}(\text{first}_S, \text{first}_T)$ ;  $\Delta_l \leftarrow \text{DeepDiff}(\text{last}_S, \text{last}_T)$  ... order-insensitive, 3
   significant digits
3: if  $\Delta_c = \emptyset \wedge \text{dist}(\Delta_f) = 0 \wedge \text{dist}(\Delta_l) = 0$  then
4:   return EQUIVALENT ..... same order
5: end if
6:  $\Delta'_f \leftarrow \text{DeepDiff}(\text{first}_S, \text{last}_T)$ ;  $\Delta'_l \leftarrow \text{DeepDiff}(\text{last}_S, \text{first}_T)$ 
7: if  $\Delta_c = \emptyset \wedge \text{dist}(\Delta'_f) = 0 \wedge \text{dist}(\Delta'_l) = 0$  then
8:   return EQUIVALENT ..... reversed order
9: end if
10: return report  $(\Delta_c, \Delta_f, \Delta_l)$ 
```

---

<sup>8</sup>The DeepDiff documentation notes that the exact formula is subject to change and recommends deep distance for *relative* comparison; the equivalence check therefore thresholds only at exactly zero and never ranks by magnitude.



Note that the harnesses impose a deterministic ordering so that “first” and “last” are well-defined, but the two sides may realize that ordering in opposite directions – an ascending index scan on one engine, a descending one on the other. If the source’s first sample equals the target’s *last* and vice versa (with the same tolerances), the results are the same set traversed in the opposite direction, and the queries are declared equivalent. There is a risk that a non-unique sort key can make the samples themselves nondeterministic across engines – this is mitigated by the harness convention of sorting on a unique key wherever one exists, and revisited in section 9.3.

#### 5.7.4 A worked example

To make it concrete, consider one query of an EF Core to Spring Data MongoDB run against the *WideWorldImporters* dataset: all order lines whose `pickingCompletedWhen` is inside a given date window. Listings 5.1 and 5.2 show the two digests as the harnesses actually emit them (showing only four fields for brevity). Note that each side also records the query’s *native* form – the SQL text that EF Core generated on the source side, the MongoDB filter document on the target side – which the comparison ignores but the judge and the human reviewer see.

```

1 { "query1": {
2   "count": 1883,
3   "firstSample": {
4     "description": "Tape dispenser (Red)",
5     "orderLineId": 33036,
6     "pickingCompletedWhen": "2014-12-29T11:00:00.000Z",
7     "unitPrice": 32, "...": "..."},
8   "lastSample": {
9     "description": "Air cushion film 200mmx200mm 325m",
10    "orderLineId": 128234,
11    "pickingCompletedWhen": "2014-12-30T11:00:00.000Z",
12    "unitPrice": 90, "...": "..."},
13  "sqlString": "SELECT [o].[OrderLineID], ... FROM
    ↳ [Sales].[OrderLines] AS [o] WHERE [o].[PickingCompletedWhen] >=
    ↳ @from AND [o].[PickingCompletedWhen] <= @to" } }
```

**Listing 5.1** The source-side (.NET, EF Core) result digest for one query: exact count, boundary (first, last) samples under the same serialization, and the query’s native SQL form.



```

1 { "query1": {
2   "count": 1883,
3   "firstSample": {
4     "description": "Tape dispenser (Red)",
5     "id": "69d7d2bf7d34d59f1cfbbaf1",
6     "orderLineId": 33036,
7     "pickingCompletedWhen": "2014-12-29T12:00:00.000Z",
8     "unitPrice": 32, "...": "..." },
9   "lastSample": {
10    "description": "Air cushion film 200mmx200mm 325m",
11    "id": "69d7d2c77d34d59f1cfd429d",
12    "orderLineId": 128234,
13    "pickingCompletedWhen": "2014-12-30T12:00:00.000Z",
14    "unitPrice": 90, "...": "..." },
15   "mongoQuery": {
16     "collection": "orderLines",
17     "filter": { "pickingCompletedWhen": {
18       "$gte": "2014-12-20T00:00:00.000Z",
19       "$lte": "2014-12-31T00:00:00.000Z" } } } } }

```

**Listing 5.2** The target-side (Java, Spring Data MongoDB) result digest for the same query: identical count and samples, plus the document identifier introduced by the [ETL](#) step and the query’s native MongoDB filter form.

Running algorithm 2 on this pair yields the report in listing 5.3: the counts agree exactly, but both boundary samples differ – the target documents carry an `id` field that does not exist on the relational side (`dictionary_item_added`), and the two timestamp fields are shifted by exactly one hour (`values_changed`), yielding a deep distance of about 0.058 per sample. Because the distance is non-zero in both the direct and the swapped comparison, the queries are *not* declared equivalent. The report is recorded in the state and rendered for the judge, whose task is now exactly the interpretation problem of the [LLM-as-a-judge](#) Evaluator: “The added `ObjectId` is an artifact of the [ETL](#) data population and arguably harmless, whereas the systematic one-hour shift indicates a genuine timezone-handling discrepancy in the translation that must be repaired.” This interpretation is plausible but not certain, and the uncertainty runs both ways: the one-hour offset could instead be a benign artifact of how the two engines render a boundary value, and – more dangerously – an offset the judge waved through as harmless could conceal a real timezone defect. The judge’s authority to *accept* a non-equivalent result is therefore itself a source of error, which is why strict equivalence (never routed through the judge) is the headline metric of chapter 8 and the judge is scored against deterministic ground truth rather than trusted; section 9.3 treats this as

a construct-validity threat. The full digests and report for this example are reproduced in section B.4.

```
1 { "query1": {
2   "status": "Differences Found",
3   "diffs": {
4     "deepdiff_mapping": { "old": ".NET Entity Framework Core",
5                           "new": "Java Spring Data MongoDB" },
6     "countDiff": {},
7     "firstSampleDiff": {
8       "dictionary_item_added": [ "root['id']" ],
9       "values_changed": {
10        "root['lastEditedWhen']": {
11          "new_value": "2014-12-29T12:00:00.000Z",
12          "old_value": "2014-12-29T11:00:00.000Z" },
13        "root['pickingCompletedWhen']": {
14          "new_value": "2014-12-29T12:00:00.000Z",
15          "old_value": "2014-12-29T11:00:00.000Z" } },
16     "deep_distance": 0.0577 },
17     "lastSampleDiff": { "...": "...",
18       "deep_distance": 0.0577 } } } }
```

**Listing 5.3** The equivalence report produced for the digests of listings 5.1 and 5.2: counts agree, the target adds a document identifier, and both timestamps are shifted by one hour.

## 5.8 Bounded Loops, Retries, and Fallbacks

LLM code generation fails in characteristic ways – invented APIs, type mismatches, malformed structured output [16]. The workflow mitigates them through four nested layers, each bounded by a small constant, ordered from the innermost outward. At the innermost layer, a model whose structured response fails schema validation is re-invoked *in place* with a concise description of exactly what was wrong. At the second layer, every external tool invocation – sandbox provisioning, compilation, the equivalence check – is retried with exponential backoff against transient infrastructure errors, and a terminal tool failure is converted into a structured failure message, so the run continues and the failure becomes repair feedback. At the third layer, each generative stage carries an ordered *fallback chain* of alternative models: a model that is unavailable or persistently erroring is replaced by the next entry, ending at a locally served model that keeps the assistant alive even if the last models are not that performant. At the outermost layer there are the loop thresholds already described: a fixed number of extraction attempts and a fixed number of translation-repair attempts.

The stages described in section 4.3 are connected so that every verification failure feeds back into a new generation attempt, making the workflow a *self-repairing* system in the sense of Chen et al. [28]: rather than discarding a failed attempt, the model is shown the consequences of its output – compiler diagnostics, the equivalence diff, the judge’s verdict/reasoning – and asked to repair itself. Execution feedback of this kind has been shown to improve generated-code accuracy substantially over single-shot generation [28].

# Chapter 6

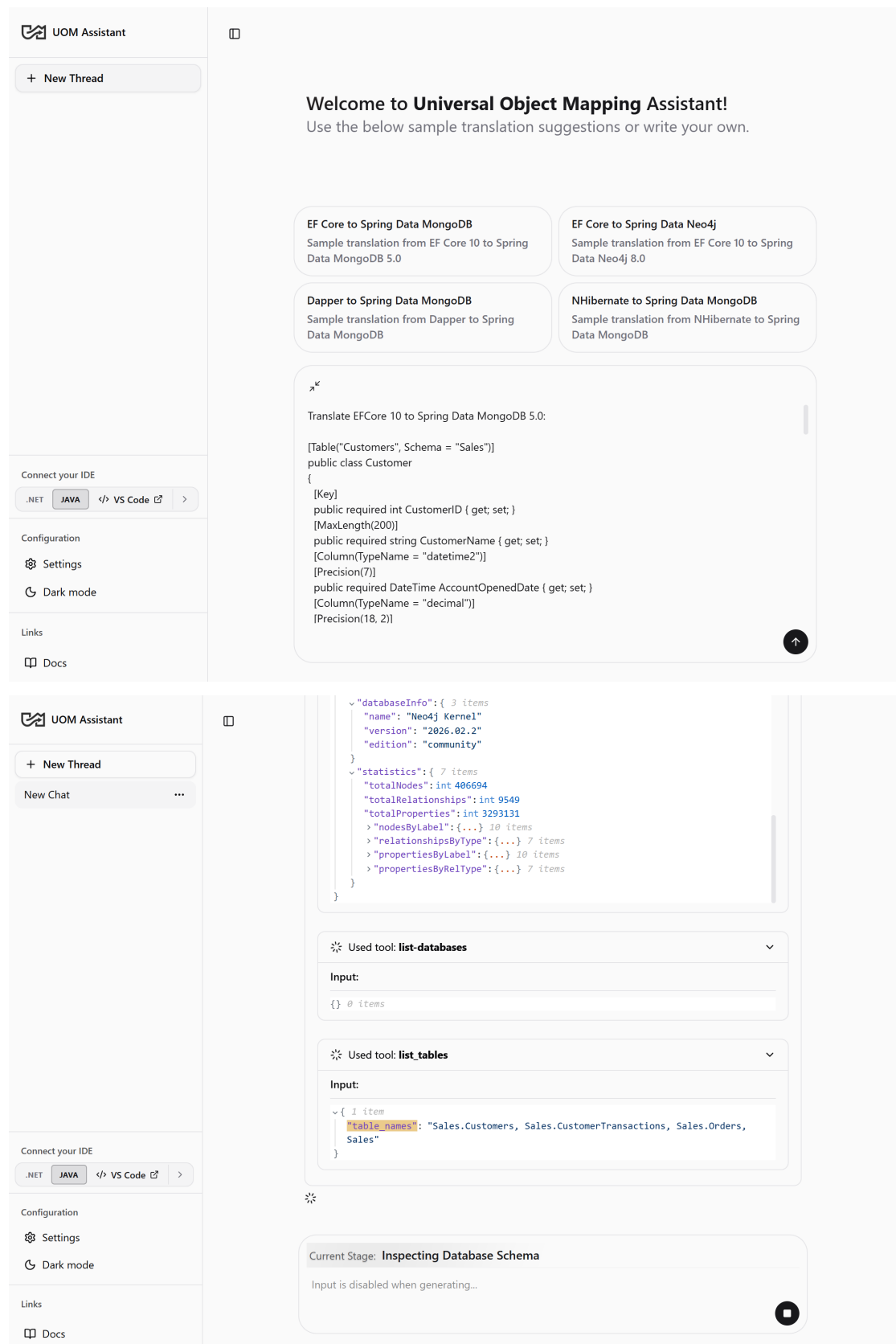
## Input and User Interaction

This chapter describes the assistant from the user’s side, addressing the usability and observability goal [G4](#). It is delivered as an interactive web application in which a user submits source artifacts, follows the translation stage by stage, inspects every intermediate result, and intervenes when the workflow asks. Because the assistant is deliberately *not* a general-purpose chatbot (non-goal [N4](#)), the quality of a run depends on the quality of the request that starts it, so a substantial part of this chapter covers how an input message should be authored. It closes with the programmatic interfaces – [Representational State Transfer \(REST\)](#), [Model Context Protocol \(MCP\)](#), and [Agent Client Protocol \(ACP\)](#) – through which the same pipeline is reachable from [IDEs](#) and other agents.

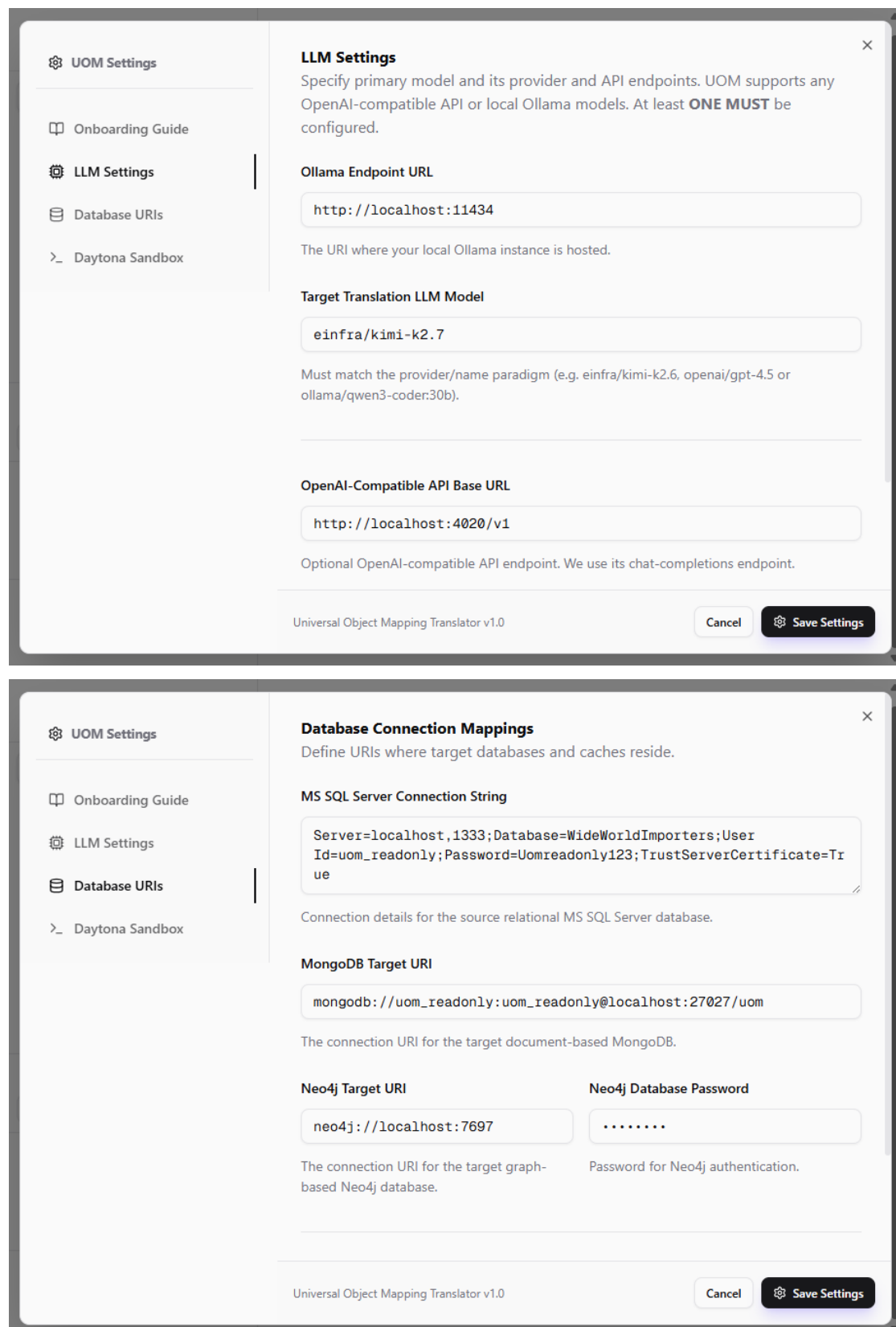
### 6.1 The Workspace

The entry point is a conversational *translation workspace* (figure [6.1](#)). A sidebar manages *threads* – each a migration session backed by its own checkpointed graph state, revisitable, archivable, or deletable; reopening one restores its conversation and generated artifacts from the checkpoint history. The main area holds the message thread and, at the bottom, the *composer* where the user authors the request. The workspace is intentionally chat-shaped, but the conversation it hosts is the structured, single-purpose protocol of section [6.2](#).

A first-run onboarding dialog collects the configuration the pipeline depends on, editable later in a settings panel (figure [6.2](#)): the language-model backend (e-INFRA endpoint and credentials or a local Ollama host, with a model selection), the three database connections (source [SQL](#) Server, target MongoDB and Neo4j), and the Daytona sandbox endpoint with its region and compilation timeout. It persists client-side and is injected per run, so different users within a single deployment can point the same pipeline at different databases and models.



**Figure 6.1** The translation workspace: the thread sidebar on the left, the suggestion cards for pre-configured framework pairs, and the composer holding a source schema and query ready for submission. Upon running the request, the thread shows the live progress of the workflow and the intermediate artifacts it produces (reasoning, outputs, tool calls, etc.).



**Figure 6.2** The configuration panel: first, the language-model backend selection, and second, the database connections. The values are persisted client-side and passed to the orchestrator with every run.

## 6.2 Anatomy of a Request

The first message of a thread is the sole input from which the extraction stage (section 4.3.1) recovers every parameter the pipeline needs. A well-formed request supplies four things.

1. *The translation direction.* Both frameworks must be identifiable – by name, ideally with versions (“Translate from Entity Framework Core 10 to Spring Data MongoDB 5.0”)<sup>1</sup>. The extractor matches against the supported framework list; an unsupported or missing framework ends the run with an explanation rather than a guess.
2. *The scope.* Whether the request concerns the *schema*, the *queries*, or *both*. The scope need not be stated explicitly – the extractor classifies it from what code is present – but stating it removes a degree of freedom.
3. *The schema code*, when in scope: the entity classes together with the mapping configuration that gives them meaning – attributes and the context class for Entity Framework Core, mapping-by-code classes for NHibernate, plain data classes for Dapper. Supplying entities *without* their mapping forces the model to guess table and column names it could have been told.
4. *The query code*, when in scope: the query methods, preferably as complete methods rather than bare expressions, so that parameter types, return types, and method names – all of which the translation preserves – are unambiguous.

Code should be pasted inside fenced blocks (not necessary, but recommended), exactly as it appears in the source project; the extractor preserves formatting line by line, and the original indentation survives into the validation harness. Everything the request does *not* say is either observed (the database shapes come from live inspection, not from the prompt) or fixed by configuration (versions default to the pinned ones of section 2.3), so the user is never obliged to describe their database – only their code.

When a request is incomplete or unintelligible, the assistant does not improvise: the extraction stage re-reads the conversation a bounded number of times, carrying over fields already recovered, and terminates with a message naming what was missing if it cannot recover the required ones. Malformed input therefore costs seconds, not a run of tens of minutes that validates the wrong thing.

---

<sup>1</sup>Based on section 2.3, we support only newest specific versions; this is to ensure the generator queries framework documentation for specific versions, but it is not tested further in this thesis.

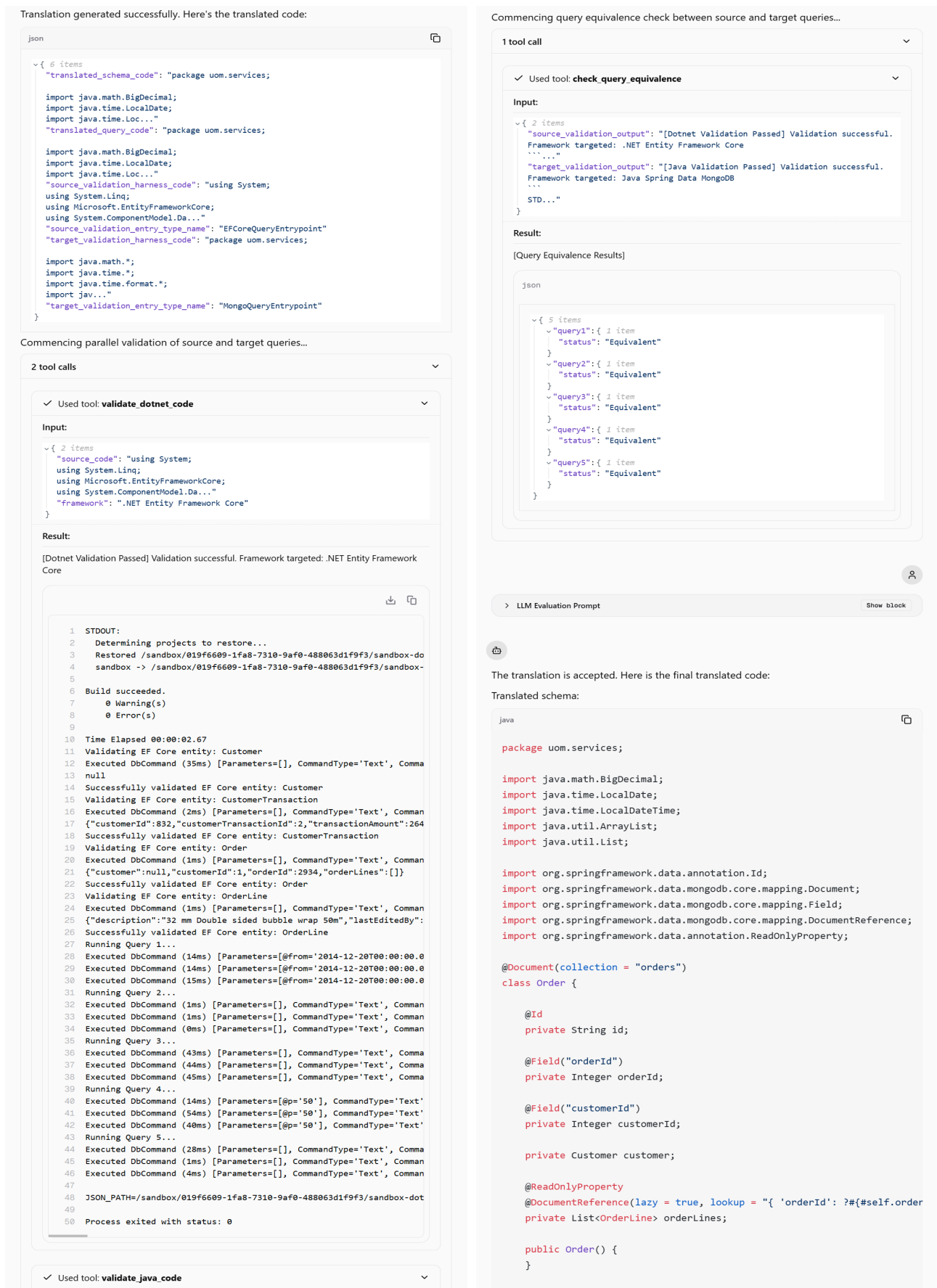
## 6.3 Ready-Made Pairs and Custom Configuration

For a new thread the workspace offers four *suggestion cards*, one per representative migration scenario over the *WideWorldImporters* dataset [40]: [EF Core to Spring Data MongoDB](#), [EF Core to Spring Data Neo4j](#), [Dapper to Spring Data MongoDB](#), and [NHibernate to Spring Data MongoDB](#). Clicking a card loads a complete, well-formed request – direction, schema, and queries – into the composer, where it can be submitted as-is or edited. The cards serve two purposes: they are live documentation of what a good request looks like (each is a worked instance of section 6.2), and they correspond to the migration scenarios of the experimental evaluation, so a reader can reproduce an evaluation run with one click. Nothing restricts the user to the cards: any code in a supported source framework may be pasted, provided the configured databases hold the data the queries address – for custom schemas, the one-time [ETL](#) population step of section 2.3 must have been run for the relevant entities.

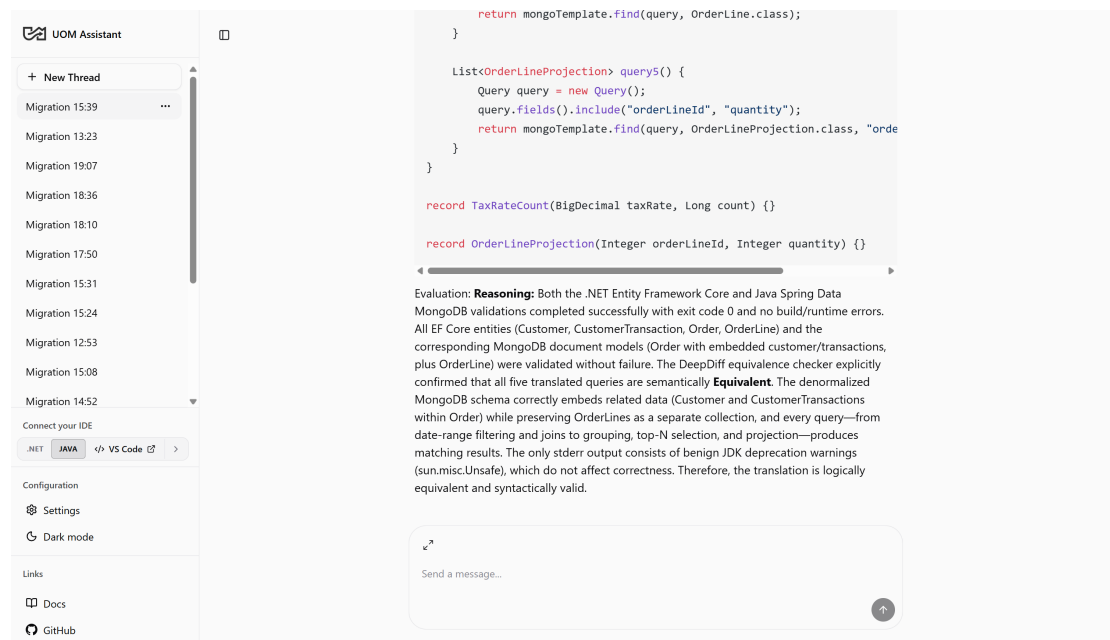
## 6.4 Following a Run

Observability is a design goal: every state transition of the graph is streamed to the client over [Server-Sent Events \(SSE\)](#) or WebSockets, and the interface renders the run as it happens (figure 6.3). A progress banner names the active stage – extracting input, inspecting the database schema, translating, validating, evaluating – and each stage deposits its artifacts into the thread as it completes. The intermediate artifacts are deliberately not hidden: the extracted structured input is shown as data; every database-inspection tool call and its result can be expanded; the schema summary handed to the generator is readable in full; the sandbox compilation logs stream live from both containers; and the equivalence stage renders the *DeepDiff* report for every query. Also, the user is shown the exact prompts (not system prompts) used in each stage. A user who doubts the result can thus trace exactly which evidence produced it – the interface-level counterpart of the trust argument of section 5.6 – and the same events are captured by the tracing backends for offline audit (section 7.5). Upon successful translation, the user receives the translated schema and queries along with a validation summary (figure 6.4).





**Figure 6.3** Following a run: live sandbox compilation and execution logs of the validation stage (left) and the rendered DeepDiff equivalence report with the judge’s evaluation and final translation (right).



**Figure 6.4** A completed run: the finalized, validated target schema and query code as delivered to the user.

## 6.5 The Human-in-the-Loop Hand-off

When the repair loop exhausts its budget (section 4.3.7), the graph suspends and the thread displays an *execution-suspended* card – the interface realization of the [Human-in-the-Loop \(HITL\)](#) stage – presenting the state a reviewer needs: the judge’s explanation of the last rejection, the equivalence diff when one exists, and the current validated artifacts. *Accept & Save* resumes toward finalization, appropriate when the flagged difference is immaterial or the reviewer prefers to fix a detail by hand; *Reject & Correct* opens a free-text field for line-level correction pointers – a missing getter, a wrong relationship direction – and resumes into a fresh generation attempt with the feedback injected and the retry budget renewed. Because the suspension lives in the checkpointed state, it survives page reloads and thread switches, waiting indefinitely for its reviewer. Every hand-off is also a trace event, which is how the evaluation counts the manual-intervention metric of requirement 2.5.

## 6.6 IDE Integration

A translation is ultimately code, and two paths connect the assistant to an [Integrated Development Environment \(IDE\)](#). The lighter one goes through the validation sand-

boxes: each is a persistent, inspectable workspace, and *Connect Your IDE* opens the running sandbox in *Visual Studio Code*<sup>2</sup> or *Cursor*<sup>3</sup>. There the user finds the project exactly as validated – source file, pinned manifest, a `run.sh` reproducing compilation and execution – plus a generated README with the database connection details, recommended extensions, an `AGENTS.md`, and a pre-wired MCP configuration, so an AI coding assistant in the IDE can immediately talk to the orchestrator and databases. Re-running and hand-editing the validated harness in place is thus easier for the user.

The heavier path embeds the translator inside an IDE-native *coding agent*, described next.

## 6.7 MCP and ACP Protocol Interfaces

Beneath the web interface, the orchestrator is an ordinary REST service: the LangGraph API server exposes thread, run, and streaming endpoints, and the frontend is only one client of them<sup>4</sup>. Two protocol layers make the same capability available to programmatic clients.

First, the orchestrator is itself exposed as an MCP server [57], so any MCP-capable client – an IDE assistant, a desktop agent – can invoke the translation pipeline as a tool, alongside the database-inspection tools the pipeline itself consumes. This is the same protocol the assistant uses internally for schema inspection and documentation search (section 5.3), applied in the opposite direction.

Second, for full IDE embedding the project provides an agent built on the LangChain’s *DeepAgents* framework<sup>5</sup> and served over the *Agent Client Protocol* (ACP)<sup>6</sup>. This *deep agent* wraps the translation graph of chapter 4 as a subagent: a conversational coordinator with filesystem access to the user’s workspace routes ordinary requests locally and delegates translation requests (the user’s code passed verbatim) to the same validated pipeline. A composite backend maps path prefixes to execution environments, so the agent reads the user’s project from the local filesystem while compilation under the sandbox prefixes runs inside the respective Daytona containers. An editor-native agent can thus take a file from the open project, run it through the full validated translation, and write the result back – without the web interface at all. The deep agent is an integration layer only: correctness still comes exclusively from the graph it delegates to. Note that this *UOM Deep Agent* assistant has not been systematically evaluated; it is part of the future work (section 9.4).

---

<sup>2</sup><https://code.visualstudio.com/>

<sup>3</sup><https://cursor.com/>

<sup>4</sup>The frontend proxies all requests through a server-side passthrough that injects credentials, so the browser never holds backend secrets.

<sup>5</sup><https://docs.langchain.com/oss/python/deepagents/overview>

<sup>6</sup>An open protocol for connecting coding agents to editors: <https://agentclientprotocol.com/>.

# Chapter 7

## Architecture and Implementation

The translation workflow of chapter 4 runs inside a considerably larger system: a web frontend, an orchestration service, protocol servers, sandbox infrastructure, five databases, and two language-model providers, all of which must be provisioned, connected, and observable for a single translation to succeed. This chapter descends through the abstraction levels of the C4 model [61] – system context, containers, and the components of the two containers this thesis contributes – and closes with the deployment topology, the observability stack, and the extensibility argument promised by goal G3. Verbatim implementation details (schemas, prompts, harness preludes) live in chapter B; the full developer documentation is published with the project<sup>1</sup> [45].

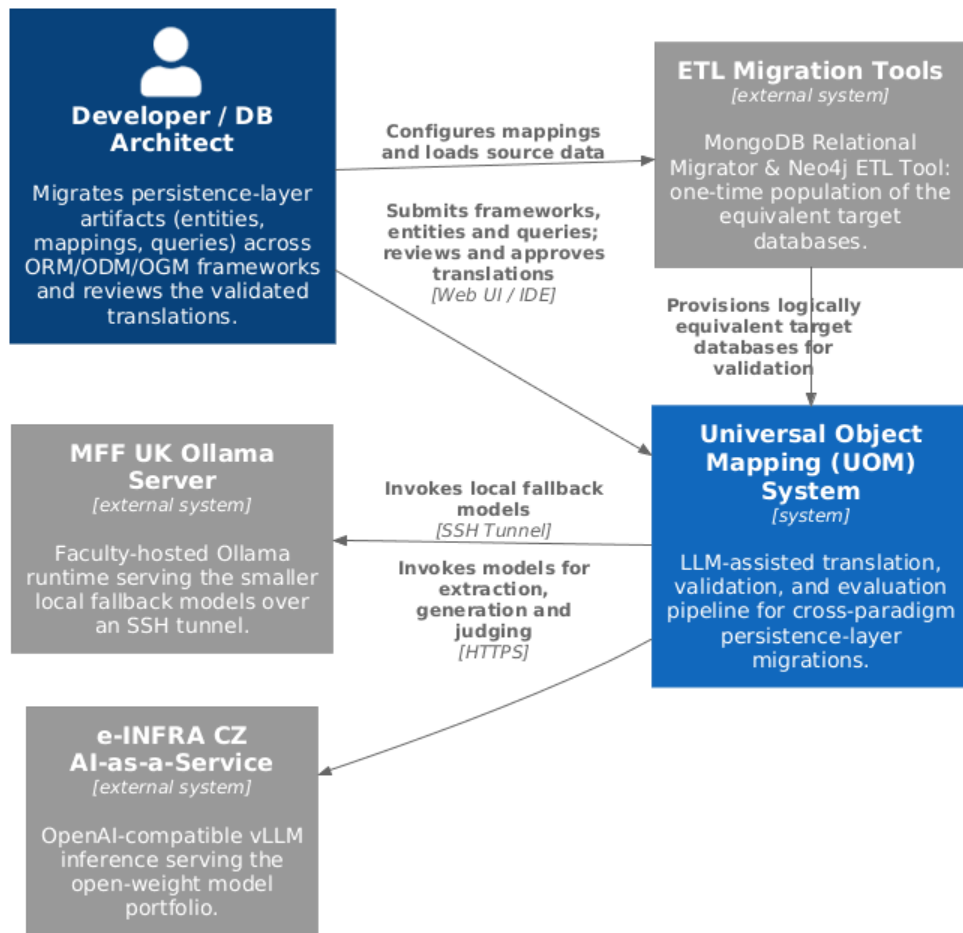
### 7.1 System Context

Figure 7.1 places the assistant in its environment. The system serves one kind of user – a developer or database architect migrating a persistence layer – and depends on three external systems. First, the language-model inference infrastructure: the e-INFRA CZ AI-as-a-service platform [48], serving the open-weight portfolio of section 4.1.2 behind an OpenAI-compatible API, and a faculty-hosted Ollama server<sup>2</sup> reached over an SSH tunnel for the smaller local fallback models. Second, the ETL tooling – MongoDB Relational Migrator [37] and the Neo4j ETL Tool [38] – for the one-time data-population step that creates logically equivalent target databases (non-goal N1); the migrator is containerized alongside the system for reproducibility. Third, the user’s environment: a browser and, optionally, an IDE connected through the protocol interfaces of section 6.7.

---

<sup>1</sup><https://uom-demo.vercel.app/docs>; Mirror: <https://uom-cd0f649f.mintlify.app>

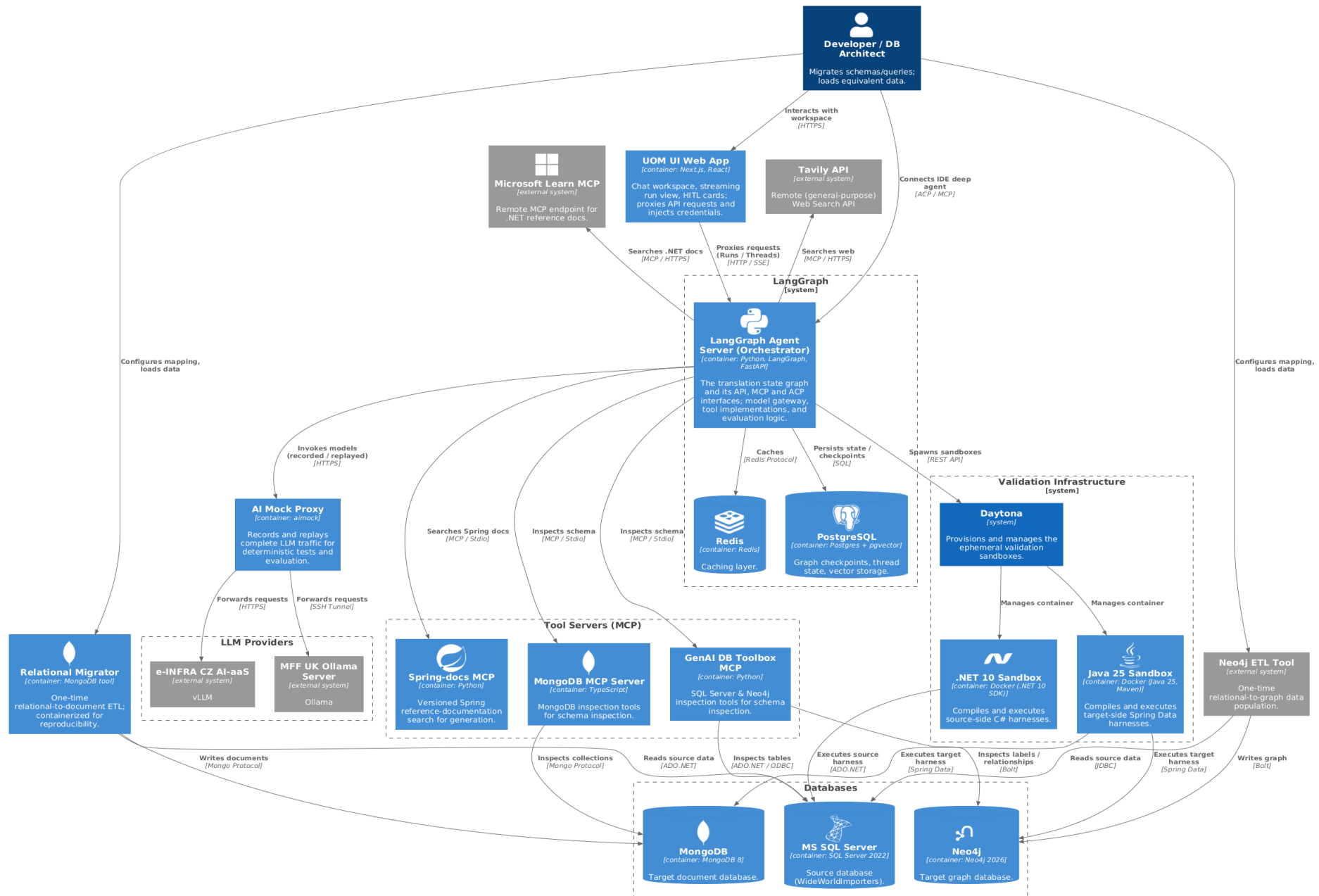
<sup>2</sup>Ollama, a local model-serving runtime: <https://ollama.com/>.



**Figure 7.1** The C4 system-context view. A single kind of user – the developer or database architect – drives the UOM system, which depends on three external systems: the two language-model providers (e-INFRA CZ and the faculty Ollama server) and the ETL tooling that populates the logically equivalent target databases used for validation.

## 7.2 Container View

Figure 7.2 shows the deployable containers, their technologies, and the connections among them. All run under a single Docker Compose orchestration, and the production deployment mirrors the same topology (section 7.6). The layout makes the central role of the orchestrator visible at a glance: aside from the frontend proxy and the language-model providers, every arrow originates there. Three groups of containers deserve commentary.



**Figure 7.2** The C4 container view of the UOM system. The orchestrator is the only container that talks to everything else: it invokes the language-model providers (through the recording proxy), consumes the **MCP** tool servers, commands the Daytona daemon that provisions the two language sandboxes, and persists its state in PostgreSQL and Redis. The three **MCP** servers, the sandbox infrastructure, and the five databases are grouped by boundary; the containerized Relational Migrator and the external Neo4j ETL Tool perform the one-time data population that produces the equivalent target databases, and Microsoft Learn is consumed as a remote **MCP** endpoint.

*The orchestrator* is the contribution of this thesis and the subject of section 7.3, and it is the only container that talks to everything else.

*The tool servers* exist because the assistant deliberately accesses every external capability through the Model Context Protocol [57] rather than through bespoke integrations. Database inspection is served by two dedicated MCP containers – a database toolbox exposing SQL Server and Neo4j inspection tools<sup>3</sup> and the MongoDB MCP server<sup>4</sup> – and documentation search by a community-based Spring-documentation MCP server [62], with Microsoft Learn consumed as a remote MCP endpoint<sup>5</sup>. The uniform protocol is what makes the tool surface of each stage a configuration decision rather than a code change (section 5.3). Standardizing on one protocol buys *interoperability and reuse* – any conformant server, including third-party ones, drops in without bespoke glue [57] – but it also widens the *trust surface* and adds *protocol overhead*: every connected server is a place a prompt-injection or an over-broad credential could leak [63], and each call adds a serialization hop.<sup>6</sup> The assistant therefore pins each server, runs it inside the project’s own boundary, and grants each stage only the tools it needs.

*The validation infrastructure* is the Daytona daemon and the two sandbox containers it manages (section 5.6.1). It superseded an earlier pair of statically built .NET and Java validation microservices, provisioning sandboxes dynamically from declarative images and providing the isolation and lifecycle management the static services would have had to reimplement.

The remaining containers in figure 7.2 are supporting infrastructure. Five databases hold the system’s state and its data: PostgreSQL stores the graph checkpoints, thread state, and vector embeddings, Redis provides the caching layer, and the SQL Server source together with the MongoDB and Neo4j targets are the databases the translated code is validated against (section 7.3). The containerized Relational Migrator and the external Neo4j ETL Tool perform the one-time relational-to-document and relational-to-graph population of those targets, and a recording proxy in front of the language-model providers captures and replays model traffic to give the tests and the evaluation harness deterministic fixtures (section 7.5).

## 7.3 The Orchestrator

The orchestrator is described in chapter 4. Its internal structure separates five concerns. The C4 Component view of the orchestrator is shown in figure 7.3.

---

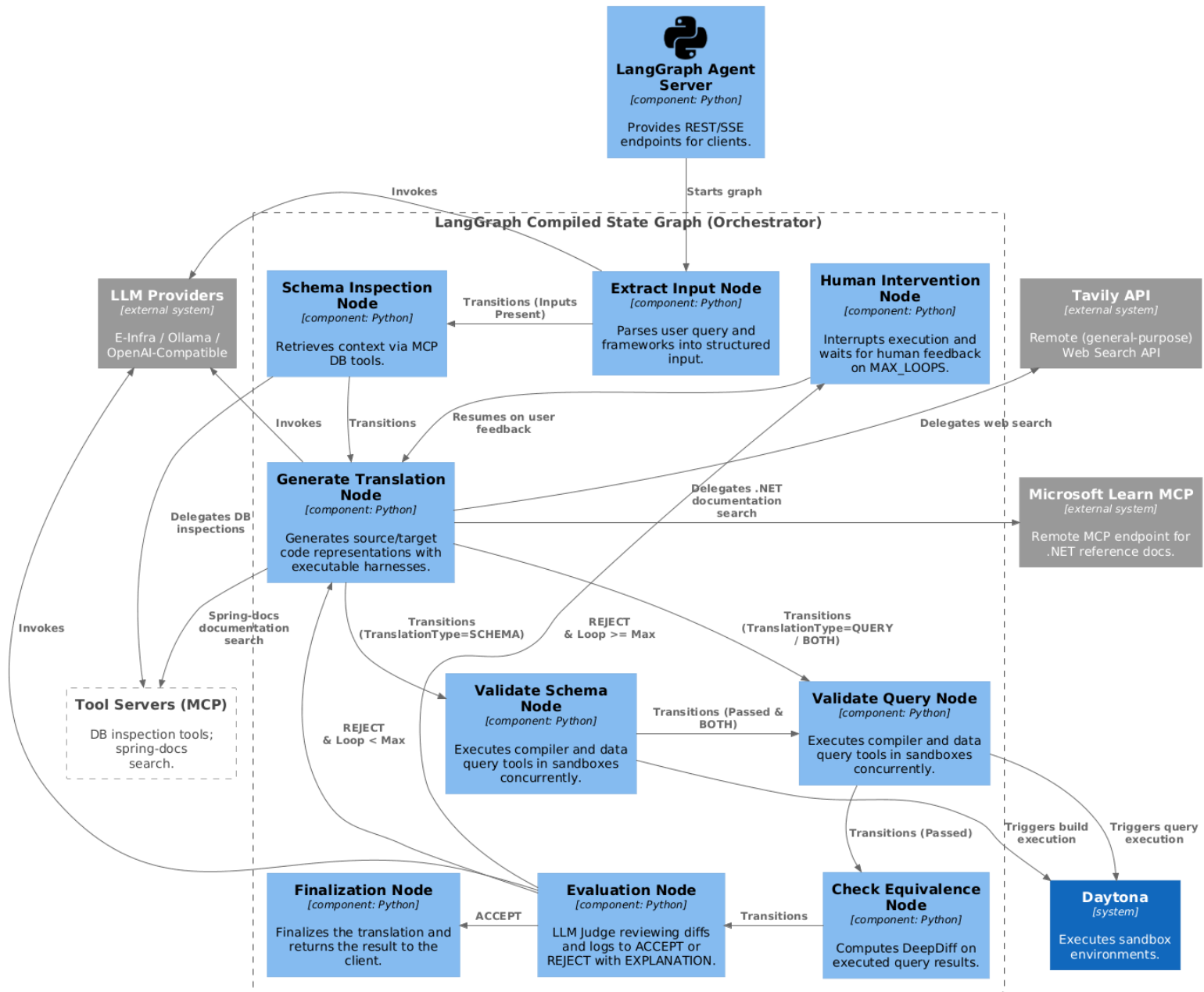
<sup>3</sup><https://github.com/googleapis/genai-toolbox>

<sup>4</sup><https://github.com/mongodb-js/mongodb-mcp-server>.

<sup>5</sup><https://learn.microsoft.com/en-us/training/support/mcp>.

<sup>6</sup><https://www.pillar.security/blog/the-security-risks-of-model-context-protocol-mcp>





**Figure 7.3** The C4 component view of The Orchestrator. For complete component descriptions, see chapter 4.



*The graph definition* declares the nodes, conditional edges, and per-node retry and cache policies, and compiles them into the executable state machine. Expensive idempotent work is cached – the schema-inspection result, for instance, with a short time-to-live, so a repair loop does not re-inspect an unchanged database.

*The state and context modules* define the typed state of section 5.2 and a separate run-scoped *context*: per-invocation configuration – database connection strings, the Daytona endpoint and timeout, model-provider credentials, and the evaluation overrides (generation-model override and single-pass flag) used by the experiment harness. Holding configuration in the context, not the state, keeps a run’s parameters immutable while its state evolves and lets one graph serve different users, databases, and models concurrently.

*The model gateway* resolves logical model names to provider endpoints. One factory constructs all chat models from a declarative model-profile registry – context-window sizes, token limits, reasoning-mode support – behind a unified interface<sup>7</sup>, so e-INFRA- and Ollama-served models are interchangeable at every call site. Each generative stage’s middleware – retry, fallback chains, context editing, summarization, structured-output repair, and the reasoning-content sanitizer of section 5.8 – is assembled around this gateway per node, so each stage carries exactly the resilience it needs.

*The tool implementations* back the deterministic stages: the two sandbox validators, the equivalence checker, and the sandbox lifecycle manager (section 5.6), together with loaders that materialize the MCP tool connections per run.

*The interface layer* exposes the graph in three ways: the LangGraph API server publishes REST and SSE endpoints for threads, runs, and streaming<sup>8</sup>; an MCP endpoint republishes the pipeline as a tool; and the ACP server wraps it as a subagent of the IDE-facing deep agent (section 6.7). All three converge on the same compiled graph, so every client receives identically validated translations.

Graph checkpoints are persisted in PostgreSQL, which is what makes threads resumable, the HITL suspension durable, and past runs inspectable; Redis provides the caching layer.

## 7.4 The Frontend

The web application is a Next.js<sup>9</sup> application built on the assistant-ui component library<sup>10</sup> with a LangGraph SDK client. Its architecture is deliberately thin: a server-side passthrough proxy intercepts all API calls, injects credentials, and shields backend

---

<sup>7</sup>Provider access is unified through LiteLLM: <https://docs.litellm.ai/>.

<sup>8</sup>LangGraph Server: <https://docs.langchain.com/langgraph-platform/langgraph-server>.

<sup>9</sup><https://nextjs.org/>

<sup>10</sup><https://www.assistant-ui.com/>

URLs, so the browser never holds secrets; a client-side graph-state context synchronizes the streamed run events with the rendered thread; and specialized components render the artifacts of each stage – expandable tool calls, streaming compiler logs, the DeepDiff report, and the suspension card of section 6.5. The frontend contains no translation logic; every decision it displays was made and recorded by the orchestrator, which keeps the displayed evidence trustworthy.

## 7.5 Observability

Goal G4 makes every run traceable through two complementary backends. LangSmith<sup>11</sup> captures the *semantic* trace – every node visit, model call with full prompt and completion, tool invocation, and token count per run – which is the raw material of the evaluation (chapter 8), reconstructing per-stage costs and outcome funnels. Pydantic Logfire<sup>12</sup> captures the *infrastructure* trace via OpenTelemetry instrumentation of the HTTP clients and model SDK, where provider-level failures – timeouts, truncated streams, malformed responses – become visible and attributable (chapter 9). A recording proxy in front of the providers can also capture and replay complete LLM interactions, giving deterministic fixtures for tests and letting the evaluation harness re-run pipelines against recorded behavior.

## 7.6 Deployment

The system is declared as a single Docker Compose<sup>13</sup> orchestration, override files layering the development and production variants, so one command brings up the full topology – databases, tool servers, sandbox daemon, orchestrator, and frontend. Production mirrors the same containers on e-INFRA CZ, exposing the API, the Daytona dashboard, the relational migrator, and the Neo4j browser under public subdomains, with the frontend on Vercel<sup>14</sup> and a public demonstration instance accompanying the thesis<sup>15</sup>. Secrets and endpoints come exclusively from environment configuration – the same values the settings panel of section 6.1 overrides per user at run time.

---

<sup>11</sup><https://docs.langchain.com/langsmith/observability>

<sup>12</sup><https://pydantic.dev/logfire>

<sup>13</sup><https://docs.docker.com/compose/>

<sup>14</sup><https://vercel.com/>

<sup>15</sup><https://uom-demo.vercel.app/> (mirror: <https://uom.dyn.cloud.e-infra.cz/>).

## 7.7 Extensibility: Adding a Framework Is Mechanical

Goal G3 requires that the approach not be tied to its five current frameworks, and the architecture discharges this constructively: every framework-specific fact lives in one of five easy-to-modify artifacts, and no workflow logic, validation machinery, or equivalence checking mentions a concrete framework (beyond using enumerations). Adding a framework – say a Java JPA (e.g. Spring Data JPA<sup>16</sup>), or a Python ORM/ODM framework (e.g. SQLAlchemy<sup>17</sup> or Django MongoDB<sup>18</sup>) target – means supplying those five artifacts.

1. *Registration*. An entry in the framework enumeration, its language classification, and its normalized name – one line each in a set of static mappings that drive all dispatching.
2. *A pinned build manifest*. The `.csproj`, `pom.xml`, or equivalent, with every dependency version fixed. This single file simultaneously grounds generation (it is injected into the prompt) and configures validation (the sandbox builds against it) – the dual use described in section 5.6.
3. *Canonical harness snippets*. A schema-validation and a query-execution entry-point file implementing the harness contract of section 5.7: the mirrored serialization rules above the seam marker and the framework-idiomatic `count / firstSample / lastSample` digest below it. The prelude becomes the injected invariant; the region below the seam becomes the structural few-shot example shown to the model. This is the artifact requiring the most engineering judgment, since it encodes how the framework executes queries and how its results serialize canonically, and it also serves as the authoritative ground truth for the skill document below.
4. *A framework skill document*. A skill folder [64] (section 8.5) capturing the framework’s permissible import surface, its type-mapping rules, and its per-idiom query recipes for the pinned version – the knowledge that keeps generation from hallucinating APIs. Because this knowledge must be *correct against the pinned dependency*, not recollected, the recommended way to author it is to have a capable coding agent draft it from the manual validation harness and the real toolchain, then compile-verify the result. Modern agent harnesses expose a skill-authoring command for exactly this – the `/skill-creator`<sup>19</sup> skill in Claude Code, and its analogues in other agents (Codex, the Antigravity CLI, and similar) – driven by a prompt of the shape:

---

<sup>16</sup><https://spring.io/projects/spring-data-jpa>

<sup>17</sup><https://www.sqlalchemy.org/>

<sup>18</sup><https://www.mongodb.com/docs/languages/python/django-mongodb>

<sup>19</sup><https://github.com/anthropics/skills/blob/main/skills/skill-creator/SKILL.md>

“Using the attached, compile-verified query and schema validation harnesses for `<framework>` as the ground truth for the correct imports and API surface, and consulting the pinned dependency manifest and the official `<version>` documentation, create a skill document that teaches a code-generating model to write `<framework>` persistence code that compiles against this exact version: the canonical import list, the source-to-target type mapping, the idiomatic query-construction recipes, and the traps that fail the build. Do not include any API you have not verified against the harness or the documentation.”

Tagging the manual harness as the ground truth is essential: it, not the model’s parametric memory, is what pins the skill to the real, compilable [API](#).

5. *Prompt fragments.* A worked mapping example and any residual idiom notes for the dynamic prompt assembly of section 5.4, complementing the on-demand skill content.

A new framework in an already-supported language runs in the existing sandboxes untouched; a new *language* also needs a sandbox image – a base-image declaration with its toolchain, from which Daytona builds the snapshot automatically – and a validator variant that knows the toolchain’s build-and-run commands. Nothing else changes: the graph, routing, equivalence checker, judge, and interfaces are framework-agnostic by construction. Extension is therefore mechanical by structure, its cost bounded by one manifest, two harness files, a verified skill document, and a prompt example – and the harness files, being the ground truth the skill is generated from, are where the real work concentrates.

# Chapter 8

## Experimental Evaluation

This chapter evaluates the assistant against the requirements of section 2.5, addressing goal G5. Four questions structure the experiments: does the generated code compile against the real toolchains (R1); does it return the same data as the source when executed (R2); does the bounded self-repair loop recover from generation failures (R3); and how often is a human needed (R4). A fifth question goes beyond the requirements: how does the purpose-built workflow compare, on the identical task and under the identical checker, against state-of-the-art general-purpose coding agents driven by frontier models. Section 8.1 fixes the experimental design, section 8.2 the input/target stores, and section 8.3 the metrics; section 8.4 measures the initial workflow design as a baseline and section 8.5 summarizes the redesign that separates the two measurement campaigns; section 8.6 presents the headline results, section 8.7 the resource profile, section 8.8 the judge-alignment analysis, and section 8.9 the head-to-head comparison. All raw experimental data accompany the thesis as an attachment (section A.2).

### 8.1 Experimental Design

**Task and workload.** Every experiment translates persistence-layer artifacts over the *WideWorldImporters* (WWI) database [40] introduced in section 2.3. For each of the six framework pairs – the three .NET sources (Entity Framework Core, NHibernate, Dapper) crossed with the two Spring Data targets (MongoDB, Neo4j) – the input is one request of scope *BOTH*: the source entity classes with their mapping configuration, plus a fixed suite of fifteen read-only queries expressed in the source idiom (LINQ, LINQ-over-ISession, or raw SQL). The suite spans the mapping concerns of the benchmark taxonomy of table 3.1: indexed and non-indexed filtering, range and set predicates, aggregation and grouping, one-to-many navigation, sorting with paging, set operations (UNION), and JSON-column access. The fifteen queries are submitted *bundled* in a single request, so one run exercises the full workflow of chapter 4 end-to-end.

**Repetitions.** Language-model output is stochastic even at fixed decoding settings [8], and on batched serving infrastructure it is not reproducible bit-for-bit [65] (section 9.3); a single run is therefore a sample, not a measurement. The headline experiment executes  $n = 10$  independent repetitions per pair – 60 runs, 900 query-translation slots – and reports means, per-repetition distributions, and  $\text{pass}@k$  (section 8.3) estimates. The baseline campaign of section 8.4 used  $n = 15$  repetitions per pair on a smaller variant.

**Configuration.** All runs use the production configuration of table 4.1: the open-weight kimi-k2.7-code as the generation model, deepseek-v4-pro as the post-workflow judge (the thinking-enabled variant as primary, the standard variant and gemma4 as ordered fallbacks; in the headline campaign the primary carried 195 of 240 verdicts, the first fallback 45, and the last was never invoked), the translation-repair budget capped at three loops, and validation in the Daytona sandboxes against the live databases. To keep repetitions statistically independent, the harness injects a per-run nonce that defeats server-side prompt caching (section 9.1). Every run is traced end to end; the recording proxy of section 7.5 preserves the complete model traffic of each run, so all analyses are replayable offline even after a model is deprecated.

## 8.2 Source Inputs and Target Stores

The input to every run is the same across the six pairs up to the source dialect: a fixed set of entity classes with their mapping configuration, and the fifteen read-only queries of table 8.1. The schema is a subset of *WideWorldImporters* large enough to exercise every mapping concern of table 3.1 yet small enough to inspect: eight persisted entities – Order and OrderLine (a one-to-many sales relationship), Person (carrying two JSON columns), Supplier, CustomerTransaction, PurchaseOrder, StockItem, and StockItemStockGroup (a many-to-many junction). Figure 8.1 gives the relational source schema; figures 8.2 and 8.3 give the two target stores, restricted to the entities the input queries reference. A complete input is reproduced in section B.5.

**Target-store provisioning.** The document and graph stores are *not* produced by the assistant; populating them is the one-time ETL step set by non-goal N1 (section 2.3). The relational instance is migrated to MongoDB with the *MongoDB Relational Migrator* [37] and to Neo4j with the *Neo4j ETL Tool CLI* [38], in both cases using the tools’ automatic mapping heuristics (primary/foreign-key relationships, cardinality inference, embedding-versus-referencing decisions) with the default settings left intact.<sup>1</sup> The schemata are reproduced in figures 8.2 and 8.3. The assistant treats the resulting stores

---

<sup>1</sup>Relational Migrator: <https://www.mongodb.com/docs/relational-migrator/mapping-rules/new-rules-suggested-mappings/>; Neo4j ETL: [https://neo4j.com/labs/etl-tool/1.5.0/#\\_inferring\\_schema\\_with\\_mapping\\_rules\\_generate\\_metadata\\_mapping](https://neo4j.com/labs/etl-tool/1.5.0/#_inferring_schema_with_mapping_rules_generate_metadata_mapping)

as an immutable *ground truth of the target schema*: it reads their structure during schema inspection (section 4.3.2) and executes against them during validation, but never modifies them. Consequently, the mapping the user supplies bounds what the assistant can achieve – an unoptimized or unidiomatic mapping (e.g., a one-to-one table-to-document mapping that keeps every relationship inter-document where a de-normalized store would embed, or a graph in which a source foreign key is materialized only as a relationship and not as a scalar property) shifts the achievable accuracy and can make a translation look harder and run longer. This input dependence is quantified as a validity threat in section 9.3. The concrete artifact visible in these experiments is graph-side: the [ETL](#) tooling renders each source foreign key as a typed relationship and does *not* carry the corresponding scalar column onto the node (e.g. an `OrderLine` node exposes no `orderId` property), so a query that filters on that key in the source must instead traverse the relationship (difficulties analyzed in section 8.6).

The suite is deliberately *coverage-driven* rather than random: each query targets one mapping concern of the reused benchmark taxonomy (table 3.1), from trivial selection (Q1–Q5) through aggregation (Q7–Q9) to the hardest transformations – the one-to-many navigation of Q10, the [JSON](#)-column access of Q13–Q14, and the set operation of Q15. Two queries are chosen specifically as traps that separate a compiling translation from a correct one: Q10, because the source `Include` navigation (listing 8.1) has no single-idiom image in either target – it becomes a `$lookup` join or a second query in MongoDB, and a relationship traversal in Neo4j – and Q13–Q14, because the [JSON](#) columns are stored as raw strings in both targets rather than as structured sub-documents.

```
// Query 10 (group D): one order with its line items eagerly loaded.
// The Include navigation is the trap: Order.OrderLines is a
↳ one-to-many relationship that has no one-to-one image in a document
↳ or graph store.
public static Order? Query10(SandboxDbContext ctx)
{
    return ctx.Orders
        .Include(o => o.OrderLines)
        .SingleOrDefault(o => o.OrderID == 530);
}
```

**Listing 8.1** A representative [EF](#) source query from the workload (Q10).



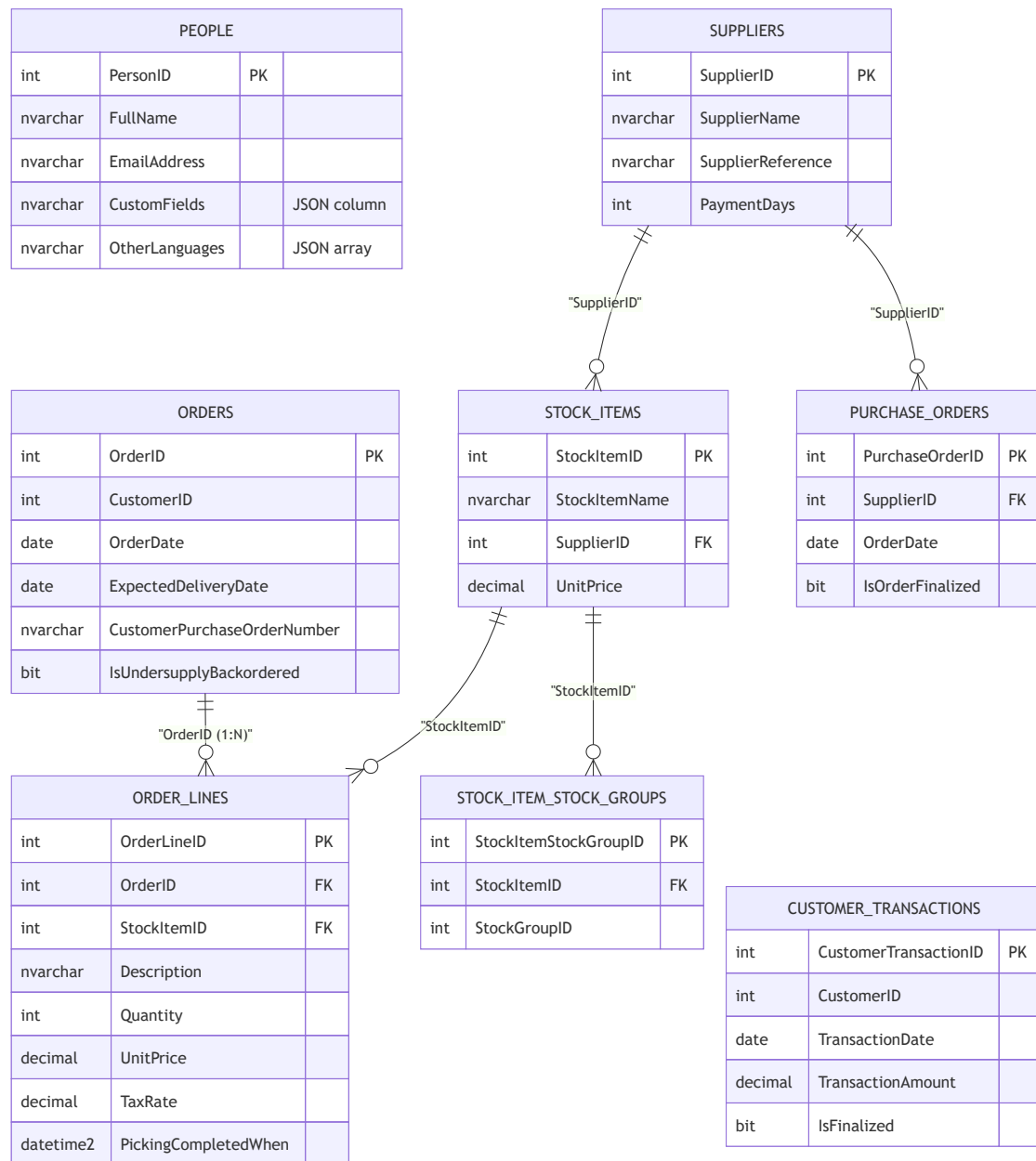
| #  | Grp | Query and mapping concern exercised                                        | Result               |
|----|-----|----------------------------------------------------------------------------|----------------------|
| 1  | B   | OrderLines filtered by an OrderID equality (foreign-key selection)         | one order's lines    |
| 2  | B   | OrderLines filtered by a decimal UnitPrice equality (non-indexed value)    | price-matched lines  |
| 3  | B   | OrderLines in a PickingCompletedWhen date range (nullable range predicate) | date-window lines    |
| 4  | B   | OrderLines with OrderID in a five-element set (IN-list)                    | five orders' lines   |
| 5  | B   | OrderLines whose Description contains a substring (text search)            | substring matches    |
| 6  | B/E | OrderLines ordered by key, Skip 1000 Take 50 (paging)                      | 50                   |
| 7  | C   | OrderLines grouped by TaxRate with counts, sorted (grouped aggregation)    | one row per tax rate |
| 8  | C   | Max(UnitPrice) over OrderLines (scalar aggregation)                        | 1 (scalar)           |
| 9  | C   | Sum(Quantity * UnitPrice) over OrderLines (aggregation over an expression) | 1 (scalar)           |
| 10 | D   | one Order with its OrderLines eagerly loaded (Include, one-to-many)        | 1 order + its lines  |
| 11 | E   | Orders sorted by ExpectedDeliveryDate, Take 1000 (sort with limit)         | 1000                 |
| 12 | E   | distinct CustomerPurchaseOrderNumber over Orders (distinct projection)     | distinct values      |
| 13 | F   | People filtered on a JSON object member CustomFields.Title                 | titled people        |
| 14 | F   | People whose JSON array OtherLanguages contains a value                    | language matches     |
| 15 | G   | union of two Supplier-id ranges, ordered (set operation)                   | 10                   |

**Table 8.1** The fifteen-query workload, shown in the Entity Framework Core (LINQ) source dialect; the NHibernate and Dapper inputs express the same fifteen logical queries in HQL and raw SQL. *Grp* is the benchmark category of table 3.1. *Result* is the source-side result cardinality over the WideWorldImporters instance; exact where the query fixes it, otherwise the shape of the returned set.

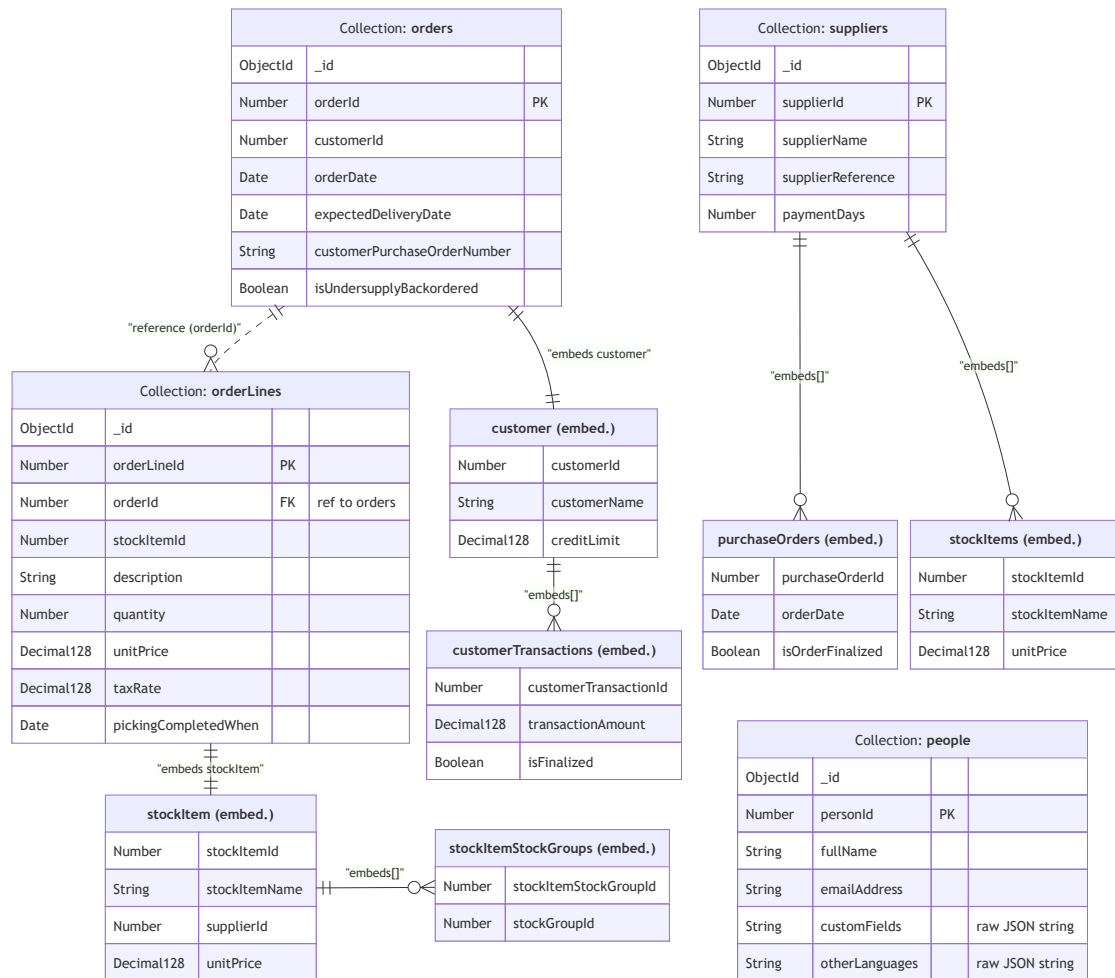
## 8.3 Metrics

The headline metric everywhere is *strict execution equivalence*, the machine-checked realization of requirement R2: the fraction of query slots whose final translated harness compiled, executed, and produced a result digest that the deterministic checker of section 5.7 declared equivalent to the source's (dist = 0 in both the direct and swapped

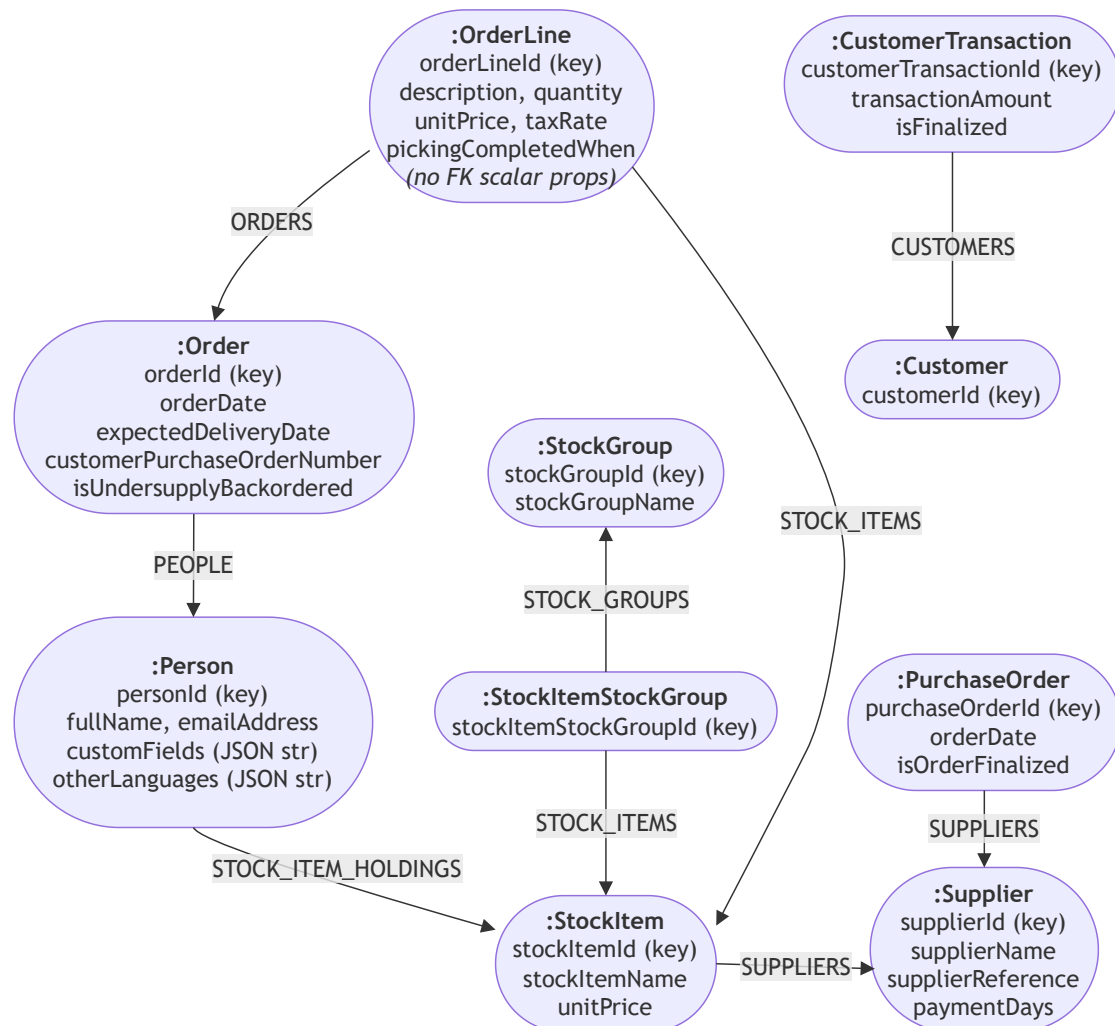




**Figure 8.1** The relational source schema (Microsoft SQL Server, WideWorldImporters), restricted to the eight entities the input references. Primary keys and the foreign keys the queries traverse are shown; edges are foreign-key relationships.



**Figure 8.2** The MongoDB store as produced by the ETL step: four collections (orders, orderLines, people, suppliers). Sub-documents (stockItem, customer) and arrays (customerTransactions, purchaseOrders, stockItems) are embedded; orders and orderLines stay in separate collections linked by orderId (inter-document reference). The JSON columns of people are stored as raw strings.



**Figure 8.3** The Neo4j store as produced by the ETL step: source tables become node labels and foreign keys become typed relationships (ORDERS, STOCK\_ITEMS, SUPPLIERS, CUSTOMERS, STOCK\_GROUPS). A foreign key materialized as a relationship leaves no scalar property on the node, so key-equality queries (Q1, Q4, Q10) must traverse rather than filter.

comparison of algorithm 2). For a run with  $q_{\text{eq}}$  equivalent of  $q_{\text{exp}}$  expected queries, the run’s equivalence is  $q_{\text{eq}}/q_{\text{exp}}$ ; a run is a *strict pass* when  $q_{\text{eq}} = q_{\text{exp}}$ . In the vocabulary of definition 1, a repetition is one terminating run  $s_0, \dots, s_T$ , and a strict pass requires  $\delta(s_T) = \text{FINALIZED}$  with every query strictly equivalent in the final state. Strict equivalence is digest agreement – exact counts plus boundary samples (section 5.7) – so even the headline metric is execution-grounded *evidence* of equivalence, not an exhaustive proof over the full result sets (section 9.2). Three cases deserve special mentions:

**Strict vs. accepted.** The pipeline’s own acceptance folds in judge verdicts: a query may be *accepted* although its digests differ, when the judge deems the difference a store-representation artifact (section 4.3.6). Accepted counts are reported only in the outcome decomposition (figure 8.7), clearly labelled, never as the headline – and the cross-arm comparison of section 8.9 uses strict equivalence exclusively, because the external arms have no judge and quoting judge-inclusive numbers would favor the pipeline.

**pass@ $k$ .** The probability that at least one of  $k$  sampled runs is a strict pass, computed with the unbiased estimator of Chen et al. [14]: for  $n$  runs of which  $c$  pass,  $\text{pass}@k = 1 - \binom{n-c}{k} / \binom{n}{k}$ . This measures the practical mode of use – a developer who can afford  $k$  attempts and keeps the verified one.

**Intervention rate.** The fraction of runs whose terminal state is the human hand-off ( $\delta(s_T) = \text{HANDOFF}$ ) – i.e. at least one query remained unaccepted when the repair budget was exhausted – the metric of requirement R4.

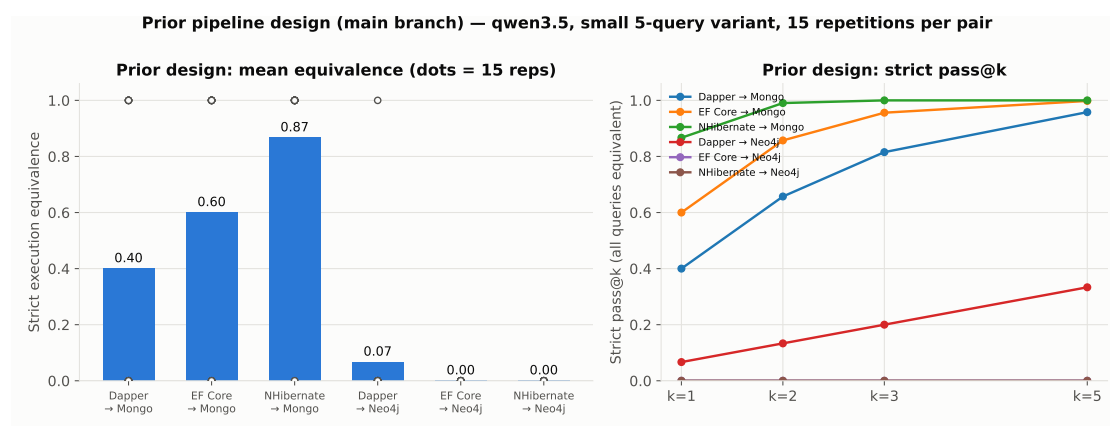
Alongside these, each run records the compile outcome of both harnesses (R1), the number of repair loops consumed (R3), and its resource profile: total LLM tokens, provider-billed cost in USD, and wall-clock time. Two supplementary metric families complete the picture. *CodeBLEU* [32, 66] – a code-similarity measure combining n-gram overlap with abstract-syntax-tree and data-flow matches<sup>2</sup> – quantifies structural similarity of the produced artifacts; since no human-authored ground-truth translation exists, the reference for each pair is *frozen from the experiment itself*: the first repetition whose fifteen queries were all strictly execution-equivalent, with that repetition excluded from the respective experiment’s aggregated metrics. CodeBLEU is thus a proxy for output stability against an execution-verified code, not for correctness (section 9.3). Finally, the graded *post-workflow LLM-judge scores* (*faithfulness*, *code correctness*, *translation equivalence*; each a fraction in  $[0, 1]$ ) are recorded per run and analyzed against the deterministic ground truth in section 8.8.

---

<sup>2</sup>CodeBLEU is a popular metric for evaluating code generation quality. We used the implementation available at <https://github.com/k4black/codebleu>. We do not explain the metric in detail for brevity.

## 8.4 Baseline: the Initial Workflow Design

The first measurement campaign evaluated the workflow as initially designed – provider-native structured output for the whole translation, framework knowledge injected directly as prompt blocks, no in-loop execution probe – with the fast qwen3.5 generator on a reduced variant of the workload (the first five queries per pair), at 15 repetitions per pair. Figure 8.4 shows the outcome. The document-target pairs already worked passably (mean strict equivalence 0.40–0.87; NHibernate→MongoDB reached pass@1 = 0.87), but the graph-target pairs failed almost categorically: Dapper→Neo4j passed one repetition of fifteen, the other two Neo4j pairs none. Compilation was *not* the bottleneck – 93% of runs compiled – the failures were semantic: hallucinated Cypher-DSL API members caught late, naive foreign-key mappings that the graph store does not materialize, and single-shot structured output that truncated code-sized fields (section 5.5). The baseline established both that the verification machinery discriminates (compiling code that returns wrong data is rejected) and that the generation stage, as then designed, could not reliably cross the relational–graph boundary.



**Figure 8.4** The initial workflow design (qwen3.5 generator, five-query variant, 15 repetitions per pair): mean strict execution equivalence with per-repetition dots (left) and strict pass@k (right). Document targets work; graph targets fail almost categorically.

## 8.5 The Redesigned Generation Stage

Between the two campaigns, the generation stage – and only the generation stage; the graph topology, the deterministic validators, and the equivalence checker are unchanged – was redesigned in six respects. The changes are summarized here because they *are* the treatment whose effect the headline experiment measures; their design rationale is developed in chapter 5.

1. *Framework knowledge as skills.* The framework-specific prompt blocks were restructured into five per-framework *skill* documents, one per supported framework, following the progressive-disclosure pattern of agent skills [64]: a compact orientation is injected always-on for both sides of the pair, and detailed reference files – the permissible import surface, type-mapping tables, and per-idiom query recipes – are retrieved on demand through a dedicated tool. Critically, the skill content is not model recollection: every import list and API idiom was compile-verified against the exact pinned dependencies, and the store-specific guidance (e.g. which source columns the ETL step does not materialize in the graph, and how to recover them by relationship traversal) was verified against the live databases.
2. *Fragment-wise persistence.* The single `save_translation` action of section 5.5 was further decomposed: the agent persists the schema fragment and each QueryN fragment through separate `save_schema_translation` / `save_query_translation` tool calls, merged deterministically into the state. A *save-first* policy (persist everything before any validation) makes the draft immune to context-management events mid-generation, and a convergence middleware forces one final model call – with saving as the only permitted action – if the agent stops with an incomplete draft.
3. *An aligned in-loop probe.* The agent gained a budgeted `validate_draft` tool that compiles and executes *both* sides of the saved draft in the same sandboxes and reports per-query equivalence from the same checker as the downstream acceptance gate. The in-loop signal is therefore aligned with the official metric – the property whose absence dominates failed general-agent repairs [30] – and it is deliberately scarce (three calls per loop, used batched and late, matching the cost-effectiveness recommendations of that same study).
4. *Deterministic harness assembly.* The model authors only the variable fragments; a deterministic assembler composes them with the canonical prelude (serializers, runner, seam marker) of section 7.7 into the runnable harness. The clean, user-facing translation is *derived* from the validated harness after acceptance, never generated afresh – keeping the finalized artifact a faithful projection of verified code.
5. *Per-stage resilience middleware.* Each generative stage carries an explicit middleware stack: model retry, ordered model fallback, tool retry with backoff, context *editing* (clearing stale tool results) in place of summarization – which had been observed to discard the task statement itself under length pressure – structured-output repair with the validator’s errors fed back, and the reasoning-content sanitizer of section 9.1.

6. *Per-query verdict bookkeeping.* The evaluation stage partitions queries each loop into deterministically verified, judge-accepted, regressed, and undecided. Deterministic acceptances are re-verified every loop – a query that a later revision breaks re-enters repair flagged as regressed – while judge acceptances remain sticky; this closed an acceptance-masking defect in which an early acceptance could conceal a later regression.

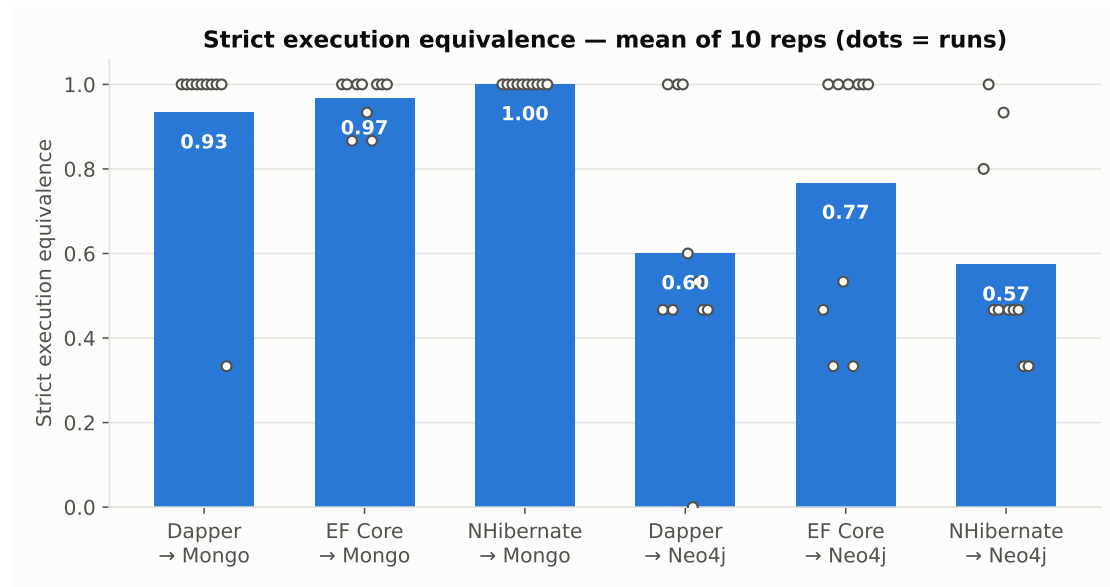
## 8.6 Headline Results

Table 8.2 and figures 8.5 and 8.6 present the headline campaign: six pairs  $\times$  fifteen bundled queries  $\times$  ten repetitions. Overall, **726 of 900 query slots (80.7%) are strictly execution-equivalent** – 96.7% on the document targets (435/450) and 64.7% on the graph targets (291/450). Under the accepted metric (judge verdicts included) the overall rate is 94.3%, and 51 of 60 runs finalized without human intervention. Every pair, including all three Neo4j pairs, produced at least one perfect 15/15 repetition, and  $\text{pass}@5$  is  $\geq 0.92$  for five of the six pairs: the capability exists on every pair, and the residual gap on the graph targets is one of *consistency*, analyzed below.

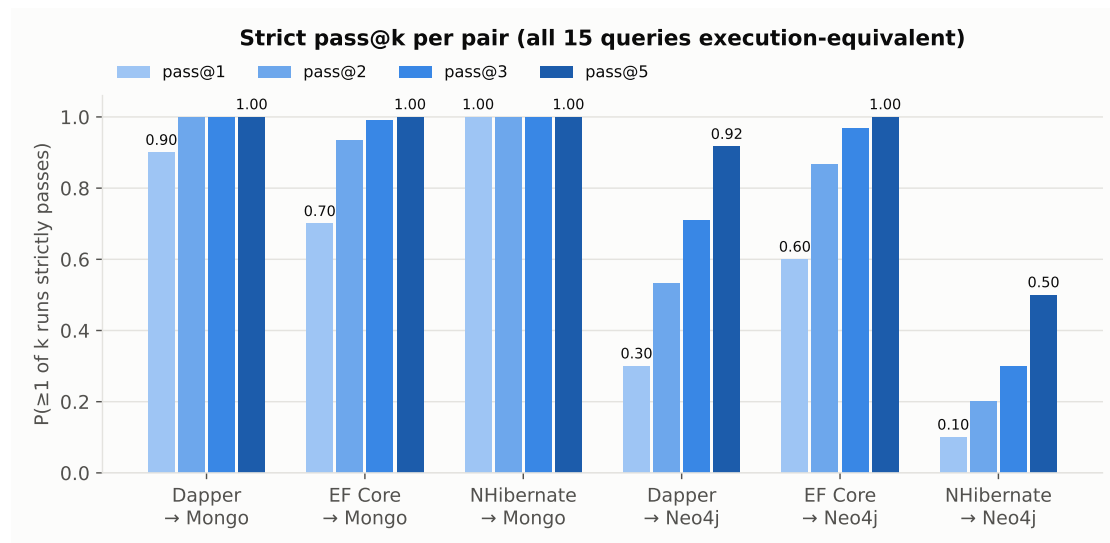
| Pair             | Equiv        | p@1  | p@5  | 15/15 | Loops | Mtok (loop) | USD  | Wall   |
|------------------|--------------|------|------|-------|-------|-------------|------|--------|
| Dapper→Mongo     | 0.93         | 0.90 | 1.00 | 9/10  | 1.0   | 1.15 (0.78) | 1.04 | 26 min |
| EF Core→Mongo    | 0.97         | 0.70 | 1.00 | 7/10  | 1.1   | 1.58 (1.14) | 1.48 | 33 min |
| NHibernate→Mongo | 1.00         | 1.00 | 1.00 | 10/10 | 1.1   | 1.28 (0.85) | 1.16 | 31 min |
| Dapper→Neo4j     | 0.60         | 0.30 | 0.92 | 3/10  | 1.9   | 2.23 (1.12) | 2.35 | 55 min |
| EF Core→Neo4j    | 0.77         | 0.60 | 1.00 | 6/10  | 2.0   | 2.58 (1.94) | 2.70 | 56 min |
| NHibernate→Neo4j | 0.57         | 0.10 | 0.50 | 1/10  | 2.3   | 3.36 (2.43) | 3.60 | 74 min |
| <b>Overall</b>   | <b>0.807</b> | 0.60 | 0.90 | 36/60 | 1.6   | 2.03 (1.38) | 2.06 | 46 min |

**Table 8.2** The headline campaign (15 bundled queries, 10 repetitions per pair, kimi-k2.7-code generator). *Equiv* is mean strict execution equivalence;  $\text{p}@k$  is strict  $\text{pass}@k$ ; *15/15* counts perfect repetitions; *Loops* is the mean number of generation attempts (cap 3); *Mtok (loop)* is whole-run millions of tokens with the translation-loop-only figure in parentheses (all stages from generation onward)<sup>3</sup>; *USD* is the provider-billed cost per run.

<sup>3</sup>The translation-loop-only figures are means over the per-node token attributions harvested from the tracing backend; for the document pairs all ten repetitions were harvested, whereas the graph pairs rest on 6–8 repetitions each, the remainder lost when the tracing platform’s monthly quota was reached (section 9.1). The whole-run token totals, which come from the run records directly, are complete for all sixty runs.



**Figure 8.5** Mean strict execution equivalence per pair with one dot per repetition. The document targets are near-saturated; the graph targets show a bimodal pattern – most repetitions are perfect or near-perfect, a minority commit to a losing modeling strategy early.



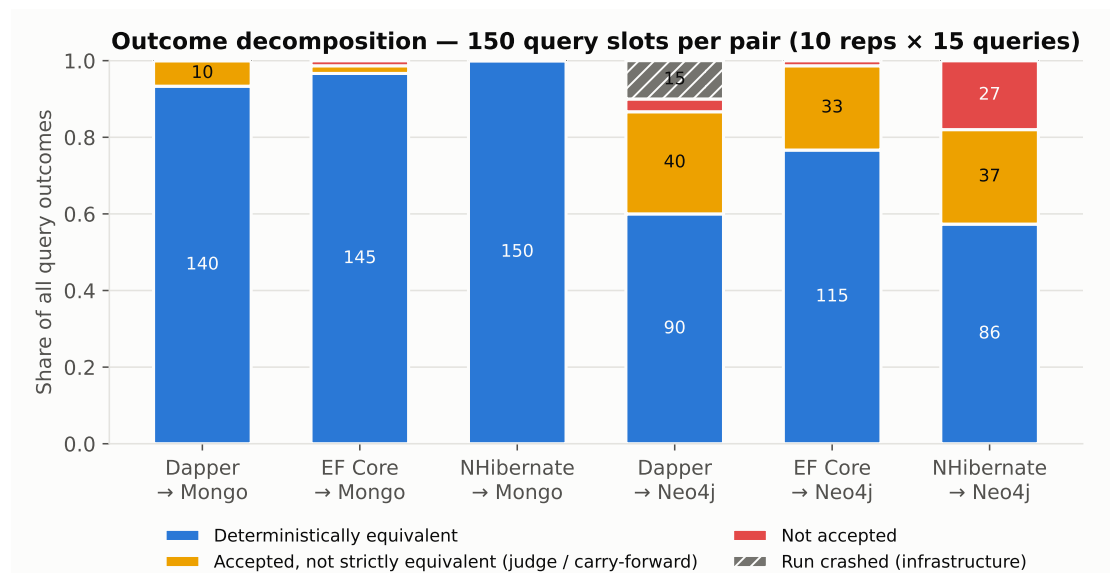
**Figure 8.6** Strict pass@ $k$  (all fifteen queries equivalent) per pair,  $k \in \{1, 2, 3, 5\}$ , with the unbiased estimator over 10 repetitions [14].

Compared with the baseline of section 8.4, the redesigned generation stage moved the graph targets from near-categorical failure (strict pass@1 of 0.00–0.07) to means of 0.57–0.77 with perfect repetitions on every pair – while tripling the per-request

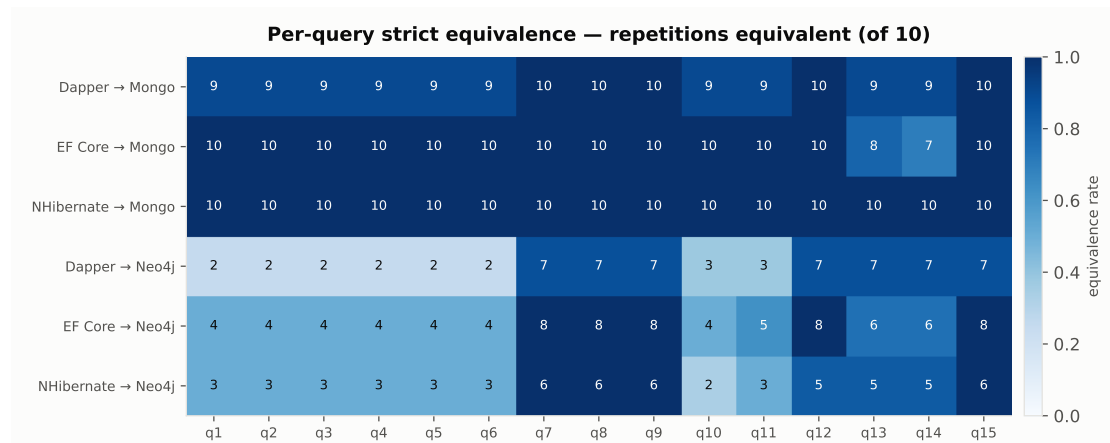


query count. The two campaigns differ in generator model (from low-class qwen3.5 to mid-class kimi-k2.7-code) and workload size as well as in design, so the comparison is a before/after of the assistant as a whole. For transparency, the per-failure-mode decomposition below attributes the remaining errors to specific, inspectable causes rather than to model choice.

**Outcome decomposition and per-query difficulty.** Figure 8.7 decomposes all 900 slots into strictly equivalent, accepted-but-not-equivalent (judge or carry-over), not accepted, and slots lost to one crashed run; figure 8.8 resolves equivalence per query. The heatmap localizes the difficulty: on the document side nearly every query succeeds in every repetition, while on the graph side the losses concentrate in the queries touching entities whose foreign-key columns the [ETL](#) step does not materialize as node properties, plus the [JSON](#)-column queries (13–14) on NHibernate→Neo4j.



**Figure 8.7** All 900 query slots per pair, decomposed into strictly equivalent, accepted-but-not-strictly-equivalent (judge acceptance of store-representation artifacts, or acceptance carried over from an earlier loop), not accepted, and slots belonging to one crashed repetition.

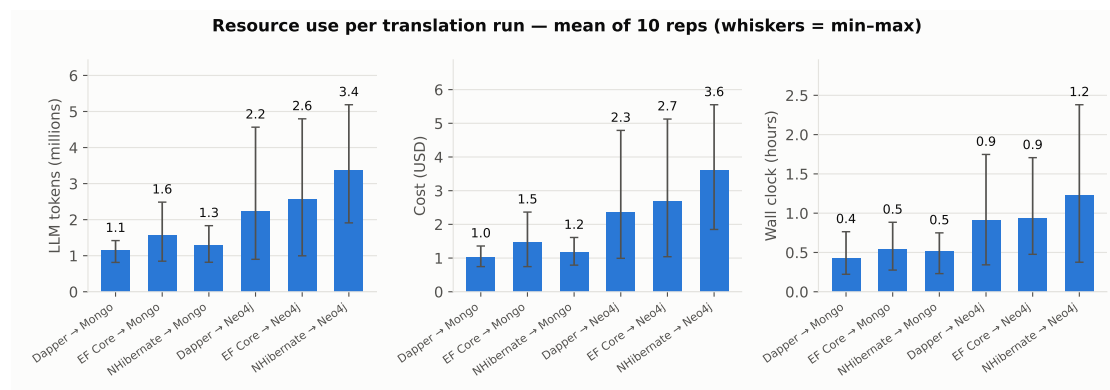


**Figure 8.8** Per-query strict-equivalence counts (of 10 repetitions) across the six pairs. Graph-target losses concentrate on the queries whose source columns the **ETL** step did not materialize in the store, and on the **JSON**-column queries.

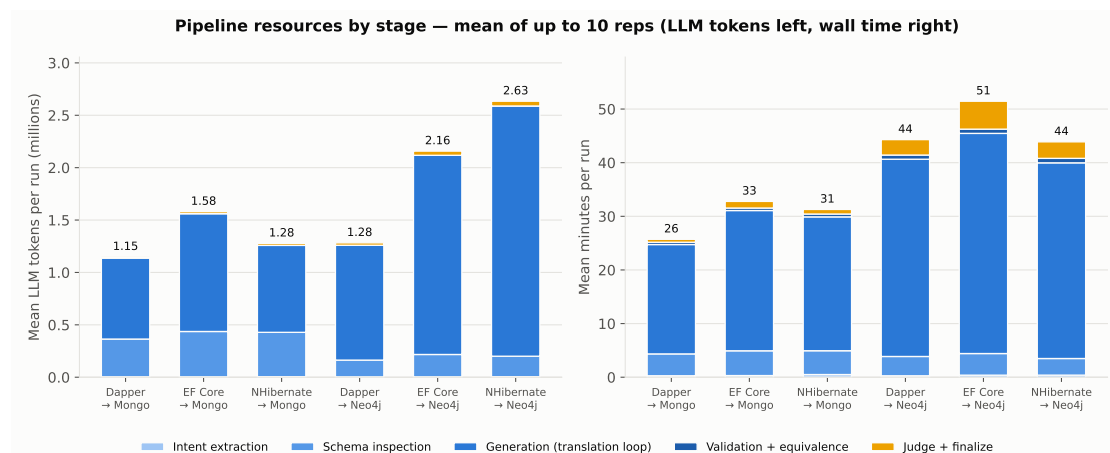
**Failure modes.** Root-cause analysis of the 174 non-equivalent slots – from the recorded model traffic and live database probes – attributes them to four causes. (i) *A serialization leak* ( $\approx 35$  slots): whenever the model omitted one Jackson ignore-annotation, the store-internal document identifier leaked into the target digest and failed the strict check with a deep distance of  $\approx 0.02$ – $0.04$  – a purely presentational defect the judge routinely accepted; it has since been eliminated deterministically in the harness prelude. (ii) *The graph modeling* ( $\approx 120$  slots): the **WWI** graph store, as produced by the **ETL** tooling, does not materialize several source foreign-key columns as node properties at all (the links exist only as relationships, and one column has no representation whatsoever). Repetitions that recognized this early translated those queries by relationship traversal and, for genuinely absent fields, synchronized the projection on both sides – all such runs reached 15/15. Repetitions that mapped the entities naively produced null-valued fields and spent their repair budget converging or not; this strategy choice, made in the first generation attempt, is the dominant source of graph-side variance and of the loop-count multiplier in table 8.2. (iii) *Infrastructure* (15 slots): one repetition crashed when a provider outage escalated the fallback chain to a misconfigured last-resort model. (iv) *Residual defects* ( $\approx 4$  slots): genuine mistranslations, e.g. a mishandled date-boundary predicate. Causes (i) and (iii) have deterministic fixes; cause (ii) has since been addressed by moving the strategy decision into the up-front skill guidance, with re-measurement left to future work.

## 8.7 Resource Profile

Figure 8.9 reports tokens, cost, and wall-clock per pair; figure 8.10 attributes them to pipeline stages. A document-target run costs  $\approx 1.1\text{--}1.6$  M tokens,  $\$1.0\text{--}1.5$ , and  $\approx 0.5$  h; a graph-target run 2.2–3.4 M tokens,  $\$2.3\text{--}3.6$ , and 0.9–1.2 h – the multiplier is exactly the repair loops of failure mode (ii). The entire 60-run campaign cost  $\approx \$123$  in provider-billed usage. The stage attribution shows that the translation loop (generation, validation, equivalence, judgment) accounts for 62–95% of a run’s tokens, growing with the loop count, while the fixed pre-loop stages (input extraction, schema inspection) are a small constant; wall-clock is dominated by generation throughput and sandbox compilation, consistent with the latency discussion of section 9.2.



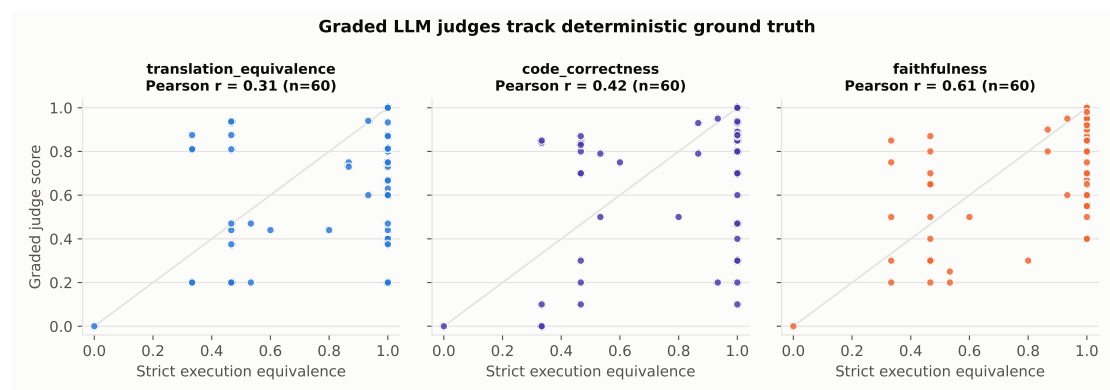
**Figure 8.9** Mean whole-run LLM tokens, provider-billed cost, and wall-clock time per pair (whiskers: min-max over 10 repetitions). The graph-target multiplier is driven by the repair loops of table 8.2.



**Figure 8.10** Mean LLM tokens (left) and wall-clock minutes (right) per run, attributed to pipeline stages. The translation loop dominates; the pre-loop stages are a small fixed cost.

## 8.8 Do the Judges Track the Ground Truth?

Because the workflow’s only non-deterministic verification layer is the LLM-as-a-judge Evaluator<sup>4</sup>, the evaluation checks it against the deterministic ground truth rather than trusting it (section 9.3). Figure 8.11 plots the three graded post-workflow judge scores against each run’s strict equivalence over all 60 runs. All three correlate positively – faithfulness  $r = 0.61$ , code correctness  $r = 0.42$ , translation equivalence  $r = 0.31$  (Pearson) – so the judges discriminate, but none is a substitute for execution: the spread at any given score is wide, and the weakest correlate is precisely the judge asked to assess equivalence, the property the deterministic checker measures directly. This is the empirical form of the thesis’s central design argument: judgment interprets execution evidence (section 5.6); it must not replace it.



**Figure 8.11** Graded LLM-judge scores against strict execution equivalence, one point per run ( $n = 60$ ), with Pearson correlations. The judges track the ground truth but with substantial spread.

## 8.9 Head-to-Head: the Workflow versus General-Purpose Agents

The final experiment asks whether the purpose-built workflow earns its complexity: could a state-of-the-art general-purpose coding agent, driven by a frontier model, do the same job from a single prompt? Two external *arms* were compared – Claude Code [5] with `claude-opus-4-8` (high reasoning) and `claude-sonnet-5` (high), and Google Antigravity CLI [6] with `gemini-3.5-flash` (high) and `gemini-3.1-pro` (high).

<sup>4</sup>We distinguish the Workflow’s LLM-as-a-judge Evaluator stage (section 4.3.6) from additional post-workflow LLM judges, used exclusively for experimental evaluation; with the same model as the workflow’s Evaluator by default.

**Protocol.** Fairness rests on three pillars. *Identical knowledge*: the live pipeline is driven through input extraction and schema inspection, and the translation stage’s exact system and user prompts – including the full skill content and the schema summary – are exported byte-identically as the external arm’s prompt; a mechanical adaptation replaces the save-tool contract with labelled fenced code blocks, since the arms run in chat mode without tools. *Identical task*: the same fifteen-query bundled request, answered in one shot; the arm’s own internal agentic iteration (extended reasoning, self-revision) is part of the treatment being measured, exactly as the repair loop is part of ours. *Identical scoring*: each captured answer is assembled by the pipeline’s own harness assembler, compiled and executed in the same sandboxes, and scored by the same per-query strict-equivalence checker – no Evaluator judge on either side. Repeated captures give the external arm the same distributional treatment as our repetitions where affordable<sup>5</sup>. The comparison covers the two document pairs on which the external arms produced scoreable output (Dapper→Mongo, run against all four models, and EF Core→Mongo, against all but the most expensive); on the graph pairs the external arms did not produce scoreable output at all – claude-sonnet-5 exceeded the harness’s 64k output-token ceiling mid-answer<sup>6</sup> (and the subscription’s entire five-hour token budget in the process), and the Antigravity outputs failed to compile.

---

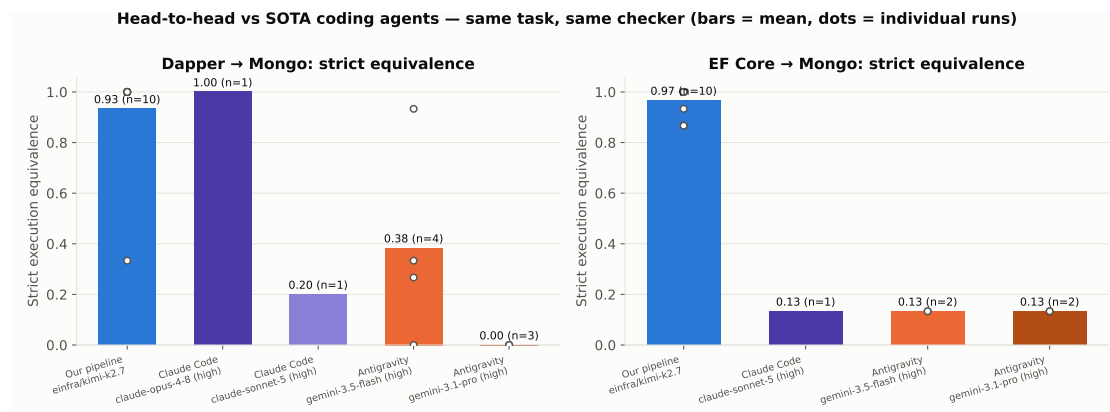
<sup>5</sup>The Claude arms are single captures ( $n = 1$ ) for a stated reason: they were executed under a personal Claude Pro subscription, where the one opus-4.8 attempt alone consumed  $\approx 253k$  tokens (\$2.91 at list price) and a large share of the subscription’s five-hour usage window, making repetitions impractical within the thesis budget. A single capture of a stochastic system is a sample, not a measurement (section 8.1), so every conclusion drawn from these arms is stated as suggestive rather than definitive; a repeated-capture replication is left to future work. The cheaper Antigravity arms have  $n = 2-4$ .

<sup>6</sup>Claude Code caps a single response at 64k output tokens (the `CLAUDE_CODE_MAX_OUTPUT_TOKENS` setting; <https://code.claude.com/docs/en/settings>). The cap is a property of the harness, not of the models: claude-opus-4-8 accepts up to 128k output tokens on the direct API (<https://platform.claude.com/docs/en/about-claude/models/overview>), but a chat-mode capture through the harness is truncated at the harness’s ceiling regardless of the model driving it.

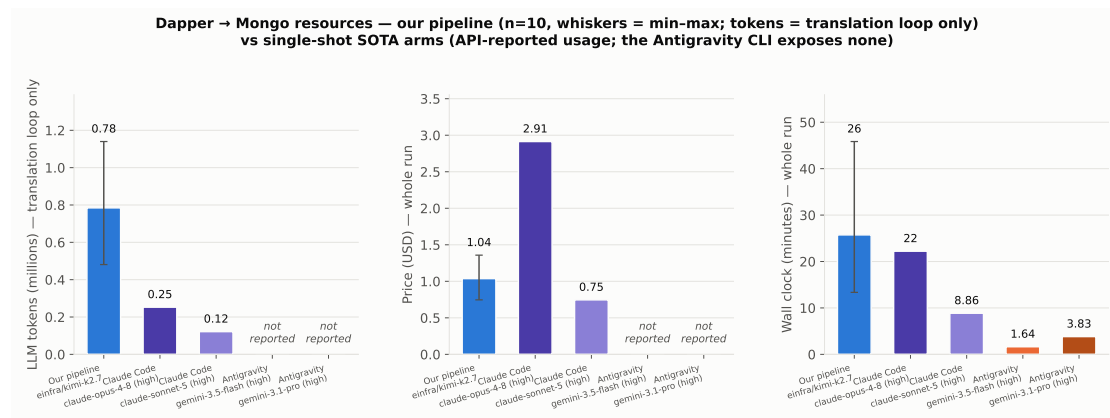
| Arm (model, effort)                  | Pair | Equiv       | <i>n</i> | Mtok | USD  | Wall     |
|--------------------------------------|------|-------------|----------|------|------|----------|
| <b>Our workflow</b> (kimi-k2.7-code) | D→M  | <b>0.93</b> | 10       | 0.78 | 1.04 | 26 min   |
| Claude Code (opus-4-8, high)         | D→M  | <b>1.00</b> | 1        | 0.25 | 2.91 | 22 min   |
| Claude Code (sonnet-5, high)         | D→M  | 0.20        | 1        | 0.12 | 0.75 | 8 min    |
| Antigravity (gemini-3.5-flash, high) | D→M  | 0.38        | 4        | —    | —    | 1.2 min* |
| Antigravity (gemini-3.1-pro, high)   | D→M  | 0.00        | 3        | —    | —    | 3.7 min* |
| <b>Our workflow</b> (kimi-k2.7-code) | E→M  | <b>0.97</b> | 10       | 1.14 | 1.48 | 33 min   |
| Claude Code (sonnet-5, high)         | E→M  | 0.13        | 1        | 0.12 | 0.72 | 7.5 min  |
| Antigravity (gemini-3.5-flash, high) | E→M  | 0.13        | 2        | —    | —    | —        |
| Antigravity (gemini-3.1-pro, high)   | E→M  | 0.13        | 2        | —    | —    | —        |

\*first capture only; — = not reported by the harness.

**Table 8.3** Strict execution equivalence, same task, same checker. Generation resources are API-reported where the harness exposes them (Claude Code session transcripts); the Antigravity CLI reports no usage, and some runs were not captured by accident, so only first-run wall clocks are known. Prices are the providers’ list prices in effect on the run dates. Our pipeline’s tokens count the translation loop only (the stages a single-shot arm also performs); its price and wall clock are whole-run.

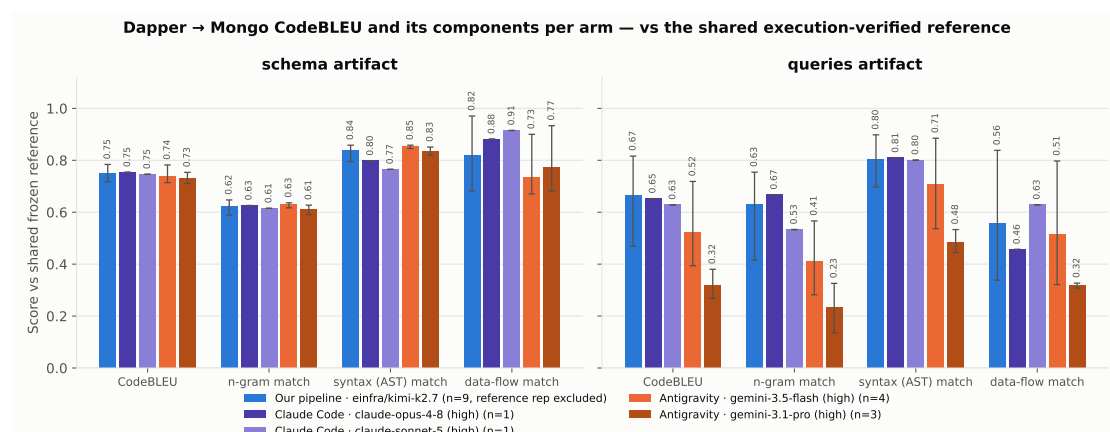


**Figure 8.12** Strict execution equivalence per arm on the two document pairs where the external arms produced scoreable output (bars: means; dots: individual runs). Only the frontier flagship matches the workflow, and only on one pair, in one shot.



**Figure 8.13** Generation resources on Dapper→Mongo: tokens (our pipeline: translation loop only), price, and wall clock. Claude Code usage is [API](#)-reported from its session transcripts; the Antigravity CLI exposes none.

**Results.** On Dapper→Mongo, the single claude-opus-4-8 capture achieved a perfect 15/15 – one session totalling  $\approx 88k$  output tokens at \$2.91 – matching the workflow’s 0.93 mean (nine of ten repetitions perfect) at 2.8 $\times$  the price per attempt (figure 8.13). Every other configuration fell far behind: claude-sonnet-5 scored 3/15 and 2/15 on the two pairs, and the Antigravity captures ranged from 14/15 to non-compiling 0/15 *across repetitions of the same model on the same prompt* – single-shot variance so large that any one-shot number is a lottery ticket rather than a measurement. On [EF](#) Core→Mongo no external arm exceeded 2/15 while the workflow held 0.97 (claude-opus-4-8 not tested; not shown in figures). Figure 8.14 adds the structural picture: CodeBLEU [32] against the shared execution-verified reference is nearly flat across arms on the schema artifact (0.73–0.75) and clusters tightly for all Claude-family outputs on queries (0.63–0.67) even where their equivalence differs by a factor of five – surface similarity does not separate correct from incorrect translations, which is precisely why the evaluation is execution-grounded.



**Figure 8.14** CodeBLEU [32] and its components per arm on Dapper→Mongo, against the shared execution-verified reference (the frozen repetition is excluded from its own arm). Structural similarity is nearly indistinguishable across arms whose execution equivalence differs by a factor of five.

**Interpretation.** Read with the single-capture caveat above, this study *suggests* – it cannot, at  $n = 1$  for the strongest arms, prove – that an open mid-size model inside the workflow matches or beats frontier models inside general-purpose harnesses. The pattern is consistent with the controlled findings of Lin et al. [30] on execution feedback in agentic repair: generic harnesses gain little from execution because their self-chosen validation is misaligned with the official oracle, and the bulk of their failed attempts pass their own tests. The workflow supplies exactly the two ingredients that study identifies as decisive and that a general harness structurally lacks here: an in-loop signal aligned with the true acceptance criterion (the same sandboxes, databases, and checker at both points – section 8.5), and curated, compile-verified API knowledge in place of budget-consuming exploration. The comparison also bounds the claim honestly: given the strongest available flagship agentic-coding-tuned model (claude-opus-4-8 at the time of writing), effectively unbounded output, and our exported context, a single-shot answer *can* reach 15/15 on the easiest pair, but more repetitions, pairs, and complex entities/queries would be needed to prove this claim – the workflow’s contribution is making that outcome cheap, repeatable, model-agnostic, and *verified* rather than asserted.



# Chapter 9

## Discussion

This chapter records what building and evaluating the assistant cost: the issues and the mitigations they forced (section 9.1), the limitations of the system (section 9.2), the threats to the validity of the experimental claims (section 9.3), and the future work (section 9.4). Where possible it separates problems inherent to the task from artifacts of the 2025–2026 open-model serving landscape.

### 9.1 Issues Encountered and Lessons Learned

**The serving platform, not the model, is the dominant failure source.** The reliability bottleneck of an LLM pipeline built on shared, open-weight inference infrastructure is the *serving layer*. On the shared e-INFRA platform [48] the assistant had to absorb transient outages, latency varying by an order of magnitude with load, models replaced under a stable alias mid-project, and behaviors that deviate from the OpenAI conventions the client assumes (reasoning models, for instance, return thinking traces in provider-specific structures that, replayed naively, are rejected with opaque errors). The fallback chains, per-node retry policies, tool backoff, and reasoning-content sanitizer of section 5.8 are direct responses: **every provider-facing assumption – response format, output length, protocol conformance – must be treated as unstable and guarded.** Two problems forced design reversals. First, the collapse of *provider-native structured output*: newer agentic-tuned models stopped honoring large single-response JSON schemas, truncating multi-line code past an *undisclosed* output budget, so the mitigation (section 5.5) decomposes the output and finalization abandons structured transport for fenced code blocks. Second, *prompt caching* corrupts measurement, so the harness injects a per-run nonce that defeats caching in experiments while leaving production prompts byte-identical. A related instability struck late: the tracing backend’s monthly quota was exhausted during the headline campaign, so the token attribution of section 8.7 rests on the subset of runs traced before the cut-off.

**Model selection is empirical, not nominal.** The provider’s portfolio of section 4.1.2 was arrived at by elimination, against most nominal indicators. Parameter count predicted little: a 122-billion-parameter Qwen3.5 variant consistently produced better code than a trillion-parameter MoE model Kimi-K2.6 [67], at a fraction of the latency. Successor versions were not uniform upgrades; the strongest model tried, GLM-5.2, was unusable – single requests took tens of minutes and were intermittently cut off by the provider. Kimi-K2.7-Code was chosen for generation, with DeepSeek-V4-Pro taking schema inspection and the independent judge. A subtler trade-off separated the finalists – *generalization versus knowledge fidelity*: the fast Qwen3.5 models follow the prompt’s worked examples and instructions closely but falter on unseen query shapes, whereas the agentic-tuned Kimi generalizes better but hallucinates imports more readily, plausibly because its undisclosed training cutoff<sup>1</sup> predates the pinned framework and C#/Java versions. The mitigation extends the grounding principle of section 5.6: per-framework *skill documents* [64] enumerate the permissible import surface of the pinned versions, so import correctness depends on *retrieval rather than recollection* – and the head-to-head experiment (section 8.9) is the strongest evidence for this stance.

**The value of autonomy is stage-dependent and non-monotonic.** The generation stage moved *twice* – from a free agent with all tools, to a plain structured-output call when the free agent proved undisciplined, then partially back to a ReAct agent with only documentation search and the state-writing save tool. The stable equilibrium – *autonomy inside a stage with a deterministic contract at its boundary* – was found empirically, not derived. A measurement lesson accompanied it: a context-summarization middleware that seemed harmless discarded the task statement itself under length pressure and was replaced by context *editing* that clears only stale tool results. Finally, infrastructure dependencies carry *licensing risk* on research timescales: the Daytona sandbox framework, adopted as open source, went closed-source two weeks before the thesis submission<sup>2</sup>, a reminder that pinning and self-hosting are as much a legal safeguard as an engineering one.

## 9.2 Limitations

*Scope of translation.* The implementation covers three .NET source frameworks and two Java Spring Data targets, one direction, pinned versions, schema and *read* queries. Write

<sup>1</sup>Undisclosed or ambiguous knowledge-cutoff dates are a documented, ecosystem-wide problem [68]; see e.g. <https://metehan.ai/articles/llm-knowledge-cutoff-dates/>.

<sup>2</sup>The complete Daytona repository is copied and published with this thesis. The Docker images use public images, but may be inaccessible in the future, so the repository is needed for reproducibility. A better solution in the future may be: Docker Sandboxes (<https://docs.docker.com/ai/sandboxes/>) or OpenSandbox (<https://github.com/opensandbox-group/OpenSandbox>).

operations, transactions, and surrounding application wiring are out of scope (non-goal N3); because the equivalence machinery is defined over query *results*, state-mutating operations would need a different observation contract. The extensibility argument of section 7.7 is structural and has not been exercised by an external contributor.

*Equivalence is evidence, not proof.* The digest – exact counts plus boundary samples under a deterministic ordering – is a deliberately economical witness: it cannot detect a translation that agrees on cardinality and on both boundary records yet differs in the *interior* of the result set, and its determinism depends on a unique sort key (section 5.7.3). The economy is intentional: a check this cheap can run inside every repair loop and over large result sets, which is what the thesis needed to validate the workflow concept end to end; validating full result sets for large queries is non-trivial (in transfer, serialization, and comparison cost) and is deferred to future work (section 9.4). Equivalence is also established against *one* database state, so when it changes mid-translation, the workflow may fail.

*Latency and cost.* An end-to-end verified translation of the fifteen-query workload takes, under normal load, roughly **half an hour** for a document target and up to about 1.2 **hours** for a graph target (section 8.7), dominated by our provider’s generation throughput and sandbox compilation, and each repair iteration repeats the expensive stages. The assistant is thus an *offline* migration tool, not an interactive aid.

*Interaction model.* The user can intervene only at the designed hand-off point, after the repair budget is exhausted (non-goal N4). The web interface has known cosmetic issues from incompatibilities between the frontend component library and the LangGraph streaming runtime, but these affect presentation, not recorded state or result validity.

## 9.3 Threats to Validity

Following the standard taxonomy of experimentation in software engineering [69], the threats are grouped in table 9.1, each with its mitigation. Two points deserve emphasis beyond the table. The most consequential construct-validity risk is that the deterministic checker reports a difference but the *judge* decides whether it matters, and that decision **can be wrong in the accepting direction**: for example, the judge accepting a rendering artifact could be a real defect. This is why *strict* equivalence – which never consults the judge – is the headline metric everywhere and the sole basis of the cross-system comparison, with judge-inclusive acceptance reported only as a labelled supplementary column (section 8.3), and the judge is scored against ground truth rather than trusted (section 8.8).

| Category   | Threat                                                                                                                                                 | Mitigation                                                                                                                                                                                      |
|------------|--------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Construct  | Sample-based equivalence (counts + boundaries); serializer tolerances can over- or under-accept; judge may wrongly <i>accept</i> .                     | Strict equivalence is the headline metric and never routed through the judge; tolerances constant across runs; judge scored against ground truth; CodeBLEU [32] used only as a stability proxy. |
| Internal   | Judge-generator lineage may correlate; prompt caching would deflate cost; the baseline changed model <i>and</i> workload alongside design.             | Judge scored against execution-derived truth; per-run nonce defeats caching; residual errors decomposed per failure mode (section 8.6); interrupted runs recorded, not silently retried.        |
| External   | One relational schema (WideWorldImporters [40]), one direction, pinned versions; graph difficulty is partly a property of <i>that</i> populated store. | Three source dialects and two target paradigms (G3); all prompts, traces, and configurations preserved for re-runs (section A.2).                                                               |
| Conclusion | Per-run cost bounds sample size (ten repetitions); outputs are not fully deterministic [8, 65]; model deprecation threatens repeatability.             | Dispersion reported with results; single-shot arm variance shown directly (section 8.9); the recording proxy preserves interactions for offline re-analysis (section 7.5).                      |

**Table 9.1** The threats to the validity of the evaluation’s claims, by category [69], with the mitigation applied to each.

*Input dependence* is a threat spanning construct and external validity: the assistant’s measured accuracy is conditioned on two artifacts the *user* supplies and the assistant treats as **immutable ground truth** – the ETL-derived target mapping and the source code itself (section 8.2). On the target side, the store is provisioned by off-the-shelf ETL tooling with default heuristics and is never modified, so an unidiomatic mapping transfers its cost to the translation: a one-to-one table-to-document mapping that keeps every relationship inter-document gives up the denormalization a document store is designed around; over-eager embedding inflates every read of an aggregate’s parent; and a graph built one-to-one from the code – every table a separate node class with the relationship declared on both endpoints – is exactly the bidirectional-mapping pattern the Spring Data Neo4j documentation warns against.<sup>3</sup> On the source side, input quality bounds accuracy just as directly: a resource-heavy or invalid query, an incorrect mapping annotation, or source that does not compile against the stated version degrades the outcome proportionally. Most subtly, because the assistant *repairs the user’s queries by default*, the judge has to compare the translation against a source that may itself be wrong – so the equivalence measured is equivalence to the input *as given*. To mitigate

<sup>3</sup>Spring Data Neo4j cautions against mapping the same relationship from both participating entities; see the relationship-mapping reference: <https://docs.spring.io/spring-data/neo4j/reference/object-mapping/metadata-based-mapping.html>.

this, the workload of section 8.2 uses idiomatic, compiling source and the tools’ default mappings, and every input is preserved for inspection (section A.2).

## 9.4 Future Work

*Widening the translation space.* The most direct extension exercises the mechanism of section 7.7 beyond its five frameworks – the *reverse* direction (Spring Data back to .NET ORMs), relational-to-relational pairs, and further ecosystems such as other popular object-mappers (Python’s *SQLAlchemy*, *Django ORM*, or JavaScript/TypeScript’s *Prisma*, etc.), each one manifest, two harness snippets, a skill document, and a prompt example. A deeper extension lifts the unit of translation from a pasted schema-and-queries pair to a *repository*: the ACP-served Deep Agent of section 6.7 already reads the workspace, so coupling its filesystem view and shell access with the validated pipeline would make the assistant a migration tool in the everyday conversational sense. Supporting *write* operations is the hardest step, since equivalence for state-mutating code requires comparing database states rather than result sets.

*Strengthening the equivalence argument.* More robust query result-set equivalence checking is needed. For example, via *adaptation to result-set size*: e.g. for small results, compare the *full* serialized output rather than boundary samples; for large results, sample structurally or use a streaming, map-reduce-style reduction in which each side hashes its canonically serialized records (e.g., using a bloom filter) and only the residual disagreement is diffed precisely. Conversely, running each query against *multiple* database states (including empty ones) would stop trivially satisfied predicates passing unnoticed; mutation analysis [35] would turn the qualitative trust argument of section 5.6 into a measured detection rate; or bounded *formal* checking could help, by reducing both queries to a common algebraic form and testing equivalence symbolically. A practical companion is validation *without a deployed target*: today the checker presupposes a live, ETL-populated target store (non-goal N1), whereas an ephemeral instance seeded on demand with a representative sample of the source data would let a team validate a candidate translation *before* committing to the data migration.

*Performance-aware translation and pipeline efficiency.* The assistant optimizes for correctness, not performance (non-goal N2). The natural next step closes the loop with the ORMorpher selection line [12, 10]: the overhead characterization of chapter 3 and the integer-linear-programming (ILP) advisor supply the cost model, and the assistant the migration mechanism, so the combined system could *recommend* a target framework and *produce* its verified implementation. The graph-target latency of section 8.7 could be improved (e.g., warm project skeletons, incremental compilation, speculative parallel generation, etc.); and every accepted run is a training pair, a corpus for fine-tuning a specialized generator. Next optimization could be an initial semi-deterministic “Preflight Stage” that checks the database and MCP availability, incorrect data types (e.g., SQL’s

datetime → Neo4j string - is an incorrect mapping, should be datetime, etc.), circular references, cartesian products, or read-only database access (so that LLM does not accidentally overwrite data). On the ecosystem side, publishing the validation sandboxes and the equivalence checker as standalone MCP services would let third-party coding agents borrow exactly what they need to verify their own translations.

*Optimizing the Translation Generation stage.* Since we already iterated on multiple designs and converged back to a ReAct agent (section 4.3.3), we could move forward with it. When the ReAct agent decides it is unable to perform its task, it could utilize a special advisor subagent with a more capable model (e.g., GLM 5.2) to handle more troublesome queries, and equip it with all the tools, especially database, documentation, and validation tools, skills loaded on demand, and a specialized prompt distilled from its parent and state. Another option is to parallelize source and target harness generation in discrete subagents, with a post-aggregation step in the parent. Furthermore, earlier experiments showed that the model tends to generate raw Cypher/MQL better than code, because its parametric memory is already heavily trained on it – we could use it as an intermediate layer, e.g. get SQL from LINQ (via ADO.NET methods), then translate SQL to MQL (via existing research tools [70, 71], or LLM), and finally MQL to target via LLM – then we would only need a skill for mapping MQL/Cypher to target, and since target can deterministically generate MQL/Cypher, we can easily compare that, and only rely on our validator for final check. Last but not least, we could give an agentic-tuned model filesystem and shell access in the sandboxes and see how it performs with/without specialized validation tools with a custom system prompt distilled from existing workflow prompts (already drafted in section 6.7 as a Deep Agent).

*Beyond the persistence layer: a Universal Framework.* The furthest-reaching direction generalizes the thesis’s core idea from the database boundary to the whole application. This work tackled the boundary between the data layer and the application layer in a three-tier architecture model [72]. Viewed from a *data-flow* perspective, an application is a network of places where structured data crosses a boundary – method input/output signatures, REST endpoints with their request/response schemas and resolver methods, classes with their attributes and methods with signatures – and wherever such a boundary can be identified, its input/output shape recovered, and its data serialized into a common inter-framework representation, the same generate–compile–execute–compare workflow applies. Using our workflow, one can now easily write a prompt – “translate my ASP.NET EF Core data-persistence layer (schemas, queries, settings) to Java Spring Data MongoDB; I already have a MongoDB database; scaffold a new Spring Boot application and use the Universal Object Mapping assistant MCP tool to perform the persistence translation” – in which a capable coding agent (e.g. Claude Code) scaffolds the new application while delegating the persistence layer to our assistant.



# Conclusion

This thesis asked three questions: whether persistence-layer migration *across data-model paradigms* can be automated with correctness evidence grounded in *execution* rather than surface plausibility; whether a language model is more reliable inside a semi-deterministic, verification-first workflow than delegated the whole task as an autonomous agent; and whether such a workflow, driven by an *open, mid-size* model, can stand with the frontier-scale coding agents of the largest *AI* companies. The *Universal Object Mapping (UOM) Assistant* built and evaluated here answers the first two in the affirmative with quantitative evidence, and provides strong preliminary evidence for the third.

Firstly, the assistant translates the persistence-layer artifacts of three .NET source dialects (*LINQ*, *HQL*, raw *SQL*) into two non-relational paradigms (document and graph), crossing the relational/document and relational/graph boundaries that have no faithful one-to-one image (goals *G1*, *G3*). Its central commitment (chapters 4 and 5) is to fix the order of generation, compilation, execution, comparison, and judgment in the graph itself, so the decisive evidence for every translation is produced by deterministic tools – real compilers, live databases, a structural equivalence check – and interpreted by an independent judge only at the boundaries (*G2*). Across sixty traced runs of the fifteen-query workload (ten repetitions on each of the six framework pairs, 900 query translations in all; chapter 8), **80.7% were strictly execution-equivalent** to their source – 96.7% on the document targets and 64.7% on the graph targets – and every pair produced at least one perfect run.

Secondly, an initial fully autonomous prototype showed exactly the failures the literature attributes to premature autonomy – skipped verification, exhausted step budgets, context poisoned by verbose tool output, runs too divergent to compare – and was abandoned (section 5.1). The semi-deterministic workflow that replaced it is measurably more dependable: 85% of runs finalized with no human intervention (requirement *R4*), and bounded, execution-grounded self-repair recovered from the routine failure modes of model code generation (*R3*). The redesign that made this possible – framework knowledge delivered as compile-verified skills, fragment-wise persistence, and an in-loop execution probe *aligned* with the final acceptance oracle – moved the graph targets from near-categorical failure to routine success.

Thirdly, in a controlled comparison under an identical task and an identical equivalence checker, the workflow driven by an *open, mid-size* model **matched the strongest frontier agent and outperformed the rest**: it equalled a state-of-the-art general-purpose agent on a frontier model at roughly a third of the cost, beat the remaining frontier arms, and offered a run-to-run repeatability that single-shot systems structurally cannot (section 8.9). Because the strongest external arms could be captured only once each (a cost-bounded limitation stated with the experiment), this study *suggests* rather than proves the parity; the claim it does support is the sharper, defensible one that *wherever a task uses a genuine execution oracle, an aligned domain workflow can substitute for raw model scale* – and can do so more cheaply and reproducibly.

Beyond the three questions, the assistant is delivered as an interactive, fully traced web application with programmatic [MCP](#) and [ACP](#) interfaces ([G4](#), chapter 6) and is architected so that adding a framework is a mechanical extension of five declarative artifacts (section 7.7); an additional benchmark quantified the [ORM](#) time and memory overhead of the .NET sources ([G6](#), chapter 3). Every goal is met and every requirement demonstrated experimentally. The limitations are stated plainly (sections 9.2 and 9.3): equivalence is execution-grounded *evidence*, not proof – established against one database state, over sampled boundaries of each result set, and verified by a non-deterministic [LLM](#) judge – and the study covers one schema, one direction, and read queries.

The future-work agenda follows directly from those boundaries (section 9.4): a size-adaptive equivalence contract that closes the sampling blind spot, support for write operations, performance-aware target selection, and with more effort, lifting the same verified-workflow from the database boundary up into the application layer.



# Declaration of AI Usage

In accordance with Charles University’s policy on the use of generative artificial intelligence in academic work,<sup>4</sup> I declare the following.

Portions of this thesis made use of AI-assisted tools, specifically large language models, to support tasks such as documenting and debugging source code, proofreading and refining the language of the text, and drafting supplementary material such as explanatory figures. Since this thesis uses language models as main part of the delivered workflow, the prompt and context engineering required iterative refinement with the help of the associated language models, including drafting the scripts used to automate experiments in chapter 8. LLM-assisted help with the rapid redesign of the generation stage was needed to handle unfortunate sudden issues encountered with unexpected external LLM provider’s system changes (e.g. deprecation of models used heavily in the previous iterations, incompatibility with OpenAI specification, and much more, explained in chapter 9) and external toolings, that were not in our own capacity to fully control, and were observed only few weeks before thesis submission deadline.

All technical and theoretical content, experimental results, methodology, and conclusions presented in this thesis – including the design of the Universal Object Mapping Assistant, the workflow architecture, the evaluation methodology, and the reported results – are the sole work of the author and were independently verified. After using the aforementioned artificial intelligence tools, I thoroughly revised and modified the content as required and accept full responsibility for the resulting form of the thesis.

---

<sup>4</sup><https://ai.cuni.cz/AI-81.html>

# Bibliography

- [1] Martin Fowler. *Patterns of Enterprise Application Architecture*. Boston, MA: Addison-Wesley Professional, 2002. ISBN: 978-0-321-12742-6.
- [2] Pramod J. Sadalage and Martin Fowler. *NoSQL Distilled: A Brief Guide to the Emerging World of Polyglot Persistence*. Upper Saddle River, NJ: Addison-Wesley Professional, 2012. ISBN: 978-0-321-82662-6.
- [3] Pavel Koupil, Jáchym Bártík, and Irena Holubová. “Model-Agnostic Evolution Management”. In: *Research Challenges in Information Science - 19th International Conference, RCIS 2025, Seville, Spain, May 20-23, 2025, Proceedings, Part I*. Ed. by Janis Grabis et al. Vol. 547. Lecture Notes in Business Information Processing. Springer, 2025, pp. 245–261. DOI: [10.1007/978-3-031-92474-3\\_15](https://doi.org/10.1007/978-3-031-92474-3_15).
- [4] Ali Davoudian, Liu Chen, and Mengchi Liu. “A Survey on NoSQL Stores”. In: *ACM Computing Surveys* 51.2 (Apr. 2018), 40:1–40:43. DOI: [10.1145/3158661](https://doi.org/10.1145/3158661).
- [5] Anthropic. *Claude Code: Deep Coding at Terminal Velocity*. 2026. URL: <https://code.claude.com/docs/en/overview> (visited on 07/11/2026).
- [6] Google. *Build with Google Antigravity, our new agentic development platform*. 2026. URL: <https://antigravity.google/> (visited on 07/11/2026).
- [7] Carlos E. Jimenez et al. “SWE-bench: Can Language Models Resolve Real-World GitHub Issues?” In: *The Twelfth International Conference on Learning Representations (ICLR)*. 2024. URL: <https://arxiv.org/abs/2310.06770>.
- [8] Shuyin Ouyang et al. “An Empirical Study of the Non-determinism of ChatGPT in Code Generation”. In: *ACM Transactions on Software Engineering and Methodology* 34.2 (2025), pp. 1–28. DOI: [10.1145/3697010](https://doi.org/10.1145/3697010). URL: <https://arxiv.org/abs/2308.02828>.
- [9] Pavel Koupil, Martin Svoboda, and Irena Holubová. “MM-cat: A Tool for Modeling and Transformation of Multi-Model Data Using Category Theory”. In: *2021 ACM/IEEE International Conference on Model Driven Engineering Languages and Systems Companion (MODELS-C)*. 2021, pp. 635–639. DOI: [10.1109/MODELS-C53483.2021.00098](https://doi.org/10.1109/MODELS-C53483.2021.00098).

- [10] Milan Abrahám and Pavel Koupil. “ORMorpher: An Interactive Framework for ORM Translation and Optimization”. In: *2025 40th IEEE/ACM International Conference on Automated Software Engineering (ASE)*. Seoul, Korea, Republic of: IEEE Press, 2025, pp. 4037–4040. DOI: [10.1109/ASE63991.2025.00369](https://doi.org/10.1109/ASE63991.2025.00369). URL: <https://dl.acm.org/doi/10.1109/ASE63991.2025.00369> (visited on 03/31/2026).
- [11] Milan Abrahám. “Framework-Agnostic Query Adaptation: Ensuring SQL Compatibility Across .NET Database Frameworks”. In: (Sept. 2025). URL: <https://dspace.cuni.cz/handle/20.500.11956/203083> (visited on 03/31/2026).
- [12] Milan Abrahám and Pavel Koupil. *A Unified Framework for Object-Relational Mapping Translation and Performance-Aware Selection*. Manuscript; Department of Software Engineering, Charles University. 2026.
- [13] Tom B. Brown et al. “Language Models Are Few-Shot Learners”. In: *Advances in Neural Information Processing Systems (NeurIPS)*. Vol. 33. 2020, pp. 1877–1901. URL: <https://arxiv.org/abs/2005.14165>.
- [14] Mark Chen, Jerry Tworek, Heewoo Jun, et al. “Evaluating Large Language Models Trained on Code”. In: *arXiv preprint arXiv:2107.03374* (2021). URL: <https://arxiv.org/abs/2107.03374>.
- [15] Baptiste Roziere et al. “Unsupervised translation of programming languages”. In: *Proceedings of the 34th International Conference on Neural Information Processing Systems*. NIPS ’20. Curran Associates Inc., 2020. ISBN: 9781713829546. URL: <https://dl.acm.org/doi/10.5555/3495724.3497454>.
- [16] Joseph Spracklen et al. “We Have a Package for You! A Comprehensive Analysis of Package Hallucinations by Code Generating LLMs”. In: *34th USENIX Security Symposium*. 2025. URL: <https://arxiv.org/abs/2406.10279>.
- [17] Minghua He et al. “ExeCoder: Empowering Large Language Models with Executability Representation for Code Translation”. In: (2025). arXiv: [2501.18460](https://arxiv.org/abs/2501.18460) [cs.SE]. URL: <https://arxiv.org/abs/2501.18460>.
- [18] Jaydeep Sen et al. “ATHENA++: Natural Language Querying for Complex Nested SQL Queries”. In: *Proceedings of the VLDB Endowment* 13.12 (2020), pp. 2747–2759. URL: <https://dl.acm.org/doi/10.14778/3407790.3407858>.
- [19] Jason Wei et al. “Chain-of-thought prompting elicits reasoning in large language models”. In: *Proceedings of the 36th International Conference on Neural Information Processing Systems*. NIPS ’22. Curran Associates Inc., 2022. ISBN: 9781713871088. URL: <https://dl.acm.org/doi/10.5555/3600270.3602070>.
- [20] Shunyu Yao et al. *Tree of Thoughts: Deliberate Problem Solving with Large Language Models*. Dec. 2023. DOI: [10.48550/arXiv.2305.10601](https://doi.org/10.48550/arXiv.2305.10601). arXiv: [2305.10601](https://arxiv.org/abs/2305.10601) [cs.CL]. URL: <http://arxiv.org/abs/2305.10601> (visited on 07/02/2026).

- [21] Shunyu Yao et al. “ReAct: Synergizing Reasoning and Acting in Language Models”. In: *International Conference on Learning Representations (ICLR)*. 2023. URL: <https://arxiv.org/abs/2210.03629>.
- [22] Anthropic. *Building Effective Agents*. Anthropic. 2024. URL: <https://www.anthropic.com/research/building-effective-agents> (visited on 07/01/2026).
- [23] Yiran Wu et al. “StateFlow: Enhancing LLM Task-Solving through State-Driven Workflows”. In: (2024). URL: <https://arxiv.org/abs/2403.11322>.
- [24] LangChain. *LangGraph: Building Stateful, Multi-Actor Applications with LLMs*. 2026. URL: <https://langchain-ai.github.io/langgraph/> (visited on 06/23/2026).
- [25] Nelson F. Liu et al. “Lost in the Middle: How Language Models Use Long Contexts”. In: *Transactions of the Association for Computational Linguistics* 12 (2024), pp. 157–173. DOI: [10.1162/tac1\\_a\\_00638](https://doi.org/10.1162/tac1_a_00638). URL: <https://aclanthology.org/2024.tac1-1.9/>.
- [26] Patrick Lewis et al. “Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks”. In: *Advances in Neural Information Processing Systems* 34 (*NeurIPS*). 2020. URL: <https://arxiv.org/abs/2005.11401>.
- [27] Anthropic. *Effective Context Engineering for AI Agents*. Anthropic. 2025. URL: <https://www.anthropic.com/engineering/effective-context-engineering-for-ai-agents> (visited on 07/01/2026).
- [28] Xinyun Chen et al. “Teaching Large Language Models to Self-Debug”. In: *The Twelfth International Conference on Learning Representations (ICLR)*. 2024. URL: <https://arxiv.org/abs/2304.05128>.
- [29] Theo X. Olausson et al. “Is Self-Repair a Silver Bullet for Code Generation?” In: *The Twelfth International Conference on Learning Representations (ICLR)*. 2024. URL: <https://arxiv.org/abs/2306.09896>.
- [30] Zhihao Lin et al. “To Run or Not to Run: Analyzing the Cost-Effectiveness of Code Execution in LLM-Based Program Repair”. In: *Proceedings of the ACM SIGSOFT International Symposium on Software Testing and Analysis (ISSTA 2026)*. arXiv:2606.26978. 2026. URL: <https://arxiv.org/abs/2606.26978>.
- [31] Lianmin Zheng et al. “Judging LLM-as-a-Judge with MT-Bench and Chatbot Arena”. In: *Advances in Neural Information Processing Systems (NeurIPS)*. 2023. URL: <https://arxiv.org/abs/2306.05685>.
- [32] Shuo Ren et al. “CodeBLEU: a Method for Automatic Evaluation of Code Synthesis”. In: *arXiv preprint arXiv:2009.10297* (2020). URL: <https://arxiv.org/abs/2009.10297>.

- [33] Sep Dehpour. *DeepDiff: Deep Difference and Search of Any Python Object/Data*. 2026. URL: <https://zepworks.com/deepdiff/current/> (visited on 06/23/2026).
- [34] Colin White et al. “LiveBench: A Challenging, Contamination-Limited LLM Benchmark”. In: *The Thirteenth International Conference on Learning Representations*. Apr. 2025. DOI: [10.48550/arXiv.2406.19314](https://arxiv.org/abs/2406.19314). URL: <http://arxiv.org/abs/2406.19314> (visited on 06/30/2026).
- [35] Yue Jia and Mark Harman. “An Analysis and Survey of the Development of Mutation Testing”. In: *IEEE Transactions on Software Engineering* 37.5 (2011), pp. 649–678. DOI: [10.1109/TSE.2010.62](https://doi.org/10.1109/TSE.2010.62).
- [36] Christopher Ireland et al. “A Classification of Object-Relational Impedance Mismatch”. In: *2009 First International Conference on Advances in Databases, Knowledge, and Data Applications (DBKDA)*. IEEE, 2009, pp. 36–43. DOI: [10.1109/DBKDA.2009.11](https://doi.org/10.1109/DBKDA.2009.11).
- [37] *What Is Relational Migrator - Relational Migrator - MongoDB Docs*. URL: <https://www.mongodb.com/docs/relational-migrator/> (visited on 06/23/2026).
- [38] *Neo4j-Contrib/Neo4j-Etl*. Neo4j Contrib. May 2026. URL: <https://github.com/neo4j-contrib/neo4j-etl> (visited on 06/23/2026).
- [39] ISO/IEC. *ISO/IEC 9075-1:2023 – Information Technology – Database Languages SQL – Part 1: Framework (SQL/Framework)*. International Organization for Standardization, June 2023. URL: <https://www.iso.org/standard/76583.html> (visited on 07/15/2026).
- [40] Microsoft. *WideWorldImporters Sample Database*. 2026. URL: <https://learn.microsoft.com/en-us/sql/samples/wide-world-importers-what-is> (visited on 06/23/2026).
- [41] Microsoft. *Cardinality Estimation (SQL Server)*. Microsoft. URL: <https://learn.microsoft.com/en-us/sql/relational-databases/performance/cardinality-estimation-sql-server> (visited on 07/14/2026).
- [42] Microsoft. *Statistics (SQL Server)*. Microsoft. 2024. URL: <https://learn.microsoft.com/en-us/sql/relational-databases/statistics/statistics> (visited on 07/14/2026).
- [43] Viktor Leis et al. “How Good Are Query Optimizers, Really?” In: *Proceedings of the VLDB Endowment*. Vol. 9. 3. 2015, pp. 204–215. URL: <https://doi.org/10.14778/2850583.2850594>.
- [44] Frans Bouma. *FransBouma/RawDataAccessBench*. June 27, 2026. URL: <https://github.com/FransBouma/RawDataAccessBench> (visited on 06/27/2026).

- [45] Martin Čorovčák. *corovcam/Universal-Object-Mapping*. 2026. URL: <https://github.com/corovcam/Universal-Object-Mapping> (visited on 06/23/2026).
- [46] Andrey Akinshin and the BenchmarkDotNet contributors. *BenchmarkDotNet: Powerful .NET Library for Benchmarking*. 2026. URL: <https://benchmarkdotnet.org/> (visited on 03/08/2026).
- [47] Matei Zaharia et al. *The Shift from Models to Compound AI Systems*. Berkeley Artificial Intelligence Research (BAIR). Feb. 2024. URL: <https://bair.berkeley.edu/blog/2024/02/18/compound-ai-systems/> (visited on 07/15/2026).
- [48] e-INFRA CZ. *e-INFRA CZ AI-as-a-Service Documentation*. 2026. URL: <https://docs.cerit.io/en/docs/ai-as-a-service/introduction> (visited on 06/23/2026).
- [49] Jie Huang et al. “Large Language Models Cannot Self-Correct Reasoning Yet”. In: *The Twelfth International Conference on Learning Representations (ICLR)*. 2024. URL: <https://arxiv.org/abs/2310.01798>.
- [50] XRG - XML and Web Engineering Research Group at KSI MFF UK. URL: <https://xrg.cz/> (visited on 04/01/2026).
- [51] Moonshot AI. *Kimi-K2.7-Code*. Hugging Face. 2026. URL: <https://huggingface.co/moonshotai/Kimi-K2.7-Code> (visited on 06/30/2026).
- [52] Qwen Team, Alibaba. *Qwen3.5-122B-A10B-FP8*. Hugging Face. 2026. URL: <https://huggingface.co/Qwen/Qwen3.5-122B-A10B-FP8> (visited on 06/30/2026).
- [53] DeepSeek-AI. *DeepSeek-V4-Pro*. Hugging Face. URL: <https://huggingface.co/deepseek-ai/DeepSeek-V4-Pro> (visited on 06/30/2026).
- [54] Qwen Team, Alibaba. *Qwen3.5-397B-A17B-FP8*. Hugging Face. 2026. URL: <https://huggingface.co/Qwen/Qwen3.5-397B-A17B-FP8> (visited on 06/30/2026).
- [55] OpenAI. *gpt-oss-120b*. Hugging Face. 2025. URL: <https://huggingface.co/openai/gpt-oss-120b> (visited on 06/30/2026).
- [56] Ziwei Ji et al. “Survey of Hallucination in Natural Language Generation”. In: *ACM Computing Surveys* 55.12 (Mar. 2023), pp. 1–38. DOI: [10.1145/3571730](https://doi.org/10.1145/3571730).
- [57] Anthropic. *Model Context Protocol*. URL: <https://modelcontextprotocol.io/> (visited on 06/23/2026).
- [58] Tiantian Gan and Qiyao Sun. *RAG-MCP: Mitigating Prompt Bloat in LLM Tool Selection via Retrieval-Augmented Generation*. May 2025. URL: <https://arxiv.org/abs/2505.03275>.
- [59] Daytona. *Daytona: Secure and Elastic Infrastructure for Running AI-Generated Code*. 2026. URL: <https://www.daytona.io/> (visited on 06/23/2026).



- [60] Grzegorz Malewicz et al. “Pregel: A System for Large-Scale Graph Processing”. In: (2010), pp. 135–146. doi: [10.1145/1807167.1807184](https://doi.org/10.1145/1807167.1807184).
- [61] Simon Brown. *The C4 Model for Visualising Software Architecture*. 2024. URL: <https://c4model.com/> (visited on 07/01/2026).
- [62] Alper Karaca. *Alpeerkaraca/Everspring-Mcp*. June 2026. URL: <https://github.com/alpeerkaraca/everspring-mcp> (visited on 07/16/2026).
- [63] Xinyi Hou et al. *Model Context Protocol (MCP): Landscape, Security Threats, and Future Research Directions*. Mar. 2025. arXiv: [2503.23278](https://arxiv.org/abs/2503.23278).
- [64] Anthropic. *Agent Skills*. 2025. URL: <https://platform.claude.com/docs/en/agents-and-tools/agent-skills/overview> (visited on 07/11/2026).
- [65] Horace He and Thinking Machines Lab. *Defeating Nondeterminism in LLM Inference*. Thinking Machines Lab. Sept. 2025. doi: [10.64434/tml.20250910](https://doi.org/10.64434/tml.20250910). URL: <https://thinkingmachines.ai/blog/defeating-nondeterminism-in-llm-inference/> (visited on 07/15/2026).
- [66] Konstantin Chernyshev. *K4black/Codebleu*. June 2026. URL: <https://github.com/k4black/codebleu> (visited on 06/23/2026).
- [67] Moonshotai/Kimi-K2.6 · Hugging Face. URL: <https://huggingface.co/moonshotai/Kimi-K2.6> (visited on 07/13/2026).
- [68] Jeffrey Cheng et al. “Dated Data: Tracing Knowledge Cutoffs in Large Language Models”. In: *COLM 2024*. 2024. URL: <https://arxiv.org/abs/2403.12958>.
- [69] Claes Wohlin et al. *Experimentation in Software Engineering*. Berlin, Heidelberg: Springer, 2012. doi: [10.1007/978-3-642-29044-2](https://doi.org/10.1007/978-3-642-29044-2).
- [70] Ramon Lawrence. “Integration and Virtualization of Relational SQL and NoSQL Systems Including MySQL and MongoDB”. In: *2014 International Conference on Computational Science and Computational Intelligence (CSCI)*. Vol. 1. Las Vegas, NV, USA: IEEE, 2014, pp. 285–290. doi: [10.1109/CSCI.2014.56](https://doi.org/10.1109/CSCI.2014.56).
- [71] Muon Ha and Yulia Alexandrovna Shichkina. “The Query Translation from MySQL to MongoDB Taking into Account the Structure of the Database”. In: *2021 IEEE Conference of Russian Young Researchers in Electrical and Electronic Engineering (ElConRus)*. IEEE, 2021, pp. 383–386. doi: [10.1109/ElConRus51938.2021.9396591](https://doi.org/10.1109/ElConRus51938.2021.9396591).
- [72] Len Bass, Paul Clements, and Rick Kazman. *Software Architecture in Practice, 4th Edition*. ISBN: 978-0-13-688597-9. URL: <https://www.oreilly.com/library/view/software-architecture-in/9780136885979/> (visited on 07/12/2026).

# List of Figures

|     |                                                                                                                                                                                                                                           |    |
|-----|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----|
| 2.1 | The same relational association translated into the two target models. The document model must choose between embedding and referencing the orders; the graph model turns the entities into nodes joined by a typed relationship. . . . . | 18 |
| 2.2 | The same relational association as figure 2.1 translated into the two target models, but using the ORM/ODM/OGM framework abstraction. . . . .                                                                                             | 19 |
| 3.1 | Per-query ORM overhead . . . . .                                                                                                                                                                                                          | 25 |
| 3.2 | Database execution masks ORM overhead . . . . .                                                                                                                                                                                           | 27 |
| 4.1 | Translation State Machine Diagram (Schema and Queries) . . . . .                                                                                                                                                                          | 30 |
| 4.2 | EFCore to Spring Data MongoDB Translation Trace . . . . .                                                                                                                                                                                 | 35 |
| 4.3 | Dapper to Spring Data Neo4j Translation Trace . . . . .                                                                                                                                                                                   | 36 |
| 5.1 | The four verification layers . . . . .                                                                                                                                                                                                    | 47 |
| 6.1 | The translation workspace . . . . .                                                                                                                                                                                                       | 57 |
| 6.2 | The configuration panel . . . . .                                                                                                                                                                                                         | 58 |
| 6.3 | Following a run . . . . .                                                                                                                                                                                                                 | 61 |
| 6.4 | A completed run . . . . .                                                                                                                                                                                                                 | 62 |
| 7.1 | UOM system context (C4 level 1) . . . . .                                                                                                                                                                                                 | 65 |
| 7.2 | UOM container view (C4 level 2) . . . . .                                                                                                                                                                                                 | 66 |
| 7.3 | UOM Orchestrator Component view (C4 level 3) . . . . .                                                                                                                                                                                    | 68 |
| 8.1 | Relational source schema (SQL Server) . . . . .                                                                                                                                                                                           | 77 |
| 8.2 | MongoDB target store . . . . .                                                                                                                                                                                                            | 78 |
| 8.3 | Neo4j target store . . . . .                                                                                                                                                                                                              | 79 |
| 8.4 | Baseline: the initial workflow design . . . . .                                                                                                                                                                                           | 81 |
| 8.5 | Strict execution equivalence per pair . . . . .                                                                                                                                                                                           | 84 |
| 8.6 | Strict pass@k per pair . . . . .                                                                                                                                                                                                          | 84 |
| 8.7 | Outcome decomposition of all query slots . . . . .                                                                                                                                                                                        | 85 |
| 8.8 | Per-query strict equivalence . . . . .                                                                                                                                                                                                    | 86 |



|      |                                                        |    |
|------|--------------------------------------------------------|----|
| 8.9  | Resource consumption per pair . . . . .                | 87 |
| 8.10 | Per-stage resource attribution . . . . .               | 87 |
| 8.11 | Graded judges vs. deterministic ground truth . . . . . | 88 |
| 8.12 | Head-to-head strict equivalence . . . . .              | 90 |
| 8.13 | Head-to-head resources . . . . .                       | 91 |
| 8.14 | CodeBLEU per arm . . . . .                             | 92 |

# List of Tables

|     |                                                                                                                                                                             |     |
|-----|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----|
| 2.1 | Supported frameworks, data models, query interfaces, and pinned versions. The source frameworks run on .NET 10; the target frameworks on Java 25 / Spring Boot 4.0. . . . . | 12  |
| 3.1 | ORM benchmark query categories . . . . .                                                                                                                                    | 22  |
| 3.2 | Testing environment for the overhead experiment. . . . .                                                                                                                    | 24  |
| 4.1 | The nodes of the translation workflow . . . . .                                                                                                                             | 31  |
| 4.2 | The model portfolio . . . . .                                                                                                                                               | 33  |
| 4.3 | Execution trace of the running example . . . . .                                                                                                                            | 34  |
| 5.1 | The state interfaces . . . . .                                                                                                                                              | 44  |
| 5.2 | Failure-to-action routing . . . . .                                                                                                                                         | 47  |
| 8.1 | The fifteen-query evaluation workload . . . . .                                                                                                                             | 76  |
| 8.2 | Headline results per framework pair . . . . .                                                                                                                               | 83  |
| 8.3 | Head-to-head comparison with general-purpose agents . . . . .                                                                                                               | 90  |
| 9.1 | Threats to validity and mitigations . . . . .                                                                                                                               | 96  |
| B.1 | Contextual snippet files and entry-point class names per framework. .                                                                                                       | 126 |

# List of Listings

|     |                                                                                                                                            |     |
|-----|--------------------------------------------------------------------------------------------------------------------------------------------|-----|
| 2.1 | Source schema – an Entity Framework Core entity (LINQ source dialect).                                                                     | 13  |
| 2.2 | Source query – a LINQ expression over the entity of listing 2.1.                                                                           | 13  |
| 2.3 | Target schema, option A – Spring Data MongoDB with the orders held as a <code>@DocumentReference</code> (inter-document, normalized).      | 14  |
| 2.4 | Target query, option A – the LINQ expression of listing 2.2 as a MongoDB aggregation pipeline over referenced documents.                   | 15  |
| 2.5 | Target schema, option B – Spring Data MongoDB with the orders <i>embedded</i> as a nested array (intra-document).                          | 15  |
| 2.6 | Target query, option B – the same LINQ expression as a single <code>find</code> with an <code>\$elemMatch</code> over the embedded array.  | 16  |
| 2.7 | Target schema, option C – Spring Data Neo4j with the orders reached over a <code>PLACED</code> relationship.                               | 16  |
| 2.8 | Target query, option C – the same LINQ expression as a Cypher-DSL relationship traversal in Spring Data Neo4j.                             | 17  |
| 5.1 | Source-side (.NET) result digest                                                                                                           | 52  |
| 5.2 | Target-side (Java) result digest                                                                                                           | 53  |
| 5.3 | A DeepDiff equivalence report                                                                                                              | 54  |
| 8.1 | A representative EF source query from the workload (Q10).                                                                                  | 75  |
| B.1 | The extraction schema (from <code>react_agent/graph.py</code> ).                                                                           | 118 |
| B.2 | The <code>save_translation</code> tool contract (from <code>react_agent/translation_draft.py</code> ).                                     | 119 |
| B.3 | The judge and human-intervention verdict schemas.                                                                                          | 119 |
| B.4 | The shared graph state (from <code>react_agent/state.py</code> ).                                                                          | 120 |
| B.5 | System prompt of the input-extraction stage.                                                                                               | 121 |
| B.6 | System prompt of the schema-inspection stage.                                                                                              | 122 |
| B.7 | Partial system prompt of the translation-generation stage.                                                                                 | 124 |
| B.8 | System prompt of the finalization stage.                                                                                                   | 125 |
| B.9 | The invariant .NET serializer configuration (from <code>EFCoreQueryEntryPoint.cs</code> ); the Java prelude mirrors every rule in Jackson. | 127 |

|      |                                                                                                                                                                    |     |
|------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----|
| B.10 | The count/boundary-sample digest producer of the EF Core query harness; each framework implements the same contract in its own idiom. .                            | 128 |
| B.11 | The source-side (.NET, EF Core) result digest of the worked example. .                                                                                             | 129 |
| B.12 | The target-side (Java, Spring Data MongoDB) result digest of the worked example. . . . .                                                                           | 130 |
| B.13 | The equivalence report of the worked example, as computed by the parameters of section 5.7.3 and rendered into the isolated message channel for the judge. . . . . | 131 |
| B.14 | A complete source schema input . . . . .                                                                                                                           | 135 |
| B.15 | The complete fifteen-query source input . . . . .                                                                                                                  | 136 |
| B.16 | Entity Framework Core 10 .csproj file for the .NET sandbox project. .                                                                                              | 137 |
| B.17 | NHibernate .csproj file for the .NET sandbox project. . . . .                                                                                                      | 137 |
| B.18 | Dapper .csproj file for the .NET sandbox project. . . . .                                                                                                          | 138 |
| B.19 | MongoDB pom.xml file for the Java sandbox project. . . . .                                                                                                         | 139 |
| B.20 | Neo4j pom.xml file for the Java sandbox project. . . . .                                                                                                           | 140 |

# Acronyms

**ACP** Agent Client Protocol. [5](#), [56](#), [63](#), [69](#), [97](#), [100](#)

**AI** Artificial Intelligence. [5](#), [63](#), [99](#)

**API** Application Programming Interface. [7](#), [12](#), [13](#), [18](#), [32](#), [38](#), [39](#), [43](#), [45](#), [47](#), [54](#), [63](#), [64](#), [69–72](#), [81](#), [82](#), [89–92](#), [116](#)

**CLI** Command Line Interface. [10](#), [39](#), [48](#), [74](#)

**CLR** Common Language Runtime. [22](#), [23](#)

**CoT** Chain-of-Thought. [33](#), [40](#)

**CPU** Central Processing Unit. [24](#)

**CSV** Comma Separated Values. [116](#)

**EF** Entity Framework. [10](#), [21](#), [23](#), [24](#), [26](#), [52](#), [60](#), [75](#), [83](#), [89](#), [91](#), [111](#), [112](#), [126](#), [128](#), [129](#)

**ETL** Extract, Transform, Load. [11](#), [13](#), [53](#), [60](#), [64](#), [65](#), [74](#), [75](#), [78](#), [79](#), [82](#), [85](#), [86](#), [96](#), [97](#)

**HITL** Human-in-the-Loop. [62](#), [69](#)

**HQL** Hibernate Query Language. [5](#), [10](#), [12](#), [76](#), [99](#)

**HTTP** Hypertext Transfer Protocol. [70](#)

**I/O** Input/Output. [22](#)

**IDE** Integrated Development Environment. [56](#), [62–64](#), [69](#)

**ILP** Integer Linear Programming. [11](#), [21](#), [97](#)

**JIT** Just-In-Time. [23](#), [24](#)

**JPA** Jakarta Persistence API. 71

**JSON** JavaScript Object Notation. 22, 23, 26, 45, 46, 48, 49, 51, 73–76, 78, 85, 86, 93, 115, 117, 127, 135

**JVM** Java Virtual Machine. 27

**LINQ** Language Integrated Query. 5, 10, 12–17, 21, 34, 73, 76, 98, 99, 111, 136

**LLM** Large Language Model. 4, 7–9, 33, 37, 39, 42–44, 53, 54, 70, 80, 87, 88, 93, 98, 100

**MCP** Model Context Protocol. 5, 37–39, 56, 63, 66, 67, 69, 97, 98, 100

**MoE** Mixture-of-Experts. 33, 94

**MQL** MongoDB Query Language. 98

**ODM** Object-Document Mapping. 9, 10, 12, 17, 19, 20, 27, 71, 108

**OGM** Object-Graph Mapping. 9, 10, 12, 17, 19, 20, 27, 108

**ORM** Object-Relational Mapping. 5, 7, 9, 10, 12, 17, 19–21, 24–27, 71, 97, 100, 108

**OS** Operating System. 24

**RAG** Retrieval-Augmented Generation. 38

**RAM** Random Access Memory. 24

**REST** Representational State Transfer. 56, 63, 69, 98

**SDK** Software Development Kit. 48, 69, 70

**SQL** Structured Query Language. 5, 7, 10, 12, 21, 22, 24, 26, 27, 48, 49, 52, 56, 67, 73, 76, 77, 97–99

**SSE** Server-Sent Events. 60, 69

**SSH** Secure Shell. 64

**UI** User Interface. 6, 10, 44

**URL** Uniform Resource Locator. 70

**UTC** Coordinated Universal Time. 49, 126

**WWI** WideWorldImporters. 73, 86

# Appendix A

## Electronic Attachments

The current state of the entire project and its supporting data is attached to this thesis as an archive; for later updates, consult the Universal Object Mapping repository [45]. The deployed system is, at the time of writing, available at <https://uom-demo.vercel.app>, with a mirror at <https://uom.dyn.cloud.e-infra.cz> and <http://uom-assistant.dev> planned. Follow these links in order; if one is unavailable, the next mirrors the same system. The documentation is also available online at <https://uom-demo.vercel.app/docs>, with mirror at <https://uom-cd0f649f.minlify.app>.

The attached archive contains:

- Universal-Object-Mapping/ – the complete project source code [45];
- uom-docs/ – the developer and user documentation (with a repository also available at <https://github.com/corovcam/uom-docs>); and
- experiments/ – the experimental deliverables bundle (section A.2).

### A.1 Complete Execution Traces

Complete, tool-level traces of end-to-end runs – every node visit, model call with its full prompt and completion, tool invocation, and token count – are exported from the LangSmith tracing backend in JSON format. The compact renderings in figures 4.2 and 4.3 are derived from these exports. The full traces are attached in the experiments/traces/ directory.

## A.2 Experimental Deliverables Bundle

The complete experimental record of chapter 8 is provided as a self-contained bundle (`experiments/`), so that every figure, table, and number in that chapter can be inspected and re-derived. Its layout, recorded in an accompanying `README.md`, is as follows.

- `charts/` – every evaluation figure, with the tidy per-repetition table (`per_rep.csv`) and the `summary.md` from which the results tables are drawn.
- `results/` – the derived metrics: per-run aggregates, the CodeBLEU table with its frozen execution-verified references (and their provenance), the per-node token/time attribution, and the external-arm scores.
- `traces/` – the per-run trace [CSVs](#) of the headline campaign and the selected prior-design experiments.
- `prompts/`, `conversations/`, `predictions/` – the exported translation-stage prompts, the external arms’ captured answers and (where the harness exposed it) their [API](#)-reported usage and full transcripts, and the per-arm prediction trees scored by the equivalence checker.
- `reference/`, `aimock-fixtures/` – the frozen harness-form reference translations, and the recorded model interactions that make the analyses replayable offline.

Token counts and prices are reported only where the provider or harness exposed them (and are marked “not reported” otherwise), and the scripts under the repository’s `services/orchestrator/evaluation/scripts/` regenerate every figure from the raw traces.



# Appendix B

## Supplementary Material from the Implementation

This appendix reproduces, verbatim, the concrete code artifacts referenced from the main text: the typed schemas, the per-node prompts, the harness snippets, a complete equivalence-check example, a complete translation input, and the pinned sandbox build configurations. All are taken from the Universal Object Mapping repository [45]; the attached archive as a whole is described in chapter A.

### B.1 Structured-Output and Tool Schemas

The typed interfaces through which model output re-enters the graph state (section 5.5) are Pydantic models, reproduced below with abbreviated field descriptions.

Listing B.1 shows the extraction schema: the code fields accept a list of strings (one per line) so the original indentation survives the provider’s JSON transport, and the error field lets the model report an unfulfillable request as data (section 4.3.1).

```

1 class ExtractionOutput(BaseModel):
2     source_schema_code: list[str] | str | None # source schema, line
3     ↪ by line
4     source_query_code: list[str] | str | None # source queries, line
5     ↪ by line
6     translation_type: TranslationType # SCHEMA / QUERY /
7     ↪ BOTH
8     source_target: FrameworkEnum # identified origin
9     ↪ framework
10    source_target_version: str | None
11    destination_target: FrameworkEnum # identified target
12    ↪ framework
13    destination_target_version: str | None
14    error: str | None = None # set when the request cannot be
15    ↪ fulfilled
16
17    @model_validator(mode="after")
18    def join_lists(self): ... # joins list-form code fields into
19    ↪ strings

```

**Listing B.1** The extraction schema (from `react_agent/graph.py`).

Listing B.2 shows the argument schema of the single state-writing `save_translation` tool that replaced monolithic structured output for the generation stage (section 5.5); the two body fields carry only the region below the harness seam.

```

1 def build_save_translation_tool(translation_type, source_entry,
  ↪ target_entry):
2     # args schema (both fields required, descriptions shortened):
3     #     source_validation_body: str
4     #     "SOURCE-side harness BODY: entity classes, query classes, and
  ↪ the
5     #     entrypoint class named exactly `{source_entry}`. No imports,
  ↪ no
6     #     redeclaration of injected serializer/runtime/factory
  ↪ classes."
7     #     target_validation_body: str
8     #     "TARGET-side harness BODY ... entrypoint named exactly
  ↪ `{target_entry}`."
9     # The tool persists the fields to state via Command(update=...) and
  ↪ is to be
10    # called EXACTLY ONCE at the end of the agent's run.

```

**Listing B.2** The save\_translation tool contract (from react\_agent/translation\_draft.py).

Listing B.3 shows the judge’s structured output and the resume payload of the human-intervention interrupt (section 4.3.7), which deliberately mirror each other so that a human decision re-enters the loop through the same shape as an automated one.

```

1 class EvaluationOutput(BaseModel):
2     decision: Literal["ACCEPT", "REJECT"]
3     explanation: str # mandatory; becomes the repair feedback
4
5 class HumanInterventionResponse(BaseModel):
6     decision: Literal["accept", "reject"]
7     feedback: str # targeted correction pointers on reject

```

**Listing B.3** The judge and human-intervention verdict schemas.

Listing B.4 shows the shared graph state formalized in section 5.2 and definition 1: State extends the narrow InputState (*I*) and OutputState (*O*) with private working fields that never cross the graph boundary. Message channels use the add\_messages reducer.

```

1  @dataclass
2  class InputState:                                # I: what a caller must supply
3      messages: Annotated[Sequence[AnyMessage], add_messages]
4      source_schema_code: str | None
5      source_query_code: str | None
6      translation_type: TranslationType | None      # SCHEMA / QUERY /
        ↪ BOTH
7      source_target: FrameworkEnum | None          # origin framework
8      source_target_version: str | None
9      destination_target: FrameworkEnum | None     # target framework
10     destination_target_version: str | None
11
12  @dataclass
13  class OutputState:                               # O: what a caller receives back
14      translated_schema_code: str | None
15      translated_query_code: str | None
16      source_validation_schema_code: str | None     # runnable harnesses
17      source_validation_harness_code: str | None
18      target_validation_schema_code: str | None
19      target_validation_harness_code: str | None
20      explanation_message: str | None
21      messages: Annotated[Sequence[AnyMessage], add_messages]
22
23  @dataclass
24  class State(InputState, OutputState):             # Sigma: full internal state
25      is_last_step: IsLastStep = False             # managed recursion
        ↪ guard
26      source_validation_entry_type_name: str | None # sandbox entrypoint
        ↪ names
27      target_validation_entry_type_name: str | None
28      source_query_validation_results: QueryValidationResults | None
29      target_query_validation_results: QueryValidationResults | None
30      query_equivalence_deep_diffs: dict[str, QueryEquivalenceDeepDiff] |
        ↪ None
31      schema_context: str = ""                     # schema summary, kept out of
        ↪ messages
32      translation_messages: Annotated[Sequence[AnyMessage], add_messages]
        ↪ # repair channel
33      extraction_loop_count: int = 0               # bounded-loop counters
34      translation_loop_count: int = 0
35      ui_messages: Annotated[Sequence[AnyMessage], add_messages]
        ↪ # UI-only channel

```

**Listing B.4** The shared graph state (from `react_agent/state.py`).

## B.2 Per-Node System Prompts

The prompts below are reproduced from `react_agent/prompts.py` (chapter 5). Placeholders in braces are interpolated at run time.

### Input extraction

```
You are an information extractor. Your goal is to extract source schema
↳ code, source query code, origin framework/version, destination
↳ framework/version, and translation type from the user's messages.

Allowed origin frameworks: {origin_frameworks}
Allowed destination frameworks: {destination_frameworks}

Extraction rules:
1. You must identify the origin framework and the destination framework
↳ from the user's messages.
2. You must identify IF the code is a schema
↳ (entities/models+context/sessions/configs, etc.) or a query for the
↳ given origin framework, or both.
3. If some data has already been extracted, you must use it as is and
↳ only extract the missing data.
4. Output specific structured outputs exactly as requested. Do not
↳ provide markdown wrapping if native tools capture the output
↳ natively.
5. Keep source_schema_code and source_query_code as raw code snippets
↳ when available.
6. Preserve the original formatting (including indentation and line
↳ breaks) of the extracted code snippets.
```

**Listing B.5** System prompt of the input-extraction stage.

### Schema inspection

```
You are a database schema inspector. Your goal is to examine source and
↳ target database schemas to provide context for code translation.

You have access to database tools that can:
- List collections/tables/node labels/relationship types in databases
- Inspect schema structures (columns, fields, nodes,
↳ relationships/edges)
- Sample documents/rows/nodes/edges to understand data shapes
```

```

Your task:
1. Inspect the SOURCE database schema relevant to the code being
  ↳ translated.
  - For MS SQL: use the prebuilt mssql tools to list tables, describe
    ↳ columns, and sample rows.
  - For MongoDB: use mongodb tools to list collections, inspect
    ↳ document schemas, and sample documents.
  - For Neo4j: use the prebuilt neo4j tools to list node labels,
    ↳ relationship types, and sample nodes/edges.
2. Inspect the TARGET database schema if applicable (e.g., if
  ↳ translating from SQL to MongoDB, inspect what MongoDB collections
  ↳ exist).
3. Focus on ONLY the entities/models/relationships and their
  ↳ relationships relevant to the code being translated.
4. Return a concise but complete summary of the relevant source and
  ↳ target schemas. Do not include a preamble or postamble. The summary
  ↳ will become part of the system prompt for another LLM agent that
  ↳ will perform the actual code translation.

System time: {system_time}

```

**Listing B.6** System prompt of the schema-inspection stage.

## Translation generation

The generation prompt is assembled dynamically per framework pair (section 5.4); the invariant contract is reproduced below, with the interpolated blocks – the pinned manifests, the worked mapping examples, and the harness body-structure references – elided for space.

```

You are a Universal Object Mapping architect. Your goal is to aid in
  ↳ translating database schema structures and query logic between
  ↳ diverse languages and frameworks.

Source Framework: {source_target}
Destination Framework: {destination_target}

Core translation contract:
1. Identify whether the user input contains schema code, query code, or
  ↳ both.
2. Translate only what is requested by translation type.

```

3. Preserve behavior, field intent, and query semantics.
4. Keep translated query methods semantically equivalent to the source query method. Do not introduce synthetic validator parameters (for example sortByField/ascending) unless they already exist in source query code.
5. Keep schema code and query code separated.
6. CRITICAL - translate ONLY what the user provided. Translate exclusively the entities and fields present in the user's <source\_schema\_code> and the queries in <source\_query\_code>. NEVER introduce an entity, field, or query that is not in the user's input. The examples below demonstrate STRUCTURE ONLY [...]
7. You finish by calling the single `save\_translation` tool EXACTLY ONCE with every required field filled. There is no separate JSON output. [...]
8. The `\*\_validation\_body` fields are the RUNNABLE harness body ONLY:
  - start directly at the schema/entity declarations, then the query classes, then the entrypoint class with `main`/`Main` that validates each entity and runs each query. The fixed boilerplate - `import`/`using`/`package`/`namespace` lines, the JSON serializer, the runtime-support and DB template-factory classes - is INJECTED FOR YOU around your body. Do NOT write those lines and do NOT redeclare those classes.
  - source\_validation\_body MUST declare the source entrypoint class
    - named exactly `{source\_entry}`.
  - target\_validation\_body MUST declare the target entrypoint class
    - named exactly `{target\_entry}`.
9. All code must be properly indented with real line breaks. DO NOT
  - wrap field values in XML tags or markdown code fences. DO NOT use comments or placeholders in code - it WILL be executed. Never save null or empty values.

#### Framework rules:

1. For Java schema classes, avoid public access modifier unless
  - explicitly required.
2. For Spring Data MongoDB queries, use MongoTemplate with
  - Query/Criteria API.
3. For Spring Data Neo4j queries, use Neo4jTemplate and Cypher-DSL
  - (Statement-based), not raw string concatenation.
4. Keep translated query method shape close to source query method
  - shape. [...]

#### Additional rules:

```

1. You do NOT run validators or compile code. After you call
  ↳ `save_translation`, the translated code is assembled with the
  ↳ canonical prelude and validated automatically by a downstream
  ↳ pipeline (compile + run on both sides, then equivalence). If it
  ↳ fails, you will be re-invoked with concrete feedback to fix and
  ↳ re-save. [...]
2. You have research tools - use them to get the exact API right before
  ↳ saving:
  - Use `search_spring_docs` to query the Spring documentation [...]
  - Use `microsoft_docs_search`, `microsoft_code_sample_search`, and
    ↳ `microsoft_docs_fetch` for Microsoft documentation and code
    ↳ samples.
  - Use `search` to query the web [...]

--- Validation setup configuration ---
[pinned .csproj of the source framework and pom.xml of the target
 ↳ framework]

--- TARGET-LANGUAGE MAPPING REFERENCE ---
[worked input/output mapping examples for the framework pair, structure
 ↳ only]

--- VALIDATION BODY STRUCTURE REFERENCE ---
[the region of each canonical harness snippet below the seam marker]

```

**Listing B.7** Partial system prompt of the translation-generation stage.

## Finalization

```

You are a Universal Object Mapping finalizer.

A translation has just been VALIDATED end-to-end: the harness you are
 ↳ given compiled, executed against the database, and passed semantic
 ↳ equivalence against the source. The translation is already CORRECT.
 ↳ Your job is NOT to translate. Your job is to EXTRACT the clean,
 ↳ production-ready target code from the validated harness, removing
 ↳ only the validation scaffolding.

Hard rules:

```



1. SOURCE OF TRUTH is the validated harness. Copy the entity
  - definitions and the query logic VERBATIM from it. Do NOT
  - re-translate, rename, reorder fields, or change any query
  - predicate, filter, projection, sort, or aggregation - those exact
  - semantics are what equivalence verified.
2. REMOVE only validation-only scaffolding:
  - the JSON serializer class, the runtime-support class, the
    - template/context/db factory class, logging setup, and their
    - imports.
  - the entrypoint class's main/Main method, validate\*/ValidateEntity
    - methods, and all results-collection / JSON-writing / env-path
    - code.
  - per-query validation helpers (harness()/RunQuery,
    - count/firstSample/lastSample, and deterministic ORDER BY / Sort
    - / limit / skip that exist ONLY to make validation deterministic
    - and were not in the user's original query).
  - JSON-serialization annotations that are not part of the object
    - mapping (e.g. [JsonPropertyName], @JsonIgnoreProperties,
    - @ReadOnlyProperty added for validation). KEEP genuine ORM
    - mapping annotations: @Document, @Field, @Id, @Node, @Property,
    - @DocumentReference, [Table], [Key], [ForeignKey], [Column], etc.
3. KEEP the genuine production code: the entity/model classes with
  - their ORM mapping, and the production query method(s) carrying the
  - exact validated predicate. Place the queries in a single clean
  - query class with one method per source query, named to match the
  - source (query1..queryN / Query1..QueryN), each returning the
  - production result (not the validation metadata map).
4. Include the minimal imports needed for the production code to
  - compile. [...]
5. Emit CANONICAL, STABLE structure - this code is compared across runs
  - with a CodeBleu (AST + data-flow) metric, so structure must be
  - deterministic: entities in the same declaration order as the
  - validated harness (one class per entity), then the query class with
  - methods in source order. Consistent 4-space indentation. No
  - comments, no markdown code fences, no prose, no placeholders.

**Listing B.8** System prompt of the finalization stage.

The judge's prompt (section 4.3.6) is a short instruction constructed inline: it wraps the recent validation outputs in a <validation\_results> block and asks, "Is the translation logically equivalent and syntactically valid? Provide your reasoning and output ACCEPT or REJECT," with the response bound to the EvaluationOutput schema of listing B.3.

## B.3 Contextual Validation Harness Snippets

Each framework contributes two contextual snippets – a schema-validation and a query-execution entry point – split by the marker `// --- Schema and Related Settings` into the invariant prelude the harness assembler injects and the body region whose *structure* the model imitates (section 5.6). Table B.1 lists the files and their deterministic entry-point class names.

| Framework           | Schema validation                    | Query execution           |
|---------------------|--------------------------------------|---------------------------|
| EF Core             | EFCoreSchemaValidationEntrypoint     | EFCoreQueryEntrypoint     |
| Dapper              | DapperSchemaValidationEntrypoint     | DapperQueryEntrypoint     |
| NHibernate          | NHibernateSchemaValidationEntrypoint | NHibernateQueryEntrypoint |
| Spring Data MongoDB | MongoSchemaValidationEntrypoint      | MongoQueryEntrypoint      |
| Spring Data Neo4j   | Neo4jSchemaValidationEntrypoint      | Neo4jQueryEntrypoint      |

**Table B.1** Contextual snippet files and entry-point class names per framework.

Listing B.9 reproduces the core of the invariant .NET prelude: the serializer configuration that realizes the serialization contract of section 5.7. The Java prelude enforces the same rules through Jackson (ISO-8601 UTC timestamps at millisecond precision, `BigDecimal.setScale(3, HALF_UP)`, alphabetical property and map-key ordering).

```

1 public static readonly JsonSerializerOptions Options = new()
2 {
3     PropertyNamingPolicy = new CustomCamelCaseNamingPolicy(),    //
4     ↪ CustomerID -> customerId
5     DictionaryKeyPolicy = new CustomCamelCaseNamingPolicy(),
6     DefaultIgnoreCondition = JsonIgnoreCondition.Never,
7     ReferenceHandler = ReferenceHandler.IgnoreCycles,
8     TypeInfoResolver = new DefaultJsonTypeInfoResolver
9     { Modifiers = { AlphabetizeProperties() } },                //
10    ↪ fixed field order
11    Converters = {
12        new StrictIsoDateTimeConverter(),                        //
13        ↪ yyyy-MM-dd'T'HH:mm:ss.fff'Z', UTC
14        new StrictIsoDateTimeOffsetConverter(),
15        new FixedDecimalConverter(),                             // "0.000" fixed
16        ↪ 3-decimal scale
17        new FixedDoubleConverter(),
18        new JsonStringEnumConverter(new CustomCamelCaseNamingPolicy())
19    }
20 };

```

**Listing B.9** The invariant .NET serializer configuration (from `EFCoreQueryEntrypoint.cs`); the Java prelude mirrors every rule in Jackson.

Listing B.10 shows the per-query digest producer that every query harness uses: each query is reduced to its count and its two boundary samples under a deterministic ordering, keyed query1 . . queryN in the results JSON that the equivalence check of section 5.7 consumes.

```

1 private static object RunQuery<T>(Func<IQueryable<T>> q,
2                                 Func<T, object>? orderBySelector =
3                                 ↪ null)
4 {
5     var query = q();
6     var count = query.Count();
7     return new {
8         sqlString = query.ToQueryString(), count,
9         firstSample = count > 0
10            ? (orderBySelector != null
11              ? q().OrderBy(orderBySelector).FirstOrDefault()
12              : query.FirstOrDefault()) : default,
13         lastSample = count > 1
14            ? (orderBySelector != null
15              ? q().OrderByDescending(orderBySelector).FirstOrDefault()
16              ↪ t()
17              : query.LastOrDefault()) : default };
18 }

```

**Listing B.10** The count/boundary-sample digest producer of the EF Core query harness; each framework implements the same contract in its own idiom.

## B.4 A Complete Equivalence-Check Example

The listings below reproduce the source-side and target-side result digests and the equivalence report of the worked example of section 5.7.4 (Entity Framework Core → Spring Data MongoDB over the WideWorldImporters OrderLines data), taken from the integration-test fixtures of the repository [45].

```

{
  "query1": {
    "count": 1883,
    "firstSample": {
      "description": "Tape dispenser (Red)",
      "lastEditedBy": 19,
      "lastEditedWhen": "2014-12-29T11:00:00.000Z",
      "orderId": 10417,
      "orderLineId": 33036,
      "packageTypeId": 7,
      "pickedQuantity": 80,
      "pickingCompletedWhen": "2014-12-29T11:00:00.000Z",

```

```

    "quantity": 80,
    "stockItemId": 204,
    "taxRate": 15,
    "unitPrice": 32
  },
  "lastSample": {
    "description": "Air cushion film 200mmx200mm 325m",
    "lastEditedBy": 16,
    "lastEditedWhen": "2014-12-30T11:00:00.000Z",
    "orderId": 40554,
    "orderLineId": 128234,
    "packageTypeId": 7,
    "pickedQuantity": 10,
    "pickingCompletedWhen": "2014-12-30T11:00:00.000Z",
    "quantity": 10,
    "stockItemId": 214,
    "taxRate": 15,
    "unitPrice": 90
  },
  "sqlString": "DECLARE @from datetime2 =
    ↳ '2014-12-20T00:00:00.0000000';\nDECLARE @to datetime2 =
    ↳ '2014-12-31T00:00:00.0000000';\n\nSELECT [o].[OrderLineID],
    ↳ [o].[Description], [o].[LastEditedBy], [o].[LastEditedWhen],
    ↳ [o].[OrderID], [o].[PackageTypeID], [o].[PickedQuantity],
    ↳ [o].[PickingCompletedWhen], [o].[Quantity], [o].[StockItemID],
    ↳ [o].[TaxRate], [o].[UnitPrice]\nFROM [Sales].[OrderLines] AS
    ↳ [o]\nWHERE [o].[PickingCompletedWhen] >= @from AND
    ↳ [o].[PickingCompletedWhen] <= @to"
}
}

```

**Listing B.11** The source-side (.NET, EF Core) result digest of the worked example.

```

{
  "query1": {
    "count": 1883,
    "firstSample": {
      "description": "Tape dispenser (Red)",
      "id": "69d7d2bf7d34d59f1cfbbaf1",
      "lastEditedBy": 19,
      "lastEditedWhen": "2014-12-29T12:00:00.000Z",
      "orderId": 10417,
      "orderLineId": 33036,
    }
  }
}

```

```

        "packageTypeId": 7,
        "pickedQuantity": 80,
        "pickingCompletedWhen": "2014-12-29T12:00:00.000Z",
        "quantity": 80,
        "stockItemId": 204,
        "taxRate": 15,
        "unitPrice": 32
    },
    "lastSample": {
        "description": "Air cushion film 200mmx200mm 325m",
        "id": "69d7d2c77d34d59f1cfd429d",
        "lastEditedBy": 16,
        "lastEditedWhen": "2014-12-30T12:00:00.000Z",
        "orderId": 40554,
        "orderLineId": 128234,
        "packageTypeId": 7,
        "pickedQuantity": 10,
        "pickingCompletedWhen": "2014-12-30T12:00:00.000Z",
        "quantity": 10,
        "stockItemId": 214,
        "taxRate": 15,
        "unitPrice": 90
    },
    "mongoQuery": {
        "collection": "orderLines",
        "filter": {
            "pickingCompletedWhen": {
                "$gte": "2014-12-20T00:00:00.000Z",
                "$lte": "2014-12-31T00:00:00.000Z"
            }
        }
    }
}

```

**Listing B.12** The target-side (Java, Spring Data MongoDB) result digest of the worked example.

```

{
  "query1": {
    "status": "Differences Found",
    "diffs": {
      "deepdiff_mapping": {

```

```

    "old": ".NET Entity Framework Core",
    "new": "Java Spring Data MongoDB"
  },
  "countDiff": {},
  "firstSampleDiff": {
    "dictionary_item_added": [ "root['id']" ],
    "values_changed": {
      "root['lastEditedWhen']": {
        "new_value": "2014-12-29T12:00:00.000Z",
        "old_value": "2014-12-29T11:00:00.000Z"
      },
      "root['pickingCompletedWhen']": {
        "new_value": "2014-12-29T12:00:00.000Z",
        "old_value": "2014-12-29T11:00:00.000Z"
      }
    },
    "deep_distance": 0.057692307692307696
  },
  "lastSampleDiff": {
    "dictionary_item_added": [ "root['id']" ],
    "values_changed": {
      "root['lastEditedWhen']": {
        "new_value": "2014-12-30T12:00:00.000Z",
        "old_value": "2014-12-30T11:00:00.000Z"
      },
      "root['pickingCompletedWhen']": {
        "new_value": "2014-12-30T12:00:00.000Z",
        "old_value": "2014-12-30T11:00:00.000Z"
      }
    },
    "deep_distance": 0.057692307692307696
  }
}
}
}
}
}

```

**Listing B.13** The equivalence report of the worked example, as computed by the parameters of section 5.7.3 and rendered into the isolated message channel for the judge.

## B.5 A Complete Translation Input

This section reproduces one complete user-authored input: the Entity Framework Core source of the Entity Framework Core → Spring Data MongoDB pair used throughout

chapter 8 (section 8.2). It consists of the entity classes with their mapping configuration (listing B.14) and the fifteen source queries (listing B.15). The NHibernate and Dapper inputs express the same eight entities and fifteen logical queries in their own idioms.

```

1  [Table("OrderLines", Schema = "Sales")]
2  public class OrderLine
3  {
4      [Key] public int OrderLineID { get; set; }
5      [ForeignKey(nameof(Order))] public int OrderID { get; set; }
6      public int StockItemID { get; set; }
7      public required string Description { get; set; }
8      public int Quantity { get; set; }
9      public decimal? UnitPrice { get; set; }
10     public decimal TaxRate { get; set; }
11     public int PickedQuantity { get; set; }
12     public DateTime? PickingCompletedWhen { get; set; }
13     public int LastEditedBy { get; set; }
14     public DateTime LastEditedWhen { get; set; }
15 }
16
17 [Table("Orders", Schema = "Sales")]
18 public class Order
19 {
20     [Key] public int OrderID { get; set; }
21     public int CustomerID { get; set; }
22     public int? BackorderOrderID { get; set; }
23     public DateTime OrderDate { get; set; }
24     public DateTime ExpectedDeliveryDate { get; set; }
25     public string? CustomerPurchaseOrderNumber { get; set; }
26     public bool IsUndersupplyBackordered { get; set; }
27     public string? Comments { get; set; }
28     public string? DeliveryInstructions { get; set; }
29     public string? InternalComments { get; set; }
30     public DateTime? PickingCompletedWhen { get; set; }
31     public DateTime LastEditedWhen { get; set; }
32     public List<OrderLine> OrderLines { get; set; } = [];
33 }
34
35 [Table("People", Schema = "Application")]
36 public class Person
37 {
38     [Key] public int PersonID { get; set; }
39     public required string FullName { get; set; }
40     public required string PreferredName { get; set; }

```



```

41     public string? EmailAddress { get; set; }
42     public CustomFields? CustomFields { get; set; }
43     public List<string>? OtherLanguages { get; set; }
44 }
45
46 public class CustomFields
47 {
48     public List<string>? OtherLanguages { get; set; }
49     public DateTime? HireDate { get; set; }
50     public string? Title { get; set; }
51 }
52
53 [Table("Suppliers", Schema = "Purchasing")]
54 public class Supplier
55 {
56     [Key] public int SupplierID { get; set; }
57     public required string SupplierName { get; set; }
58     public string? SupplierReference { get; set; }
59     public int PaymentDays { get; set; }
60     public string? PhoneNumber { get; set; }
61     public string? FaxNumber { get; set; }
62     public string? WebsiteURL { get; set; }
63     public string? BankAccountName { get; set; }
64     public string? BankAccountBranch { get; set; }
65     public string? BankAccountCode { get; set; }
66     public string? BankAccountNumber { get; set; }
67     public string? BankInternationalCode { get; set; }
68 }
69
70 [Table("CustomerTransactions", Schema = "Sales")]
71 public class CustomerTransaction
72 {
73     [Key] public int CustomerTransactionID { get; set; }
74     public int CustomerID { get; set; }
75     public DateTime TransactionDate { get; set; }
76     public decimal TransactionAmount { get; set; }
77     public decimal OutstandingBalance { get; set; }
78     public bool IsFinalized { get; set; }
79 }
80
81 [Table("PurchaseOrders", Schema = "Purchasing")]
82 public class PurchaseOrder
83 {
84     [Key] public int PurchaseOrderID { get; set; }

```

```

85     public int SupplierID { get; set; }
86     public DateTime OrderDate { get; set; }
87     public DateTime ExpectedDeliveryDate { get; set; }
88     public string? SupplierReference { get; set; }
89     public bool IsOrderFinalized { get; set; }
90 }
91
92 [Table("StockItems", Schema = "Warehouse")]
93 public class StockItem
94 {
95     [Key] public int StockItemID { get; set; }
96     public required string StockItemName { get; set; }
97     public int SupplierID { get; set; }
98     public int QuantityPerOuter { get; set; }
99     public int LeadTimeDays { get; set; }
100    public bool IsChillerStock { get; set; }
101    public decimal? UnitPrice { get; set; }
102    public decimal RecommendedRetailPrice { get; set; }
103 }
104
105 [Table("StockItemStockGroups", Schema = "Warehouse")]
106 public class StockItemStockGroup
107 {
108     [Key] public int StockItemStockGroupID { get; set; }
109     public int StockItemID { get; set; }
110     public int StockGroupID { get; set; }
111 }
112
113 public class SandboxDbContext(DbContextOptions<SandboxDbContext>
114     ↪ options) : DbContext(options)
115 {
116     public DbSet<Order> Orders => Set<Order>();
117     public DbSet<OrderLine> OrderLines => Set<OrderLine>();
118     public DbSet<Person> People => Set<Person>();
119     public DbSet<Supplier> Suppliers => Set<Supplier>();
120     public DbSet<CustomerTransaction> CustomerTransactions =>
121         ↪ Set<CustomerTransaction>();
122     public DbSet<PurchaseOrder> PurchaseOrders => Set<PurchaseOrder>();
123     public DbSet<StockItem> StockItems => Set<StockItem>();
124     public DbSet<StockItemStockGroup> StockItemStockGroups =>
125         ↪ Set<StockItemStockGroup>();
126
127     protected override void OnModelCreating(ModelBuilder modelBuilder)
128     {

```

```

126     modelBuilder.Entity<Person>().OwnsOne(p => p.CustomFields, cb => {
127         ↪ cb.ToJson(); });
127     base.OnModelCreating(modelBuilder);
128 }
129 }

```

**Listing B.14** The complete Entity Framework Core source schema of the running evaluation input: eight persisted entities, an owned `JSON` type (`CustomFields`, mapped with `ToJson()`), and the `DbContext`.

```

1  public static IQueryable<OrderLine> Query1(SandboxDbContext ctx)
2      => ctx.OrderLines.Where(ol => ol.OrderID == 26866);
3
4  public static IQueryable<OrderLine> Query2(SandboxDbContext ctx)
5      => ctx.OrderLines.Where(ol => ol.UnitPrice == 25m);
6
7  public static IQueryable<OrderLine> Query3(SandboxDbContext ctx)
8  {
9      var from = new DateTime(2014, 12, 20); var to = new DateTime(2014,
10         ↪ 12, 31);
11      return ctx.OrderLines.Where(ol => ol.PickingCompletedWhen >= from &&
12         ↪ ol.PickingCompletedWhen <= to);
13  }
14
15 public static IQueryable<OrderLine> Query4(SandboxDbContext ctx)
16 {
17     var orderIds = new[] { 1, 10, 100, 1000, 10000 };
18     return ctx.OrderLines.Where(ol => orderIds.Contains(ol.OrderID));
19 }
20
21 public static IQueryable<OrderLine> Query5(SandboxDbContext ctx)
22     => ctx.OrderLines.Where(ol => ol.Description.Contains("C++"));
23
24 public static IQueryable<OrderLine> Query6(SandboxDbContext ctx)
25     => ctx.OrderLines.OrderBy(ol => ol.OrderLineID).Skip(1000).Take(50);
26
27 public static Dictionary<decimal, int> Query7(SandboxDbContext ctx)
28     => ctx.OrderLines
29         .GroupBy(ol => ol.TaxRate)
30         .Select(g => new { TaxRate = g.Key, Count = g.Count() })
31         .OrderByDescending(x => x.Count)
32         .ToDictionary(x => x.TaxRate, x => x.Count);

```

```

32 public static decimal? Query8(SandboxDbContext ctx)
33     => ctx.OrderLines.Max(ol => ol.UnitPrice);
34
35 public static decimal? Query9(SandboxDbContext ctx)
36     => ctx.OrderLines.Sum(ol => ol.Quantity * ol.UnitPrice);
37
38 public static Order? Query10(SandboxDbContext ctx)
39     => ctx.Orders.Include(o => o.OrderLines).SingleOrDefault(o =>
40         o.OrderID == 530);
41
42 public static IQueryable<Order> Query11(SandboxDbContext ctx)
43     => ctx.Orders.OrderBy(o => o.ExpectedDeliveryDate).Take(1000);
44
45 public static IQueryable<string?> Query12(SandboxDbContext ctx)
46     => ctx.Orders.Select(o => o.CustomerPurchaseOrderNumber).Distinct();
47
48 public static IQueryable<Person> Query13(SandboxDbContext ctx)
49     => ctx.People.Where(p => p.CustomFields!.Title == "Team
50         Member").OrderBy(p => p.PersonID);
51
52 public static IQueryable<Person> Query14(SandboxDbContext ctx)
53     => ctx.People.Where(p =>
54         p.OtherLanguages!.Contains("Slovak")).OrderBy(p => p.PersonID);
55
56 public static List<int> Query15(SandboxDbContext ctx)
57 {
58     var first = ctx.Suppliers.Where(s => s.SupplierID < 5).Select(s =>
59         s.SupplierID).ToList();
60     var last = ctx.Suppliers.Where(s => s.SupplierID >= 5 &&
61         s.SupplierID <= 10).Select(s => s.SupplierID).ToList();
62     return first.Union(last).OrderBy(s => s).ToList();
63 }

```

**Listing B.15** The complete fifteen-query source input in Entity Framework Core (LINQ), lightly reformatted for space. These are the queries summarized in table 8.1.

## B.6 Sandbox Configuration Files

The pinned build-tool configurations against which the model generates and the sandboxes compile (section 5.6.1). Each framework has one manifest; the model is given it as context and the sandbox builds against it verbatim.

```

1 <Project Sdk="Microsoft.NET.Sdk">
2   <PropertyGroup>
3     <OutputType>Exe</OutputType>
4     <TargetFramework>net10.0</TargetFramework>
5     <RootNamespace>efcore_sandbox</RootNamespace>
6     <ImplicitUsings>enable</ImplicitUsings>
7     <Nullable>enable</Nullable>
8   </PropertyGroup>
9   <ItemGroup>
10    <PackageReference Include="Microsoft.EntityFrameworkCore.SqlServer"
11      ↪ Version="10.0.7" />
12    <PackageReference Include="Microsoft.Data.SqlClient" Version="7.0.1" />
13  </ItemGroup>
</Project>

```

**Listing B.16** Entity Framework Core 10 .csproj file for the .NET sandbox project.

```

1 <Project Sdk="Microsoft.NET.Sdk">
2   <PropertyGroup>
3     <OutputType>Exe</OutputType>
4     <TargetFramework>net10.0</TargetFramework>
5     <RootNamespace>nhibernate_sandbox</RootNamespace>
6     <ImplicitUsings>enable</ImplicitUsings>
7     <Nullable>enable</Nullable>
8   </PropertyGroup>
9   <ItemGroup>
10    <PackageReference Include="Microsoft.Data.SqlClient" Version="7.0.1" />
11    <PackageReference Include="NHibernate" Version="5.5.2" />
12  </ItemGroup>
13 </Project>

```

**Listing B.17** NHibernate .csproj file for the .NET sandbox project.

```
1 <Project Sdk="Microsoft.NET.Sdk">
2   <PropertyGroup>
3     <OutputType>Exe</OutputType>
4     <TargetFramework>net10.0</TargetFramework>
5     <RootNamespace>dapper_sandbox</RootNamespace>
6     <ImplicitUsings>enable</ImplicitUsings>
7     <Nullable>enable</Nullable>
8   </PropertyGroup>
9   <ItemGroup>
10    <PackageReference Include="Microsoft.Data.SqlClient" Version="7.0.1" />
11    <PackageReference Include="Dapper" Version="2.1.66" />
12  </ItemGroup>
13 </Project>
```

**Listing B.18** Dapper .csproj file for the .NET sandbox project.

```

1 <?xml version="1.0" encoding="UTF-8"?>
2 <project xmlns="http://maven.apache.org/POM/4.0.0"
3       xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
4       xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
5         ↪ https://maven.apache.org/xsd/maven-4.0.0.xsd">
6   <modelVersion>4.0.0</modelVersion>
7   <parent>
8     <groupId>org.springframework.boot</groupId>
9     <artifactId>spring-boot-starter-parent</artifactId>
10    <version>4.0.3</version>
11    <relativePath/>
12  </parent>
13  <groupId>uom.services</groupId>
14  <artifactId>mongo_sandbox</artifactId>
15  <version>1.0-SNAPSHOT</version>
16  <description>
17    Minimal Maven project for MongoDB sandbox compilation of
18    ↪ LLM-generated Java code.
19  </description>
20  <properties>
21    <java.version>25</java.version>
22    <maven.compiler.source>25</maven.compiler.source>
23    <maven.compiler.target>25</maven.compiler.target>
24    <project.build.sourceEncoding>UTF-8</project.build.sourceEncoding>
25  </properties>
26  <dependencies>
27    <dependency>
28      <groupId>org.springframework.boot</groupId>
29      <artifactId>spring-boot-starter-data-mongodb</artifactId>
30    </dependency>
31    <dependency>
32      <groupId>org.springframework.boot</groupId>
33      <artifactId>spring-boot-starter-json</artifactId>
34    </dependency>
35  </dependencies>
36  <build>
37    <plugins>
38      <plugin>
39        <groupId>org.codehaus.mojo</groupId>
40        <artifactId>exec-maven-plugin</artifactId>
41        <version>3.6.3</version>
42      </plugin>
43    </plugins>
44  </build>
45 </project>

```

**Listing B.19** MongoDB pom.xml file for the Java sandbox project.

```

1  <?xml version="1.0" encoding="UTF-8"?>
2  <project xmlns="http://maven.apache.org/POM/4.0.0"
3         xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
4         xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
5                               ↪ https://maven.apache.org/xsd/maven-4.0.0.xsd">
6     <modelVersion>4.0.0</modelVersion>
7     <parent>
8         <groupId>org.springframework.boot</groupId>
9         <artifactId>spring-boot-starter-parent</artifactId>
10        <version>4.0.3</version>
11        <relativePath/>
12    </parent>
13    <groupId>uom.services</groupId>
14    <artifactId>neo4j_sandbox</artifactId>
15    <version>1.0-SNAPSHOT</version>
16    <description>
17        Minimal Maven project for Neo4j sandbox compilation of LLM-generated
18        ↪ Java code.
19    </description>
20    <properties>
21        <java.version>25</java.version>
22        <maven.compiler.source>25</maven.compiler.source>
23        <maven.compiler.target>25</maven.compiler.target>
24        <project.build.sourceEncoding>UTF-8</project.build.sourceEncoding>
25    </properties>
26    <dependencies>
27        <dependency>
28            <groupId>org.springframework.boot</groupId>
29            <artifactId>spring-boot-starter-data-neo4j</artifactId>
30        </dependency>
31        <dependency>
32            <groupId>org.springframework.boot</groupId>
33            <artifactId>spring-boot-starter-json</artifactId>
34        </dependency>
35    </dependencies>
36    <build>
37        <plugins>
38            <plugin>
39                <groupId>org.codehaus.mojo</groupId>
40                <artifactId>exec-maven-plugin</artifactId>
41                <version>3.6.3</version>
42            </plugin>
43        </plugins>
44    </build>
45 </project>

```

**Listing B.20** Neo4j pom.xml file for the Java sandbox project.